

# **Jaipur Engineering College and Research Center, Jaipur**

Department of Information Technology

(Rajasthan Technical University, KOTA)



# **LAB MANUAL**

**Lab Name** : SOFTWARE TESTING AND VALIDATION LAB

**Lab Code** : 8IT4-22

**Branch** : Information Technology

**Year** : 4<sup>th</sup> Year/ 8<sup>th</sup> Sem

## INDEX

| S.NO | CONTENTS                    |                                                                                                                                                                                  | CO's | Page No. |
|------|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|----------|
| 1    | VISION/MISION               |                                                                                                                                                                                  |      | 3        |
| 2.   | PEO                         |                                                                                                                                                                                  |      | 4        |
| 3.   | POS                         |                                                                                                                                                                                  |      | 4        |
| 4.   | COS                         |                                                                                                                                                                                  |      | 6        |
| 5.   | MAPPING OF CO & PO          |                                                                                                                                                                                  |      | 6        |
| 6.   | SYLLABUS                    |                                                                                                                                                                                  |      | 7        |
| 7.   | BOOKS                       |                                                                                                                                                                                  |      | 8        |
| 8.   | INSTRUCTIONAL METHODS       |                                                                                                                                                                                  |      | 9        |
| 9.   | LEARNING MATERIALS          |                                                                                                                                                                                  |      | 9        |
| 10.  | ASSESSMENT OF OUTCOMES      |                                                                                                                                                                                  |      | 9        |
| 11.  | TESTING TOOLS USED IN LAB   |                                                                                                                                                                                  |      | 12       |
| 12.  | Software Testing            | i)Manual Testing                                                                                                                                                                 | 1    | 12       |
|      |                             | ii)Automation Testing                                                                                                                                                            |      |          |
| 13.  | Software Testing            | Methods & Documentation                                                                                                                                                          | 1    | 15       |
|      |                             | Viva Questions                                                                                                                                                                   |      |          |
| 14.  | Manual Testing Experiments- | Exp1. Write a c program to demonstrate the working of the fallowing constructs:<br>i) do...while<br>ii) while...do<br>iii) if ...else<br>iv)switch<br>v) for Loops in C language | 1    | 29       |
|      |                             | Exp2. A program written in c language for matrix multiplication fails —Introspect the causes for its failure and write down the possible reasons for its failure.                | 1    | 36       |
| 15.  | I JaBUTi                    | Manual and Installation                                                                                                                                                          |      | 41       |
|      |                             | Exp3. A program written in java to compute the factorial of a given integer number study its test cases import the test cases from Junit .                                       | 1,2  | 63       |
|      |                             | Exp 4. Write the program and study of their possible test cases programs to find<br>a)FindLast<br>b)LastZero<br>c)CountPositive                                                  | 1,2  | 65       |

|     |             |                                                                                                                                                                                                                                                                                                                                                     |     |     |
|-----|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|-----|
| 16. |             | d)NumZero<br>e)OddOrPos                                                                                                                                                                                                                                                                                                                             |     |     |
|     |             | Exp 5. Write a program to study the sorting and draw a control flow graph                                                                                                                                                                                                                                                                           | 1,2 | 71  |
|     |             | Exp 6. Create a test set for the following application:<br>The software read three integer values that represents the edges of a triangle.<br>For those values, the software should write if the triangle is isosceles, scalene, or equilateral. The input values should obey a condition: the sum of two edges must be bigger than the third edge. | 1,2 | 74  |
|     |             | Viva Questions                                                                                                                                                                                                                                                                                                                                      |     |     |
|     | II JMETER   | Manual and Installation                                                                                                                                                                                                                                                                                                                             |     | 78  |
|     |             | Exp 7. Analyse the performance of amazon website using JMeter.                                                                                                                                                                                                                                                                                      | 2   | 82  |
|     |             | Exp 8. Analyse the performance of railway reservation website using JMeter.                                                                                                                                                                                                                                                                         | 2   | 90  |
|     |             | Viva Questions                                                                                                                                                                                                                                                                                                                                      |     |     |
|     | III JUMBLE  | Manual and Installation                                                                                                                                                                                                                                                                                                                             |     | 100 |
|     |             | Exp 9. Write a program in java that detects the first occurrence of a duplicate and returns the value to the calling function analyse its mutation score and also try it using the Jumble tool.                                                                                                                                                     | 1,2 | 101 |
|     | IV ECLEMMMA | Manual and Installation                                                                                                                                                                                                                                                                                                                             |     | 105 |
|     |             | Exp 10. Calculate the coverage analysis of fabonacci series using eclemma tool                                                                                                                                                                                                                                                                      | 1,2 | 106 |
|     |             | Viva Questions                                                                                                                                                                                                                                                                                                                                      |     |     |
|     | V SELENIUM  | Manual and Installation                                                                                                                                                                                                                                                                                                                             |     | 113 |
|     |             | Exp 11. Generate Test sequences and validate using Selenium tool for calculator website.                                                                                                                                                                                                                                                            | 1,2 | 130 |
|     |             | Exp 12. Automate and test gmail using Google chrome                                                                                                                                                                                                                                                                                                 | 2   | 142 |
|     |             | Viva Questions                                                                                                                                                                                                                                                                                                                                      |     |     |
|     |             | Exp 13. Study of Open-source Tool- TestLink                                                                                                                                                                                                                                                                                                         | 1,2 | 149 |
|     |             | [ Content Beyond ]                                                                                                                                                                                                                                                                                                                                  |     |     |

**JAIPUR ENGINEERING COLLEGE AND RESEARCH CENTER**

**Department of Information Technology**

Branch: Information Technology

Semester: 8<sup>th</sup>

Course Name: STV Lab

Code: 6IT4-24

External Marks: 30

Practical hrs: 2 hr/week

Internal Marks: 20

Total Marks: 50

## **Vision and Mission of the Institute**

### **Vision :**

Vision To become a renowned centre of outcome based learning, and work towards academic, professional, cultural and social enrichment of the lives of individuals and communities.

### **Mission:**

- Focus on evaluation of learning outcomes and motivate students to inculcate research aptitude by project based learning.
- Identify, based on informed perception of Indian, regional and global needs, areas of focus and provide platform to gain knowledge and solutions.
- Offer opportunities for interaction between academia and industry.
- Develop human potential to its fullest extent so that intellectually capable and imaginatively gifted leaders can emerge in a range of professions.

## **Vision and Mission of the Department**

### **VISION:**

To establish outcome based excellence in teaching, learning and commitment to support IT Industry.

### **MISION:**

- To provide outcome based education.
- To provide fundamental & Intellectual knowledge with essential skills to meet current and future need of IT Industry across the globe.
- To inculcate the philosophy of continuous learning, ethical values & Social Responsibility.

### **Program Educational Objectives (PEOs)**

1. To enrich students with fundamental knowledge, effective computing, problem solving and communication skills enable them to have successful career in Information Technology.
2. To enable students in acquiring Information Technology's latest tools, technologies and management principles to give them an ability to solve multidisciplinary engineering problems.
3. To impart students with ethical values and commitment towards sustainable development in collaborative mode.
4. To imbibe students with research oriented and innovative approaches which help them to identify, analyze, formulate and solve real life problems and motivates them for lifelong learning.
5. To empower students with leadership quality and team building skills that prepare them for employment, entrepreneurship and to become competent professionals to serve societies and global needs

## Program Outcomes (POs):

1. **Engineering Knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems in IT.
2. **Problem analysis:** Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences in IT.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations using IT.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions using IT.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations in IT.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice using IT.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development in IT.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice using IT.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings in IT.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

**11. Project Management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage IT projects and in multidisciplinary environments.

**12. Life –long Learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological changes needed in IT.

**PSO of the Department:**

PSO1: Graduates of the program would be able to develop mobile and web based IT solutions for real time problems.

PSO2: Graduates of the program would be able to apply the concepts of artificial intelligence, machine learning and deep learning

# RAJASTHAN TECHNICAL UNIVERSITY

## 8IT4-22: STV Lab

Class: VIII Sem. B.Tech.

Branch: Information Technology

Schedule per Week Practical Hrs.: 2

Examination Time = Two (2) Hours

Maximum Marks = 50 [Sessional/Mid-term (30) & End-term (20)]



### RAJASTHAN TECHNICAL UNIVERSITY, KOTA

Scheme & Syllabus

IV Year- VII Semester: B. Tech. (Computer Science & Engineering)

#### SCS4-22: Software Testing and Validation Lab

Credit: 1  
OL+OT+2P

Max. Marks:50 (IA:30, ETE:20)  
End Term Exam: 2 Hours

| SN                  | List of Experiments                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                     |  |      |         |      |        |            |          |           |              |          |                     |             |                     |                 |          |                 |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|--|------|---------|------|--------|------------|----------|-----------|--------------|----------|---------------------|-------------|---------------------|-----------------|----------|-----------------|
| 1                   | <p>a) Write a program that calculates the area and perimeter of the circle. And find the Coverage &amp; Test Cases of that program using JaButi Tool.</p> <p>b) Write a program which read the first name and last name from console and matching with expected result by using JaBuTi.</p> <p>c) Write a program that takes three double numbers from the java console representing , respectively, the three coefficients a,b, and c of a quadratic equation.</p> <p>d) Write a program that reads commercial website URL from a url from file .you should expect that the URL starts with www and ends with .com. retrieve the name of the site and output it. For instance, if the user inputs www.yahoo.com, you should output yahoo. After that find the test cases and coverage using JaButi.</p> <p>e) Write a program for a calculator and find the test case and coverage and Def-use-graph.</p> <p>f) Write a program that reads two words representing passwords from the java console and outputs the number of character in the smaller of the two. For example, if the words are open and sesame, then the output should be 4, the length of the shorter word, open. And test this program using JaButi</p> |                     |  |      |         |      |        |            |          |           |              |          |                     |             |                     |                 |          |                 |
| 2                   | <p>Analyse the performance of following website using JMeter.</p> <table><tr><th>Site</th><th>Website</th><th>Type</th></tr><tr><td>Amazon</td><td>Amazon.com</td><td>shopping</td></tr><tr><td>Flip kart</td><td>Flipkart.com</td><td>shopping</td></tr><tr><td>Railway reservation</td><td>Irctc.co.in</td><td>Ticket booking site</td></tr><tr><td>Train searching</td><td>Erail.in</td><td>Train searching</td></tr></table>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                     |  | Site | Website | Type | Amazon | Amazon.com | shopping | Flip kart | Flipkart.com | shopping | Railway reservation | Irctc.co.in | Ticket booking site | Train searching | Erail.in | Train searching |
| Site                | Website                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Type                |  |      |         |      |        |            |          |           |              |          |                     |             |                     |                 |          |                 |
| Amazon              | Amazon.com                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | shopping            |  |      |         |      |        |            |          |           |              |          |                     |             |                     |                 |          |                 |
| Flip kart           | Flipkart.com                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | shopping            |  |      |         |      |        |            |          |           |              |          |                     |             |                     |                 |          |                 |
| Railway reservation | Irctc.co.in                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | Ticket booking site |  |      |         |      |        |            |          |           |              |          |                     |             |                     |                 |          |                 |
| Train searching     | Erail.in                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Train searching     |  |      |         |      |        |            |          |           |              |          |                     |             |                     |                 |          |                 |
| 3                   | <p>Calculate the mutation score of programs given in 1(a) to 1 (f) using jumble Tool.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                     |  |      |         |      |        |            |          |           |              |          |                     |             |                     |                 |          |                 |
| 4                   | <p>Calculate the coverage analysis of programs given in 1 (a) to 1 (f) using Eclemma Free open source Tool.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                     |  |      |         |      |        |            |          |           |              |          |                     |             |                     |                 |          |                 |

5

Generate Test sequences and validate using Selenium tool for given websites below:

|                     |              |                     |
|---------------------|--------------|---------------------|
| Site                | Website      | Type                |
| Amazon              | Amazon.com   | shopping            |
| Flip kart           | Flipkart.com | shopping            |
| Railway reservation | Irctc.co.in  | Ticket booking site |
| Train searching     | Erail.in     | Train searching     |

## **INSTRUCTIONS OF LAB**

### **DO's**

1. Please switch off the Mobile/Cell phone before entering Lab.
2. Enter the Lab with complete source code and data.
3. Check whether all peripheral are available at your desktop before proceeding for program.
4. Intimate the lab In charge whenever you are incompatible in using the system or in case software get corrupted/ infected by virus.
5. Arrange all the peripheral and seats before leaving the lab.
6. Properly shutdown the system before leaving the lab.
7. Keep the bag outside in the racks.
8. Enter the lab on time and leave at proper time.
9. Maintain the decorum of the lab.
10. Utilize lab hours in the corresponding experiment.
11. Get your Cd / Pendrive checked by lab In charge before using it in the lab.

### **DON'TS**

1. No one is allowed to bring storage devices like Pan Drive /Floppy etc. in the lab.
2. Don't mishandle the system.
3. Don't leave the system on standing for long
4. Don't bring any external material in the lab.
5. Don't make noise in the lab.
6. Don't bring the mobile in the lab. If extremely necessary then keep ringers off.
7. Don't enter in the lab without permission of lab Incharge.
8. Don't litter in the lab.
9. Don't delete or make any modification in system files.
10. Don't carry any lab equipments outside the lab.

We need your full support and cooperation for smooth functioning of the.

## **INSTRUCTIONS FOR STUDENT**

### **BEFORE ENTERING IN THE LAB**

- All the students are supposed to prepare the theory regarding the next program.
- Students are supposed to bring the practical file and the lab copy.
- Assignment given in previous labs should be written in the practical file.
- Print out of diagram should be pasted in the lab file.
- Any student not following these instructions will be denied entry in the lab.

### **WHILE WORKING IN THE LAB**

- Adhere to experimental schedule as instructed by the lab Incharge.
- Get the previously executed program signed by the instructor.
- Get the output of the current program checked by the instructor in the lab copy.
- Each student should work on his/her assigned computer at each turn of the lab.
- Take responsibility of valuable accessories.
- Concentrate on the assigned practical and do not play games.
- If anyone caught red handed carrying any equipment of the lab, then he will have to face serious consequences.

**Course Outcomes:****Subject: 8IT4-22: STV Lab**

Graduates would be able:

**CO1: To design and develop various test cases, understand and implement automation testing through various tools.****CO2: To analyze and validate the performance, test sequences of various websites using JMeter, Selenium and similar tools.****MAPPING OF CO & PO**

| CO  | PO 1 | PO 2 | PO 3 | PO 4 | PO 5 | PO 6 | PO 7 | PO 8 | PO 9 | PO1 0 | PO1 1 | PO1 2 |
|-----|------|------|------|------|------|------|------|------|------|-------|-------|-------|
| CO1 | 2    | 3    | 3    | 2    | 3    | 2    | 2    | 2    | 2    | 3     | 2     | 3     |
| CO2 | 2    | 3    | 3    | 2    | 3    | 2    | 1    | 2    | 2    | 3     | 2     | 3     |

[L=1, M=2, H=3]

**MAPPING OF PEOs & POS**

| PROGRAM OBJECTIVES | PROGRAM OUTCOMES |   |   |   |   |   |   |   |   |    |    |    |
|--------------------|------------------|---|---|---|---|---|---|---|---|----|----|----|
|                    | 1                | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| I                  | 3                | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 0  | 3  |
| II                 | 2                | 3 | 2 | 3 | 0 | 3 | 0 | 0 | 0 | 0  | 1  | 3  |
| III                | 1                | 3 | 2 | 3 | 1 | 0 | 0 | 0 | 0 | 0  | 2  | 0  |
| IV                 | 0                | 0 | 0 | 1 | 2 | 0 | 3 | 2 | 3 | 0  | 2  | 0  |
| V                  | 0                | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 2  | 0  | 0  |

[L=1, M=2, H=3]

## **INSTRUCTIONAL METHODS:-**

### **Direct Instructions:**

- Black board presentation

### **Interactive Instruction:**

- Coding

### **Indirect Instructions:**

- Problem solving

## **LEARNING MATERIALS:-**

Text/Lab Manual

## **ASSESSMENT OF OUTCOMES:-**

- End term Practical exam (Conducted by RTU, KOTA)
- Daily Lab interaction.

## **OUTCOMES WILL BE ACHIEVED THROUGH FOLLOWING:-**

1. Lab Teaching (through chalk and board).
2. Discussion on website work

## Software Tools Used:

### Various tools in our experiments for software testing

**JaBUTi** is an open source software testing tool that implements state of the art techniques for control-flow and data-flow based testing of Java applications.

**jMeter** is designed to cover various categories of tests such as load testing, functional testing, performance testing, regression testing, etc.

**Jumble** is a class level mutation testing tool that works in conjunction with JUnit. The purpose of mutation testing is to provide a measure of the effectiveness of test cases. A single mutation is performed on the code to be tested, the corresponding test cases are then executed.

**EclEmma** records which parts of your Java code are executed during a particular program launch. This technique is called code coverage analysis and typically used with automated testing like JUnit unit tests. It helps to identify untested parts of a code base and improve the corresponding tests.

**Selenium** is a free (open-source) automated testing framework used to validate web applications across different browsers and platforms. You can use multiple programming languages like Java, C#, Python etc to create Selenium Test Scripts. Testing done using the Selenium testing tool is usually referred to as Selenium Testing/Automation

### What is Testing?

Testing is the process of evaluating a system or its component(s) with the intent to find whether it satisfies the specified requirements or not. In simple words, testing is executing a system in order to identify any gaps, errors, or missing requirements in contrary to the actual requirements.

According to ANSI/IEEE 1059 standard, Testing can be defined as - A process of analyzing a software item to detect the differences between existing and required conditions (that is defects/errors/bugs) and to evaluate the features of the software item.

### Who does Testing?

It depends on the process and the associated stakeholders of the project(s). In the IT industry, large companies have a team with responsibilities to evaluate the developed software in context of the given requirements. Moreover, developers also conduct testing which is called **Unit Testing**. In most cases, the following professionals are involved in testing a system within their respective capacities –

- Software Tester
- Software Developer
- Project Lead/Manager
- End User

Different companies have different designations for people who test the software on the basis of their experience and knowledge such as Software Tester, Software Quality Assurance Engineer, QA Analyst, etc.

It is not possible to test the software at any time during its cycle. The next two sections state when testing should be started and when to end it during the SDLC.

#### When to Start Testing?

An early start to testing reduces the cost and time to rework and produce error-free software that is delivered to the client. However in Software Development Life Cycle (SDLC), testing can be started from the Requirements Gathering phase and continued till the deployment of the software.

It also depends on the development model that is being used. For example, in the Waterfall model, formal testing is conducted in the testing phase; but in the incremental model, testing is performed at the end of every increment/iteration and the whole application is tested at the end.

Testing is done in different forms at every phase of SDLC –

- During the requirement gathering phase, the analysis and verification of requirements are also considered as testing.
- Reviewing the design in the design phase with the intent to improve the design is also considered as testing.
- Testing performed by a developer on completion of the code is also categorized as testing.

#### When to Stop Testing?

It is difficult to determine when to stop testing, as testing is a never-ending process and no one can claim that a software is 100% tested. The following aspects are to be considered for stopping the testing process –

- Testing Deadlines
- Completion of test case execution
- Completion of functional and code coverage to a certain point
- Bug rate falls below a certain level and no high-priority bugs are identified
- Management decision

#### Software Testing - Types of Testing

This section describes the different types of testing that may be used to test a software during SDLC.

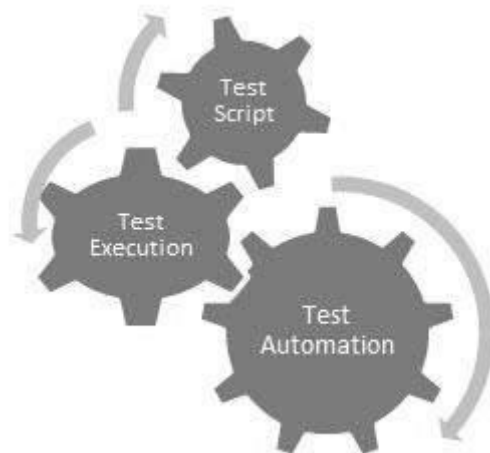
#### Manual Testing

Manual testing includes testing a software manually, i.e., without using any automated tool or any script. In this type, the tester takes over the role of an end-user and tests the software to identify any unexpected behavior or bug. There are different stages for manual testing such as unit testing, integration testing, system testing, and user acceptance testing.

Testers use test plans, test cases, or test scenarios to test a software to ensure the completeness of testing. Manual testing also includes exploratory testing, as testers explore the software to identify errors in it.

### Automation Testing

Automation testing, which is also known as Test Automation, is when the tester writes scripts and uses another software to test the product. This process involves automation of a manual process. Automation Testing is used to re-run the test scenarios that were performed manually, quickly, and repeatedly.



Apart from regression testing, automation testing is also used to test the application from load, performance, and stress point of view. It increases the test coverage, improves accuracy, and saves time and money in comparison to manual testing.

#### What to Automate?

It is not possible to automate everything in a software. The areas at which a user can make transactions such as the login form or registration forms, any area where large number of users can access the software simultaneously should be automated.

Furthermore, all GUI items, connections with databases, field validations, etc. can be efficiently tested by automating the manual process.

#### When to Automate?

Test Automation should be used by considering the following aspects of a software –

- Large and critical projects
- Projects that require testing the same areas frequently
- Requirements not changing frequently
- Accessing the application for load and performance with many virtual users
- Stable software with respect to manual testing
- Availability of time

#### How to Automate?

Automation is done by using a supportive computer language like VB scripting and an automated software application. There are many tools available that can be used to write automation scripts. Before mentioning the tools, let us identify the process that can be used to automate the testing process –

- Identifying areas within a software for automation
- Selection of appropriate tool for test automation
- Writing test scripts
- Development of test suits
- Execution of scripts
- Create result reports
- Identify any potential bug or performance issues

#### Software Testing Tools

The following tools can be used for automation testing –

- HP Quick Test Professional
- Selenium
- IBM Rational Functional Tester
- SilkTest
- TestComplete
- Testing Anywhere
- WinRunner
- LoadRunner
- Visual Studio Test Professional
- WATIR

#### Software Testing - Methods

There are different methods that can be used for software testing. This chapter briefly describes the methods available.

## Black-Box Testing

The technique of testing without having any knowledge of the interior workings of the application is called black-box testing. The tester is oblivious to the system architecture and does not have access to the source code. Typically, while performing a black-box test, a tester will interact with the system's user interface by providing inputs and examining outputs without knowing how and where the inputs are worked upon.

The following table lists the advantages and disadvantages of black-box testing.

| Advantages                                                                                                                                            | Disadvantages                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Well suited and efficient for large code segments.                                                                                                    | Limited coverage, since only a selected number of test scenarios is actually performed.               |
| Code access is not required.                                                                                                                          | Inefficient testing, due to the fact that the tester only has limited knowledge about an application. |
| Clearly separates user's perspective from the developer's perspective through visibly defined roles.                                                  | Blind coverage, since the tester cannot target specific code segments or errorprone areas.            |
| Large numbers of moderately skilled testers can test the application with no knowledge of implementation, programming language, or operating systems. | The test cases are difficult to design.                                                               |

## White-Box Testing

White-box testing is the detailed investigation of internal logic and structure of the code. White-box testing is also called **glass testing** or **open-box testing**. In order to perform **white-box** testing on an application, a tester needs to know the internal workings of the code.

The tester needs to have a look inside the source code and find out which unit/chunk of the code is behaving inappropriately.

The following table lists the advantages and disadvantages of white-box testing.

| Advantages                                                                                                                                           | Disadvantages                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| As the tester has knowledge of the source code, it becomes very easy to find out which type of data can help in testing the application effectively. | Due to the fact that a skilled tester is needed to perform white-box testing, the costs are increased.                                            |
| It helps in optimizing the code.                                                                                                                     | Sometimes it is impossible to look into every nook and corner to find out hidden errors that may create problems, as many paths will go untested. |
| Extra lines of code can be removed which can bring in hidden defects.                                                                                | It is difficult to maintain white-box testing, as it requires specialized tools like code analyzers and debugging tools.                          |
| Due to the tester's knowledge about the code, maximum coverage is attained during test scenario writing.                                             |                                                                                                                                                   |

### Grey-Box Testing

Grey-box testing is a technique to test the application with having a limited knowledge of the internal workings of an application. In software testing, the phrase the more you know, the better carries a lot of weight while testing an application.

Mastering the domain of a system always gives the tester an edge over someone with limited domain knowledge. Unlike black-box testing, where the tester only tests the application's user interface; in grey-box testing, the tester has access to design documents and the database. Having this knowledge, a tester can prepare better test data and test scenarios while making a test plan.

| Advantages                                                                     | Disadvantages                                                                                                   |
|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Offers combined benefits of black-box and white-box testing wherever possible. | Since the access to source code is not available, the ability to go over the code and test coverage is limited. |

|                                                                                                                                                                     |                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Grey box testers don't rely on the source code; instead they rely on interface definition and functional specifications.                                            | The tests can be redundant if the software designer has already run a test case.                                                                         |
| Based on the limited information available, a grey-box tester can design excellent test scenarios especially around communication protocols and data type handling. | Testing every possible input stream is unrealistic because it would take an unreasonable amount of time; therefore, many program paths will go untested. |
| The test is done from the point of view of the user and not the designer.                                                                                           |                                                                                                                                                          |

### A Comparison of Testing Methods

The following table lists the points that differentiate black-box testing, grey-box testing, and white-box testing.

| <b>Black-Box Testing</b>                                                      | <b>Grey-Box Testing</b>                                                                                   | <b>White-Box Testing</b>                                                    |
|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| The internal workings of an application need not be known.                    | The tester has limited knowledge of the internal workings of the application.                             | Tester has full knowledge of the internal workings of the application.      |
| Also known as closed-box testing, data-driven testing, or functional testing. | Also known as translucent testing, as the tester has limited knowledge of the insides of the application. | Also known as clear-box testing, structural testing, or code-based testing. |
| Performed by end-users and also by testers and developers.                    | Performed by end-users and also by testers and developers.                                                | Normally done by testers and developers.                                    |

|                                                                                              |                                                                                      |                                                                                    |
|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Testing is based on external expectations - Internal behavior of the application is unknown. | Testing is done on the basis of high-level database diagrams and data flow diagrams. | Internal workings are fully known and the tester can design test data accordingly. |
| It is exhaustive and the least time-consuming.                                               | Partly time-consuming and exhaustive.                                                | The most exhaustive and time-consuming type of testing.                            |
| Not suited for algorithm testing.                                                            | Not suited for algorithm testing.                                                    | Suited for algorithm testing.                                                      |
| This can only be done by trial-and-error method.                                             | Data domains and internal boundaries can be tested, if known.                        | Data domains and internal boundaries can be better tested.                         |

#### Software Testing - Levels

There are different levels during the process of testing. In this chapter, a brief description is provided about these levels.

Levels of testing include different methodologies that can be used while conducting software testing. The main levels of software testing are –

- Functional Testing
- Non-functional Testing

#### Functional Testing

This is a type of black-box testing that is based on the specifications of the software that is to be tested. The application is tested by providing input and then the results are examined that need to conform to the functionality it was intended for. Functional testing of a software is conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements.

There are five steps that are involved while testing an application for functionality.

| Steps | Description                                                                               |
|-------|-------------------------------------------------------------------------------------------|
| I     | The determination of the functionality that the intended application is meant to perform. |
| II    | The creation of test data based on the specifications of the application.                 |
| III   | The output based on the test data and the specifications of the application.              |
| IV    | The writing of test scenarios and the execution of test cases.                            |
| V     | The comparison of actual and expected results based on the executed test cases.           |

An effective testing practice will see the above steps applied to the testing policies of every organization and hence it will make sure that the organization maintains the strictest of standards when it comes to software quality.

### Unit Testing

This type of testing is performed by developers before the setup is handed over to the testing team to formally execute the test cases. Unit testing is performed by the respective developers on the individual units of source code assigned areas. The developers use test data that is different from the test data of the quality assurance team.

The goal of unit testing is to isolate each part of the program and show that individual parts are correct in terms of requirements and functionality.

### Limitations of Unit Testing

Testing cannot catch each and every bug in an application. It is impossible to evaluate every execution path in every software application. The same is the case with unit testing.

There is a limit to the number of scenarios and test data that a developer can use to verify a source code. After having exhausted all the options, there is no choice but to stop unit testing and merge the code segment with other units.

### Integration Testing

Integration testing is defined as the testing of combined parts of an application to determine if they function correctly. Integration testing can be done in two ways: Bottom-up integration testing and Top-down integration testing.

| Sr.No. | Integration Testing Method                                                                                                                                                 |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1      | <b>Bottom-up integration</b><br><br>This testing begins with unit testing, followed by tests of progressively higher-level combinations of units called modules or builds. |
| 2      | <b>Top-down integration</b><br><br>In this testing, the highest-level modules are tested first and progressively, lower-level modules are tested thereafter.               |

In a comprehensive software development environment, bottom-up testing is usually done first, followed by top-down testing. The process concludes with multiple tests of the complete application, preferably in scenarios designed to mimic actual situations.

### System Testing

System testing tests the system as a whole. Once all the components are integrated, the application as a whole is tested rigorously to see that it meets the specified Quality Standards. This type of testing is performed by a specialized testing team.

System testing is important because of the following reasons –

- System testing is the first step in the Software Development Life Cycle, where the application is tested as a whole.
- The application is tested thoroughly to verify that it meets the functional and technical specifications.
- The application is tested in an environment that is very close to the production environment where the application will be deployed.
- System testing enables us to test, verify, and validate both the business requirements as well as the application architecture.

### Regression Testing

Whenever a change in a software application is made, it is quite possible that other areas within the application have been affected by this change. Regression testing is performed to verify that a fixed bug hasn't resulted in another functionality or business rule violation. The intent of

regression testing is to ensure that a change, such as a bug fix should not result in another fault being uncovered in the application.

Regression testing is important because of the following reasons –

- Minimize the gaps in testing when an application with changes made has to be tested.
- Testing the new changes to verify that the changes made did not affect any other area of the application.
- Mitigates risks when regression testing is performed on the application.
- Test coverage is increased without compromising timelines.
- Increase speed to market the product.

### Acceptance Testing

This is arguably the most important type of testing, as it is conducted by the Quality Assurance Team who will gauge whether the application meets the intended specifications and satisfies the client's requirement. The QA team will have a set of pre-written scenarios and test cases that will be used to test the application.

More ideas will be shared about the application and more tests can be performed on it to gauge its accuracy and the reasons why the project was initiated. Acceptance tests are not only intended to point out simple spelling mistakes, cosmetic errors, or interface gaps, but also to point out any bugs in the application that will result in system crashes or major errors in the application.

By performing acceptance tests on an application, the testing team will reduce how the application will perform in production. There are also legal and contractual requirements for acceptance of the system.

### Alpha Testing

This test is the first stage of testing and will be performed amongst the teams (developer and QA teams). Unit testing, integration testing and system testing when combined together is known as alpha testing. During this phase, the following aspects will be tested in the application –

- Spelling Mistakes
- Broken Links
- Cloudy Directions
- The Application will be tested on machines with the lowest specification to test loading times and any latency problems.

### Beta Testing

This test is performed after alpha testing has been successfully performed. In beta testing, a sample of the intended audience tests the application. Beta testing is also known as **pre-release**

**testing.** Beta test versions of software are ideally distributed to a wide audience on the Web, partly to give the program a "real-world" test and partly to provide a preview of the next release. In this phase, the audience will be testing the following –

- Users will install, run the application and send their feedback to the project team.
- Typographical errors, confusing application flow, and even crashes.
- Getting the feedback, the project team can fix the problems before releasing the software to the actual users.
- The more issues you fix that solve real user problems, the higher the quality of your application will be.
- Having a higher-quality application when you release it to the general public will increase customer satisfaction.

### Non-Functional Testing

This section is based upon testing an application from its non-functional attributes. Non-functional testing involves testing a software from the requirements which are nonfunctional in nature but important such as performance, security, user interface, etc.

Some of the important and commonly used non-functional testing types are discussed below.

### Performance Testing

It is mostly used to identify any bottlenecks or performance issues rather than finding bugs in a software. There are different causes that contribute in lowering the performance of a software –

- Network delay
- Client-side processing
- Database transaction processing
- Load balancing between servers
- Data rendering

Performance testing is considered as one of the important and mandatory testing type in terms of the following aspects –

- Speed (i.e. Response Time, data rendering and accessing)
- Capacity
- Stability
- Scalability

Performance testing can be either qualitative or quantitative and can be divided into different sub-types such as **Load testing** and **Stress testing**.

## Load Testing

It is a process of testing the behavior of a software by applying maximum load in terms of software accessing and manipulating large input data. It can be done at both normal and peak load conditions. This type of testing identifies the maximum capacity of software and its behavior at peak time.

Most of the time, load testing is performed with the help of automated tools such as Load Runner, AppLoader, IBM Rational Performance Tester, Apache JMeter, Silk Performer, Visual Studio Load Test, etc.

Virtual users (VUsers) are defined in the automated testing tool and the script is executed to verify the load testing for the software. The number of users can be increased or decreased concurrently or incrementally based upon the requirements.

## Stress Testing

Stress testing includes testing the behavior of a software under abnormal conditions. For example, it may include taking away some resources or applying a load beyond the actual load limit.

The aim of stress testing is to test the software by applying the load to the system and taking over the resources used by the software to identify the breaking point. This testing can be performed by testing different scenarios such as –

- Shutdown or restart of network ports randomly
- Turning the database on or off
- Running different processes that consume resources such as CPU, memory, server, etc.

## Usability Testing

Usability testing is a black-box technique and is used to identify any error(s) and improvements in the software by observing the users through their usage and operation.

According to Nielsen, usability can be defined in terms of five factors, i.e. efficiency of use, learn-ability, memory-ability, errors/safety, and satisfaction. According to him, the usability of a product will be good and the system is usable if it possesses the above factors.

Nigel Bevan and Macleod considered that usability is the quality requirement that can be measured as the outcome of interactions with a computer system. This requirement can be fulfilled and the end-user will be satisfied if the intended goals are achieved effectively with the use of proper resources.

Molich in 2000 stated that a user-friendly system should fulfill the following five goals, i.e., easy to Learn, easy to remember, efficient to use, satisfactory to use, and easy to understand.

In addition to the different definitions of usability, there are some standards and quality models and methods that define usability in the form of attributes and sub-attributes such as ISO-9126, ISO-9241-11, ISO-13407, and IEEE std.610.12, etc.

### UI vs Usability Testing

UI testing involves testing the Graphical User Interface of the Software. UI testing ensures that the GUI functions according to the requirements and tested in terms of color, alignment, size, and other properties.

On the other hand, usability testing ensures a good and user-friendly GUI that can be easily handled. UI testing can be considered as a sub-part of usability testing.

### Security Testing

Security testing involves testing a software in order to identify any flaws and gaps from security and vulnerability point of view. Listed below are the main aspects that security testing should ensure –

- Confidentiality
- Integrity
- Authentication
- Availability
- Authorization
- Non-repudiation
- Software is secure against known and unknown vulnerabilities
- Software data is secure
- Software is according to all security regulations
- Input checking and validation
- SQL insertion attacks
- Injection flaws
- Session management issues
- Cross-site scripting attacks
- Buffer overflows vulnerabilities
- Directory traversal attacks

### Portability Testing

Portability testing includes testing a software with the aim to ensure its reusability and that it can be moved from another software as well. Following are the strategies that can be used for portability testing –

- Transferring an installed software from one computer to another.
- Building executable (.exe) to run the software on different platforms.

Portability testing can be considered as one of the sub-parts of system testing, as this testing type includes overall testing of a software with respect to its usage over different environments. Computer hardware, operating systems, and browsers are the major focus of portability testing. Some of the pre-conditions for portability testing are as follows –

- Software should be designed and coded, keeping in mind the portability requirements.
- Unit testing has been performed on the associated components.
- Integration testing has been performed.
- Test environment has been established.

#### Software Testing - Documentation

Testing documentation involves the documentation of artifacts that should be developed before or during the testing of Software.

Documentation for software testing helps in estimating the testing effort required, test coverage, requirement tracking/tracing, etc. This section describes some of the commonly used documented artifacts related to software testing such as –

- Test Plan
- Test Scenario
- Test Case
- Traceability Matrix

#### Test Plan

A test plan outlines the strategy that will be used to test an application, the resources that will be used, the test environment in which testing will be performed, and the limitations of the testing and the schedule of testing activities. Typically the Quality Assurance Team Lead will be responsible for writing a Test Plan.

A test plan includes the following –

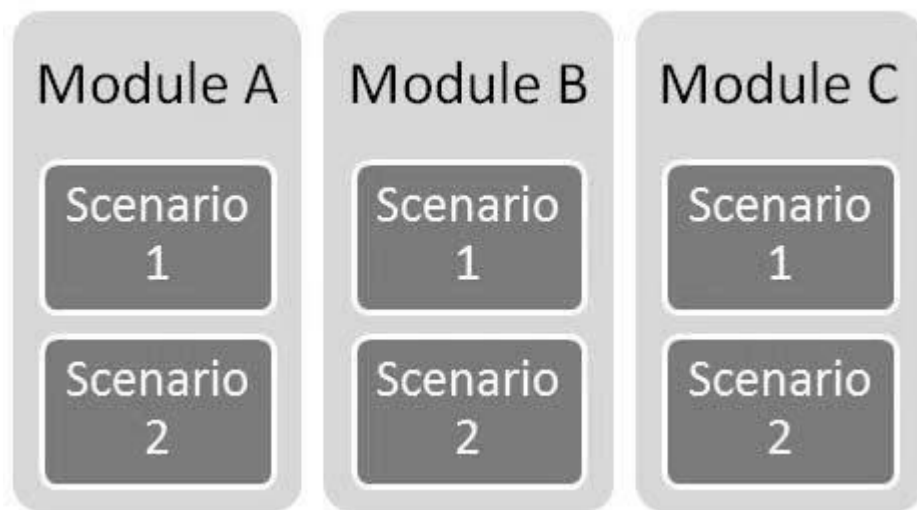
- Introduction to the Test Plan document
- Assumptions while testing the application
- List of test cases included in testing the application
- List of features to be tested

- What sort of approach to use while testing the software
- List of deliverables that need to be tested
- The resources allocated for testing the application
- Any risks involved during the testing process
- A schedule of tasks and milestones to be achieved

### Test Scenario

It is a one line statement that notifies what area in the application will be tested. Test scenarios are used to ensure that all process flows are tested from end to end. A particular area of an application can have as little as one test scenario to a few hundred scenarios depending on the magnitude and complexity of the application.

The terms 'test scenario' and 'test cases' are used interchangeably, however a test scenario has several steps, whereas a test case has a single step. Viewed from this perspective, test scenarios are test cases, but they include several test cases and the sequence that they should be executed. Apart from this, each test is dependent on the output from the previous test.



### Test Case

Test cases involve a set of steps, conditions, and inputs that can be used while performing testing tasks. The main intent of this activity is to ensure whether a software passes or fails in terms of its functionality and other aspects. There are many types of test cases such as functional, negative, error, logical test cases, physical test cases, UI test cases, etc.

Furthermore, test cases are written to keep track of the testing coverage of a software. Generally, there are no formal templates that can be used during test case writing. However, the following components are always available and included in every test case –

- Test case ID

- Product module
- Product version
- Revision history
- Purpose
- Assumptions
- Pre-conditions
- Steps
- Expected outcome
- Actual outcome
- Post-conditions

Many test cases can be derived from a single test scenario. In addition, sometimes multiple test cases are written for a single software which are collectively known as test suites.

### Traceability Matrix

Traceability Matrix (also known as Requirement Traceability Matrix - RTM) is a table that is used to trace the requirements during the Software Development Life Cycle. It can be used for forward tracing (i.e. from Requirements to Design or Coding) or backward (i.e. from Coding to Requirements). There are many user-defined templates for RTM.

Each requirement in the RTM document is linked with its associated test case so that testing can be done as per the mentioned requirements. Furthermore, Bug ID is also included and linked with its associated requirements and test case. The main goals for this matrix are –

- Make sure the software is developed as per the mentioned requirements.
- Helps in finding the root cause of any bug.
- Helps in tracing the developed documents during different phases of SDLC.
- 

### Software Testing - Estimation Techniques

Estimating the efforts required for testing is one of the major and important tasks in SDLC. Correct estimation helps in testing the software with maximum coverage. This section describes some of the techniques that can be useful in estimating the efforts required for testing.

### Functional Point Analysis

This method is based on the analysis of functional user requirements of the software with the following categories –

- Outputs
- Inquiries
- Inputs

- Internal files
- External files

### Test Point Analysis

This estimation process is used for function point analysis for black-box or acceptance testing. The main elements of this method are: Size, Productivity, Strategy, Interfacing, Complexity, and Uniformity.

## EXPERIMENT 1

1. Write a c program to demonstrate the working of the following

constructs: i) do...while

ii) while...do

iii) if ...else

iv) switch

v) for Loops in C language

i) AIM: To demonstrate the working of do...while construct

**Objective**

To understand the working of do while with different range of values and test cases

```
#include <stdio.h>
```

```
void main ()
```

```
{
    int i, n=5, j=0;
    clrscr();
    printf("enter a no");
    scanf("%d", &i);
    do
    {
        if(i%2==0)
        {
            printf("%d", i);
            printf("is a even no.");
            i++;
            j++;
        }
        else
        {
            printf("%d", i);
            printf("is a odd no.\n");

            i++;
            j++;
        }
    }
    while(i>0&& j<n);
    getch();
}
```

| Input | Actual output    |
|-------|------------------|
| 2     | 2 is even number |
|       | 3 is odd number  |
|       | 4 is even number |
|       | 5 is odd number  |
|       | 6 is even number |

Test cases:

Test case no: 1

Test case name: Positive values within range

| Input =2 | Expected output  | Actual output    | Remarks |
|----------|------------------|------------------|---------|
|          | 2 is even number | 2 is even number |         |
|          | 3 is odd number  | 3 is odd number  | success |
|          | 4 is even number | 4 is even number |         |
|          | 5 is odd number  | 5 is odd number  |         |
|          | 6 is even number | 6 is even number |         |

Test case no: 2

Test case name: Negative values within a range

| Input = -2 | Expected output   | Actual output        | Remarks |
|------------|-------------------|----------------------|---------|
|            | -2 is even number | -2 is an even number |         |
|            | -3 is odd number  |                      | fail    |
|            | -4 is even number |                      |         |
|            | -5 is odd number  |                      |         |
|            | -6 is even number |                      |         |

Test case no: 3

Test case name: Out of range values testing

| Input              | Expected output | Actual output        | Remarks |
|--------------------|-----------------|----------------------|---------|
| 123456789122222222 | 123456789122222 | 13 23456789122222215 | fail    |

ii) AIM: To demonstrate the working of while construct Objective

To understand the working of while with different range of values and test cases

```
#include<stdio.h>
```

```
#include <conio.h>
```

```
void main ()
```

```
{
```

```
    int i, n=5,j=1;
```

```
    clrscr();
```

```
    printf("enter a no");
```

```

scanf("%d",&i);
while (i>0 && j<n)
{
    if(i%2==0)
    {
        printf("%d",i);
        printf("is a even number");
        i++;
        j++;
    }
    else
    {
        printf("%d",i);
        printf("is a odd number");
        i++;
        j++;
    }
}
getch();
}

```

| Input | Actual output    |
|-------|------------------|
| 2     | 2 is even number |
|       | 3 is odd number  |
|       | 4 is even number |
|       | 5 is odd number  |
|       | 6 is even number |

Test cases:

Test case no:1

Test case name: Positive values within range

| Input =2 | Expected output  | Actual output    | Remarks |
|----------|------------------|------------------|---------|
|          | 2 is even number | 2 is even number |         |
|          | 3 is odd number  | 3 is odd number  | success |
|          | 4 is even number | 4 is even number |         |
|          | 5 is odd number  | 5 is odd number  |         |
|          | 6 is even number | 6 is even number |         |



**Test case no:2****Test case name:** Negative values within a range

| Input = -2 | Expected output   | Actual output        | Remarks |
|------------|-------------------|----------------------|---------|
|            | -2 is even number | -2 is an even number |         |
|            | -3 is odd number  |                      | fail    |
|            | -4 is even number |                      |         |
|            | -5 is odd number  |                      |         |
|            | -6 is even number |                      |         |

**Test case no: 3****Test case name:** Out of range values testing

| Input              | Expected output    | Actual output         | Remarks |
|--------------------|--------------------|-----------------------|---------|
| 123456789122222222 | 123456789122222222 | 13 234567891222222215 | fail    |

**iii) AIM: To demonstrate the working of if else construct****Objective** To understand the working of if else with different range of values and test cases

#include&lt;stdio.h&gt;

#include &lt;conio.h&gt;

void main ()

```

{
    int i;
    clrscr();
    printf("enter a number");
    scanf("%d",&i);
    if(i%2==0)
    {
        printf("%d",i);
        printf("is a even number");
    }
    else
    {
        printf("%d",i);
        printf("is a odd number");
    }
    getch();
}

```

| Input | Actual output    |
|-------|------------------|
| 2     | 2 is even number |
|       | 3 is odd number  |
|       | 4 is even number |



```

        printf("The sum of a & b is: %d",c);
        break;
    case 2: c=a-b;
        printf("The Diff of a & b is: %d",c);
        break;
    case 3: c=a*b;
        printf("The Mul of a & b is: %d",c);
        break;
    case 4: c=a/b;
        printf("The Div of a & b is: %d",c);
        break;
    default: printf("Enter your choice");
        break;
    }
    getch();
}

```

#### Output:

| Input                   | Output                  |
|-------------------------|-------------------------|
| Enter Ur choice: 1      |                         |
| Enter a, b Values: 3, 2 | The sum of a & b is:5   |
| Enter Ur choice: 2      |                         |
| Enter a, b Values: 3, 2 | The diff of a & b is: 1 |
| Enter Ur choice: 3      |                         |
| Enter a, b Values: 3, 2 | The Mul of a & b is: 6  |
| Enter Ur choice: 4      |                         |
| Enter a, b Values: 3, 2 | The Div of a & b is: 1  |

#### Test cases:

##### Test case no: 1

Test case name: Positive values within range

| Input                   | Expected output         | Actual output | Remarks |
|-------------------------|-------------------------|---------------|---------|
| Enter Ur choice: 1      |                         |               |         |
| Enter a, b Values: 3, 2 | The sum of a & b is:5   | 5             |         |
| Enter Ur choice: 2      |                         |               |         |
| Enter a, b Values: 3, 2 | The diff of a & b is: 1 | 1             | Success |
| Enter Ur choice: 3      |                         |               |         |
| Enter a, b Values: 3, 2 | The Mul of a & b is: 6  | 6             |         |
| Enter Ur choice: 4      |                         |               |         |



```

        printf("is a odd no");
        i++;
    }
}
getch();
}

```

**Output:**

Enter a no: 5  
 0 is a even no  
 1 is a odd no  
 2 is a even no  
 3 is a odd no  
 4 is a even no  
 5 is a odd no

**Test cases:**

**Test case no: 1**

**Test case name:** Positive values within range

| Input =2 | Expected output  | Actual output    | Remarks |
|----------|------------------|------------------|---------|
|          | 0 is even number | 0 is even number |         |
|          | 1 is odd number  | 1 is odd number  | success |
|          | 2 is even number | 2 is even number |         |

**Test case no: 2**

**Test case name:** Negative values within a range

| Input = -2 | Expected output   | Actual output       | Remarks |
|------------|-------------------|---------------------|---------|
|            | 0 is even number  | 0 is an even number |         |
|            | -1 is odd number  | -1 is even no       | fail    |
|            | -2 is even number | -2 is odd no        |         |

**Test case no: 3**

**Test case name:** Out of range values testing

| Input               | Expected output     | Actual output         | Remarks |
|---------------------|---------------------|-----------------------|---------|
| 1234567891222222222 | 1234567891222222222 | 13 234567891222222215 | fail    |

## EXPERIMENT 2

**AIM:** A program written in c language for matrix multiplication fails —Introspect the causes for its failure and write down the possible reasons for its failure.

**Objective:** Understand the failures of matrix multiplication

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a[3][3],b[3][3],c[3][3],i,j,k,m,n,p,q;
    clrscr();
    printf("Enter 1st matrix no.of rows & cols");
    scanf("%d%d",&m,&n);
    printf("Enter 2nd matrix no.of rows & cols");
    scanf("%d%d",&p,&q);
    printf("\n enter the matrix elements");
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&a[i][j]);
        }
    }
    printf("\n a matrix is\n");
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            printf("%d\t",a[i][j]);
        }
        printf("\n");
    }
    for(i=0;i<p;i++)
    {
        for(j=0;j<q;j++)
```

```

        {
            scanf("%d\t",&b[i][j]);
        }
    }
    printf("\n b matrix is\n");
    for(i=0;i<p;i++)
    {
        for(j=0;j<q;j++)
        {
            printf("%d\t",b[i][j]);
        }
        printf("\n");
    }
    for(i=0;i<m;i++)
    {
        for(j=0;j<q;j++)
        {
            c[i][j]=0;
            for(k=0;k<n;k++)
            {
                c[i][j]=c[i][j]+a[i][k]*b[k][j];
            }
        }
    }
    for(i=0;i<m;i++)
    {
        for(j=0;j<q;j++)
        {
            printf("%d\t",c[i][j]);
        }
        printf("\n");
    }
    getch();
}

```

Matrix1 rows & cols= 2 2

Matrix2 rows & cols= 2 2

|            |             |      |
|------------|-------------|------|
| 1234567891 | 2222222222  | fail |
| 2234567891 | 2222222221  |      |
| 234567891  | 22222221533 |      |
| 213242424  | 56456475457 |      |

Viva Questions:

Q1. What is Acceptance Testing?

Testing conducted to enable a user/customer to determine whether to accept a software product. Normally performed to validate the software meets a set of agreed acceptance criteria.

Q2. What is Accessibility Testing?

Verifying a product is accessible to the people having disabilities (deaf, blind, mentally disabled etc.).

Q3. What is Ad Hoc Testing?

A testing phase where the tester tries to 'break' the system by randomly trying the system's functionality. Can include negative testing as well. See also Monkey Testing.

Q4. What is Agile Testing?

Testing practice for projects using agile methodologies, treating development as the customer of testing and emphasizing a test-first design paradigm. See also Test Driven Development.

Q5. What is Application Binary Interface (ABI)?

A specification defining requirements for portability of applications in binary forms across different system platforms and environments.

What is Application Programming Interface (API)?

A formalized set of software calls and routines that can be referenced by an application program in order to access supporting system or network services.

What is Automated Software Quality (ASQ)?

The use of software tools, such as automated testing tools, to improve software quality.

Q6. What is Automated Testing?

Testing employing software tools which execute tests without manual intervention. Can be applied in GUI, performance, API, etc. testing.

The use of software to control the execution of tests, the comparison of actual outcomes to predicted outcomes, the setting up of test preconditions, and other test control and test reporting functions.

Q7. What is Backus-Naur Form?

A metalanguage used to formally describe the syntax of a language.

Q8. What is Basic Block?

A sequence of one or more consecutive, executable statements containing no branches.

Q9. What is Basis Path Testing?

A white box test case design technique that uses the algorithmic flow of the program to design tests.

Q10. What is Basis Set?

The set of tests derived using basis path testing.

# JaBUTi

## Manual and Installation

### 1 JaBUTi Functionality and Graphical Interface

JaBUTi implements a subset of the functionalities provided by *xSuds*, a complete tool suite for C and C++ programs [2]. This section describes the operations that can be performed by JaBUTi. The graphical interface allows the beginner to explore and learn the concepts of control-flow and data-flow testing. Moreover, it provides a better way to visualize which part of the classes under testing are already covered and which are not. A general view of the JaBUTi graphical interface, including its menus, is presented in Figure 1. A brief description of each option on each menu is presented below.

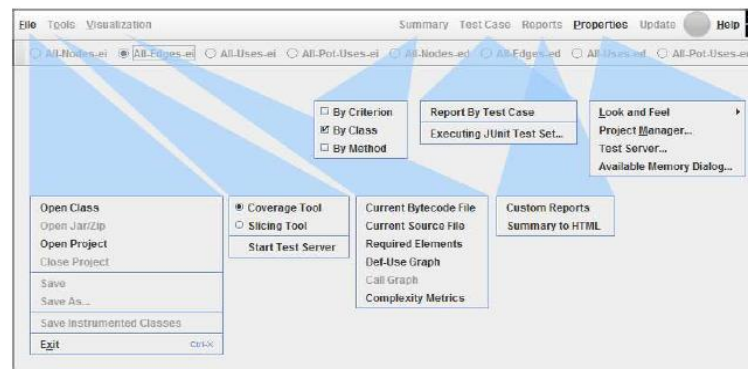


Figure 1: Operations available in the graphical interface.

- **File Menu** - provides options to create and manipulate a JaBUTi project.
  - **Open Class** - allows to select the base class file from where the classes to be tested are identified.
  - **Open Project** - opens a previously created project.
  - **Close Project** - closes the current project.
  - **Save** - saves the current project.
  - **Save As** - saves the current project with a different name.
  - **Save Instrumented Classes** - saves the classes from current project, already instrumented for external testing.
  - **Exit** - exits of the tool.
- **Tools** - provides the set of tools available in JaBUTi.
  - **Coverage Tool** - enables the JaBUTi coverage tool.
  - **Slicing Tool** - enables the JaBUTi slicing tool.
  - **Start Test Server** - starts the test server for mobile devices.
- **Visualization** - provides different forms of visualization of the classes and methods under testing.

- **Current Bytecode File** - shows the highlighted bytecode of the current selected class file.
- **Current Source File** - shows the highlighted source code of the current selected class file. Observe that this option requires that the source code is available to be performed.
- **Required Elements** - shows the set of required elements for a given method of a given class, considering the current selected criterion (shown below the main menu). The presented screen also allows to mark a testing requirement as active/deactive or feasible/infeasible.
- **Def-Use Graph** - shows the DUG of a given method of the current class.
- **Complexity Metrics** - shows the resultant value of the set of complexity metrics implemented in JaBUTi for the complete set of user classes obtained from the base class. This set of classes includes the classes under testing.
- **Summary** - provides personalized coverage information in different levels of abstractions.
  - **By Criterion** - shows the cumulative coverage information for each testing criterion, considering all classes under testing.
  - **By Class** - shows the coverage information with respect to the current selected criterion, for each individual class under testing.
  - **By Method** - shows the coverage information with respect to the current selected criterion, for each individual method of each class under testing.
- **Test Case** - provides options for test set manipulation and report generation.
  - **Report By Test Case** - shows the coverage information with respect to the current selected criterion, for each individual test case, considering all class under testing. The presented screen also allows to enable/disable and delete/undelete test cases.
  - **Importing from JUnit** - allows to import a test set generated according to the JUnit framework.
- **Reports** - provides options to save JaBUTi's reports in HTML format.
  - **Custom Reports** - allows to generate a custom HTML report from the current testing project considering different levels of granularity.
  - **Summary to HTML** - allows to generate a HTML from any tabled style report provided by the JaBUTi graphical interface.
- **Properties** - provides general configuration options.
  - **Look and Feel** - allows to change the look and feel style considering three different options: Metal (default), Motif, and Windows.
  - **Project Manager...** - allows to verify and change the current set of classes under testing in the current project.
  - **Test Server...** - configuration of the test server.
  - **Available Memory Dialog...** - shows current available memory on system.

- **Update** - provides a visual information every time an event that affect the coverage occurs. For example, such a button becomes red in case additional test cases are imported or appended in the end of the trace file to indicate that a new event that affects the coverage information occurs. As soon as it is clicked, its background color changes to grey.
- **Help** - provides only one option to show information about the authors/developers of JaBUTi.

JaBUTi requires the creation of a testing project, such that the tester can specify only once the set of classes to be instrumented and tested. Section 2 describes how to create a JaBUTi's testing project. After having created the project, JaBUTi provides to the tester a coverage analysis tool, a slicing tool and a static metric measure tool. The coverage analysis tool is described in Section 3. Section 4 describes the slicing tool, and Section 5 describes the measure tool.

## 2 How to Create a Testing Project

In JaBUTi the testing activity requires the creation of a testing project. A testing project is characterized by a file storing the necessary information about (i) the base class file, (ii) the complete set of classes required by the base class, (iii) the set of classes to be instrumented (tested), and (iv) the set of classes that are not under testing. Additional information, such as the `CLASSPATH` environment variable necessary to run the base class is also stored in the project file, whose extension is `.jbt`. During the execution of any `.class` file that belongs to the set of classes under testing of a given project, dynamic control-flow information (execution trace) is collected and saved in a separate file that has the same name of the project file but with a different extension (`.trc`).

For example, considering the vending machine example, the `vending` package is composed by three `.java` files: `VendingMachine.java`, `Dispenser.java` and `TestDriver.java`. Suppose that these files are saved in a directory named `~\example`. The directory structure is like:

```
auri@AURIMRV ~
$ ls -l example/*
-rw-r--r--  1 auri      1313 Aug  6 09:07 Dispenser.java
-rw-r--r--  1 auri      1340 Aug  6 09:07 TestDriver.java
-rw-r--r--  1 auri       923 Aug  6 09:07 VendingMachine.java
```

To compile such an application one of the following command can be used:

```
auri@AURIMRV ~
$ javac -d example example/*.java
```

or

```
auri@AURIMRV ~
$ javac -g -d example example/*.java
```

Observe that in the later command, the debug option is activated, thus, the generated `.class` files contains more information, such as the real variable names. In this report we are using class files compiled with the debug option.

After the Java source files have been compiled, the `~\example` directory contains the following structure:

```

auri@AURIMRV ~
$ ls -l example/*
-rw-r--r--  1 auri      1313 Aug  6 09:07 Dispenser.java
-rw-r--r--  1 auri      1340 Aug  6 09:07 TestDriver.java
-rw-r--r--  1 auri       923 Aug  6 09:07 VendingMachine.java
example/vending:
vending:
total 6
-rw-r--r--  1 auri      1478 Aug  6 09:49 Dispenser.class
-rw-r--r--  1 auri      1340 Aug  6 09:49 TestDriver.class
-rw-r--r--  1 auri      1253 Aug  6 09:49 VendingMachine.class

```

Now, from the generated .class files, the user can create a project using JaBUTi. To do this, the first step is to invoke JaBUTi's graphical interface. Supposing that JaBUTi is installed on `~\Tools\jabuti`, the command below causes the invocation of its graphical interface.

```

auri@AURIMRV ~/example
$ java -cp ".;..\Tools\jabuti;\
>..\Tools\jabuti\lib\BCEL.jar;\
>..\Tools\jabuti\lib\jviewsall.jar;\
>..\Tools\jabuti\lib\crimson.jar;\
>..\Tools\jabuti\lib\junit.jar" gui.JabutiGUI

```

Observe that the tool requires that some third-party libraries (BCEL [5] to manipulate the Java bytecode files, ILOG JViews [10] to visualize the DUG, Crimson to manipulate XML files<sup>1</sup>, and JUnit [3] to import test sets.) to be included in the CLASSPATH to allow its execution. The current directory, (“.” in our example), and the base directory of the tool (`..\Tools\jabuti`) are also included to allow the correct execution of our example and the tool itself. If desired, the user can set the CLASSPATH environment variable as a system variable that is initialized during the boot process. In this case, to call JaBUTi's GUI, the parameter `-cp` can be omitted. Figure 2 illustrates the JaBUTi's initial window.

To create a new project, the first step is to select a base .class file from `File → Open Class` menu. A dialog window, as illustrated in Figure 3(a) appears. From this dialog window, the tester selects the directory where the base class file is located and then select the base class file itself (Figure 3(b)). Once the base class file is selected the tool automatically identifies the package that it belongs (if any) and fills out the `Package` field with the package's name. The `Classpath` should contains only the path necessary to run the selected base class. In our example, since the tool is being called inside the `example` directory, to run `vending.TestDriver` it is necessary that the `Classpath` field contains only the current directory, as shown in Figure 3(b).

By clicking the `Open` button (Figure 3(b)), the `Project Manager` window, as illustrated in Figure 4, will be displayed. From the selected base class file (`TestDriver` in our example) the tool identifies the complete set of system and non-system class files necessary to execute `TestDriver`. Currently, JaBUTi does not allows the instrumentation of system class files. Therefore, the complete set of non-system class files (user's classes) relate to `TestDriver` can be instrumented. From the `Project Manager` window (left side) the user can select the class files that will be tested. At least one class file must be selected. In our example, we select `VendingMachine` and `Dispenser` to be instrumented. `TestDriver` was not selected

<sup>1</sup>Such a package is required when a java compiler under version 1.4.1 is used (considering the Sun compiler)).



Figure 2: JaBUTi main window.

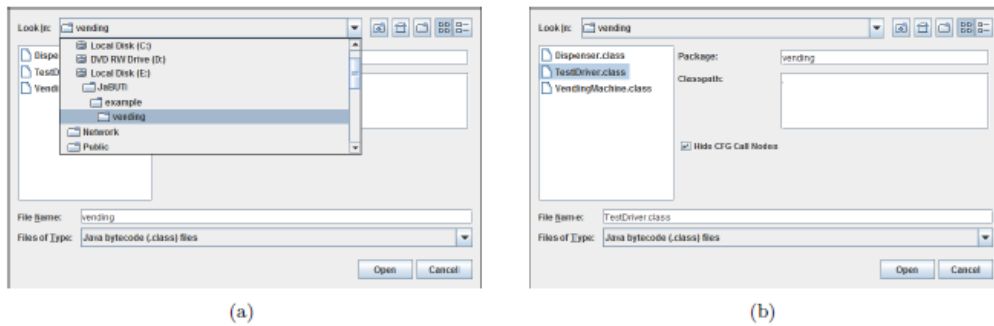


Figure 3: Open Class: (a) Selecting the directory and (b) Selecting the main class file.

since it is only the driver that allows the tester to observe whether the behavior of `VendingMachine` and `Dispenser` are correct with respect to their specification.

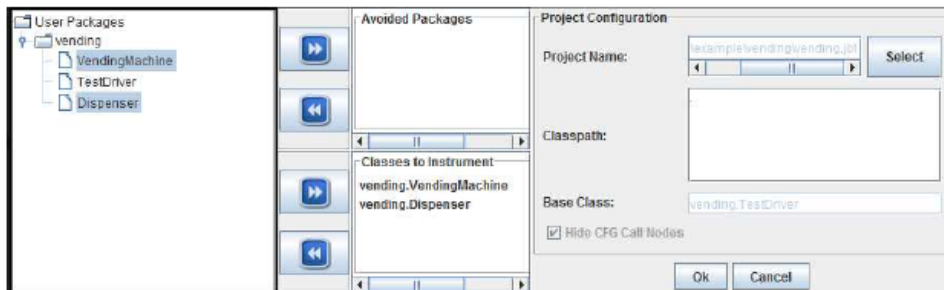


Figure 4: Selecting the class files to be tested.

Moreover, the tester must give a name to the project being created (`vending.jbt` in our example) by clicking on the **Select** button. By clicking on the **Ok** button, JaBUTi creates a new project (`vending.jbt` in our example), constructs the DUG for each method of each class under testing, derives the complete set of testing requirements for each criterion, calculates the weight of each testing requirement, and presents the bytecode of a given class under testing. By creating a project, the tester does not need to go through this entire process again if he/she intends to test the same set of class files. Next time, he/she can just go to **File** → **Open Project** menu option and reload a previously saved project.

Once the project is created, the tester just need to open it and the tool automatically detects the classes that compose it. JaBUTi can be used for analyzing the coverage of a given class file, for debugging the class file using the slice tool and for collecting static metrics information. JaBUTi also allows the visualization of the Def-Use Graph (DUG) of each method in a given class file as well as the visualization of the source code, when available. Different kinds of testing reports can also be generated considering different levels of abstraction. The following sections describe these features in detail.

### 3 How to use JaBUTi as a Coverage Analysis Tool

By default, the tool displays the bytecode instead of the Java source code, since the source code may not be available. The user can use the **Visualization** menu to change the display to show the bytecode or source code, as well as the DUG of each method in the current class. As can be observed in Figure 5, JaBUTi uses different colors to provide hints to the tester to ease the test generation. The different colors represent requirements with different weights, and the weights are calculated based on dominator and super-block analysis [1]. Informally, the weight of a testing requirement corresponds to the number of testing requirements that will be covered if this particular requirement is covered. Therefore, covering the requirement with the highest weight will increase the coverage faster<sup>2</sup>. The weight schema is used in all views of a given class/method: bytecode, source code and DUG. Figure 5 shows part of the colored bytecode of the `Dispenser.dispense()` method before the execution of any test case and Figure 6 part of the DUG of the same method. The different colors correspond to the weight of each node, considering the **All-Nodes-ei** criterion. Observe that the color bar (below the main menu) goes from white (weight 0) to red (weight 7).

In our example, observe that the node 105 in Figure 6, composed by bytecote instructions from 105 to 112 (Figure 5), is one of the highest weight. In this way, a test case that exercises node 105 increases the coverage in at least 7. A requirement with weight zero is a covered requirement and it is painted in white.

Although the source code is not required by JaBUTi, when it is available, the corresponding source code of the current class can also be displayed in colors, mapped back from the bytecode, facilitating the identification of which part of the code should be covered first. For example, Figure 7 shows that the statements on line 22 and 24 have the highest priority w.r.t. the **All-Nodes-ei** criterion. By covering one of these particular nodes, at least 7 other nodes will be covered.

There are two different types of nodes, represented by single and double line circles. Double circles represent call nodes, i.e., nodes where exists a method call to another method. This nodes are identified

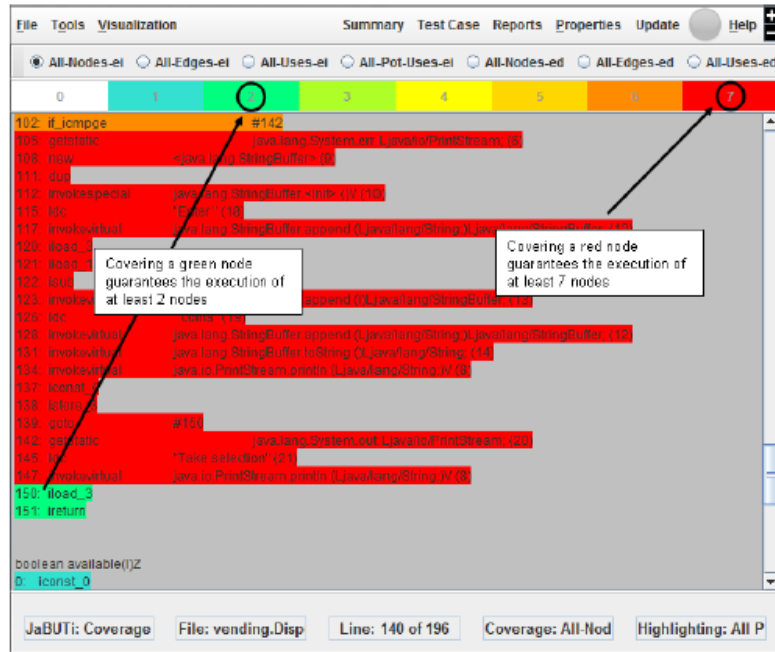


Figure 5: Dispenser.dispenser method: bytecode prioritized w.r.t. All-Pri-Nodes criterion.

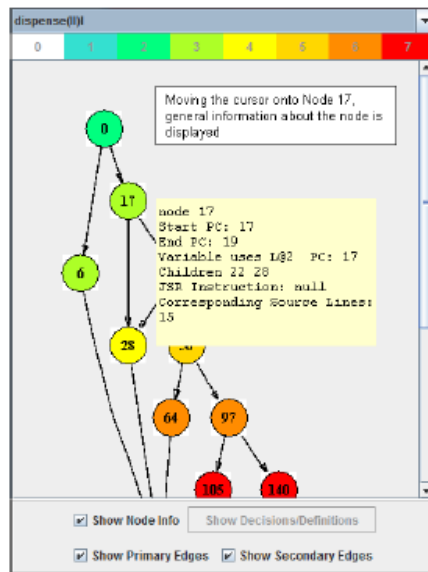


Figure 6: Dispenser.dispenser method: DUG prioritized w.r.t. All-Pri-Nodes criterion.

since we intend to extend our tool to deal, not only with intra-method testing criteria, but also with inter-method testing criteria that requires the identification of the relationship among methods. Bold circles, not visible in Figure 6, represent exit nodes. Observe that methods in Java can have more than one exit node due to the exception-handling mechanism. All the other nodes are represented



Figure 7: Dispenser.dispense method: source code prioritized w.r.t. All-Pri-Nodes criterion.

as single line circles. We also have two different types of edges to represent the “normal” control-flow (continuous line – Primary Edges) and exception control-flow (dashed lines – Secondary Edges). Figure 6 does not contain exception edges. Primary and secondary edges can be hidden by deselecting the Show Primary and Show Secondary Edges check box, respectively. The node information shown when the cursor is moved onto the node, as illustrated in Figure 6, can also be disabled by deselecting the Show Node Info check box.

### 3.1 How the testing requirements are highlighted

As can be observed above the main menu in Figure 5, JaBUTi supports the application of six structural testing criteria: All-Nodes-ei, All-Nodes-ed, All-Edges-ei, All-Edges-ed, All-Uses-ei, and All-Uses-ed. Depending on which criterion is active, the bytecode, source code or DUG is colored in a different way. Figures 5, 6, and 7 show the color schema considering the All-Nodes-ei criterion, which is the same as the All-Nodes-ed, i.e., for these criteria, each requirement is a node, therefore, since each node has its own weight, the complete bytecode, source code or DUG appears highlighted according to the weight of each node.

Considering the All-Edges-ei and All-Edges-ed criteria, their requirements (DUG edges) are colored using a 2-layer approach. For All-Edges-ei criterion, only the nodes with more than one out-going edge (decision nodes) are painted in the first layer. For example, Figure 8 shows part of the decision nodes of method Dispenser.dispense() and how they are painted in the first layer. For each decision node, its weight is the maximum weight of its branches. Suppose a decision node has two branches: one has a weight of 2 and the other has a weight of 7. The weight of this decision node is 7. A decision node has a zero weight if and only if all its branches are covered. Such decisions are highlighted in white. Observe that, in the bytecode view, only the last bytecode instruction of a decision node is

highlighted, not the entire node as the All-Nodes-ei and the All-Nodes-ed criteria. For example, DUG decision node 22 goes from offset 22 to 25, although, in the bytecode view, only bytecode instruction at offset 25 is highlighted.

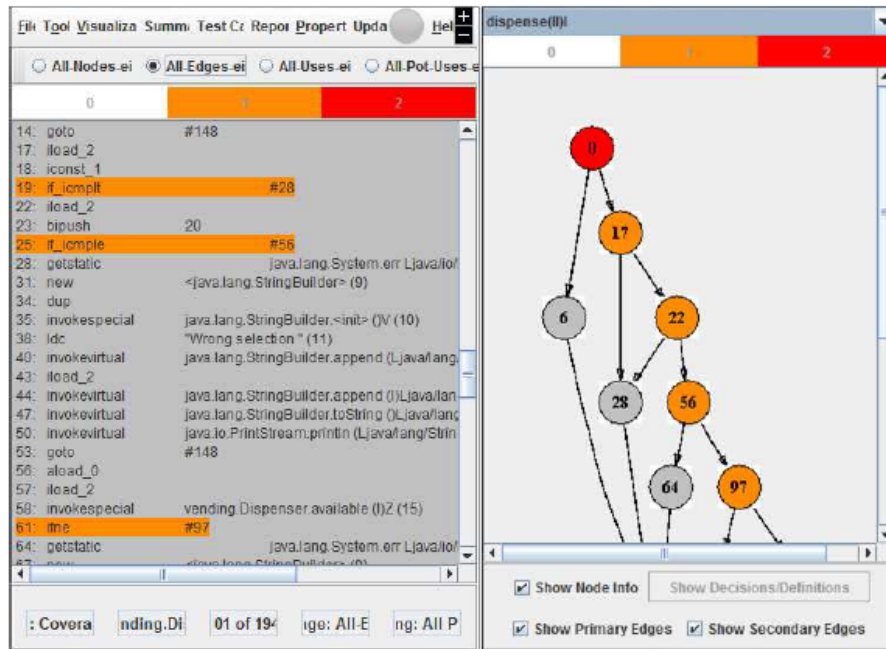


Figure 8: Color schema for All-Pri-Edges criterion: first layer.

By clicking either in a colored bytecode instruction or in a DUG node, all destination nodes of the branches associated with the selected decision node are highlighted and the decision node itself changes to a different color. For example, by clicking on the decision node with the highest weight (node 0 in the example), its children (nodes 6 and 17) are highlighted in different colors, considering their individual weights: node 6 has a weight of 2 (red) and node 17 has a weight of 1 (orange), as illustrated in Figure 9. Therefore, to improve the coverage with respect to the All-Edges-ei criterion, the tester should exercises first the edge (0,6), since it has the highest weight.

Observe that, in case a given method has no decision node, only the entry node is painted in the first layer and its child in the second layer. In this case, edges criterion is equivalent to nodes criterion, since, in a normal execution, once the entry node is exercised, all the other nodes and edges are. Figure 10 illustrates how the DUG of the default constructor of the class `VendingMachine` (`VendingMachine.init()`) is highlighted before (Figure 10(a)) and after (Figure 10(b)) the node selection.

For the All-Edges-ed criterion, the same approach is used. In the first layer, all nodes with at least one secondary out-going edge, instead of decision nodes, are highlighted. For each such a node, its weight is the maximum weight of its branches and a node with more that one secondary out-going edge is considered covered (zero weight) if and only if all exception-handler associated with such a node is covered.

Figure 11 and 12 shows the first and the second layer of `Dispenser.available()` method, considering the All-Edges-ed criterion, respectively. In this example, in the first layer, all highlighted nodes has the same weigh (one) since it has only one valid exception-handler for the entire method. In the

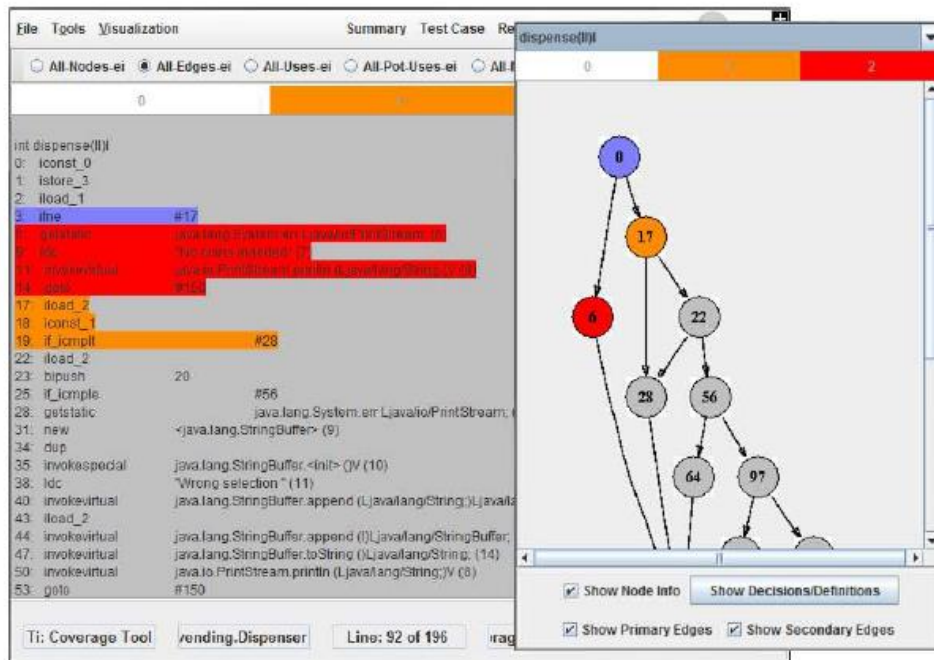


Figure 9: Color schema for All-Pri-Edges criterion: second layer.

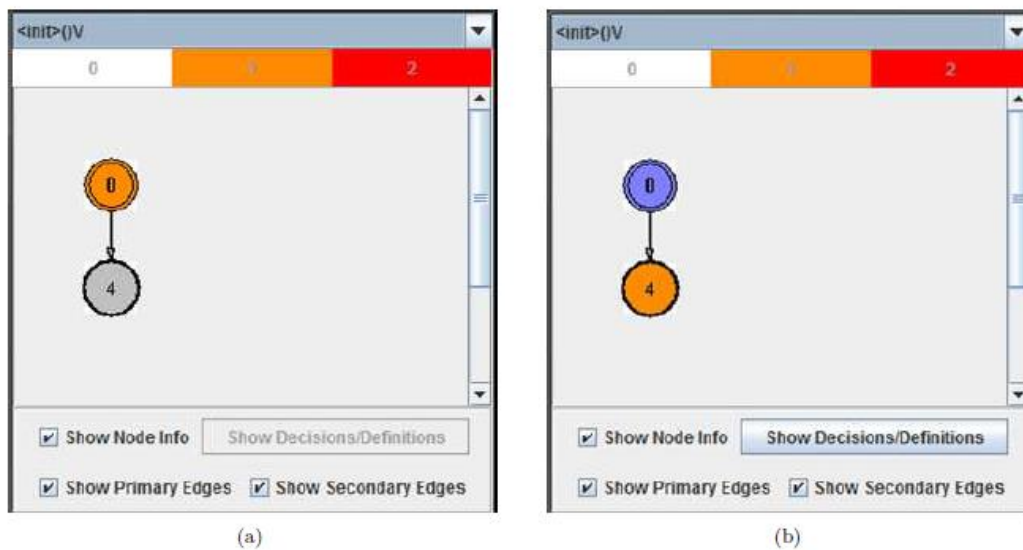


Figure 10: Special case when no decision node is found.

bytecode view, only the last bytecode instruction of each node is highlighted. For example, node 2 goes from offset 2 to 8, therefore, bytecode instruction at offset 8 is highlighted. By clicking on such an instruction or on the DUG node 2, Figure 12 shows the resultant color schema of the second layer.

Similar to a decision and its branches are displayed, a 2-layer representation is also used to display def-use associations for the All-Uses-ei and All-Uses-ed criteria. The first layer shows all the definitions

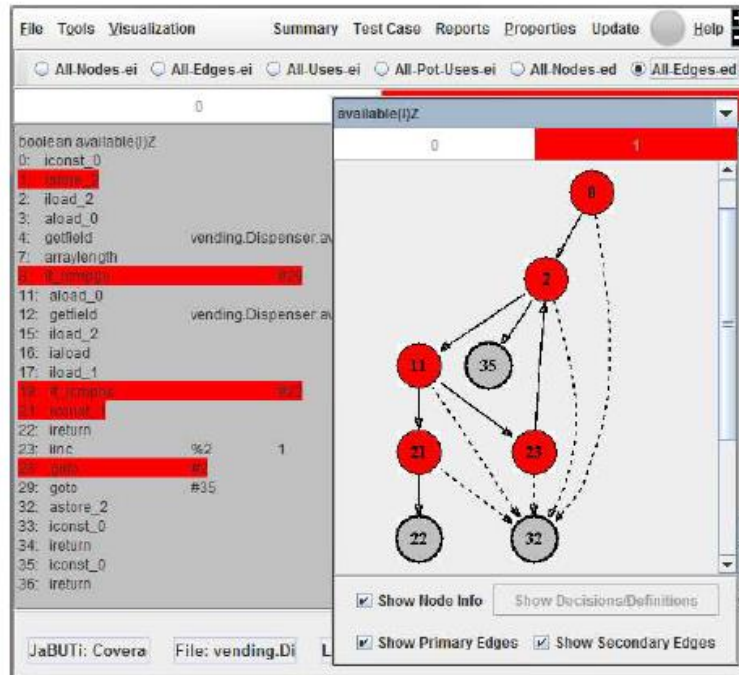


Figure 11: Color schema for All-Sec-Edges criterion: first layer.

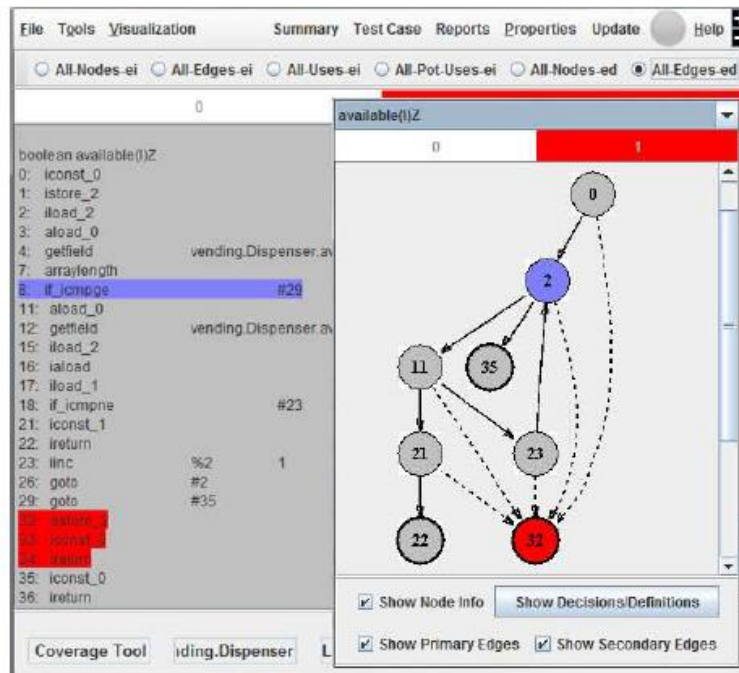


Figure 12: Color schema for All-Sec-Edges criterion: second layer.

which have at least one c-use or p-use association. Clicking on a definition goes to the second layer displaying all c-use and p-use associated with the selected definition. For each definition, its weight

is the maximum weight of its c-uses or p-uses. For example, suppose a definition has three def-use associations, one c-use and two p-uses, and the weights of these associations are 5, 3 and 7, respectively. The weight of this definition is 7. A definition has a zero weight if and only if all its c-uses and p-uses are covered. Such definitions are displayed with a white background.

Since even in the bytecode, more than one variable may be defined in the same offset, if desired, by clicking with the right mouse button over a definition point (bytecode offset, source code line or DUG node), a pop-up menu is opened showing all the variables defined in that point such that it is possible to choose which variable definition to select. For example, Figure 13 shows the set of defined variables at bytecode offset 0 that is part of the DUG node 0. Observe that in the DUG node 0 there is one additional variable definition because `L@3 (val)` is defined at bytecode offset 1 that also belongs to DUG node 0. When a definition point is clicked with the left mouse button the definition with the highest weight is considered selected. If all of them have the same weight, the first is selected. In our example, supposing that `L@1 (credit)` is selected (the definition with the highest weight), Figure 14 shows some of its uses (p-uses in this case) with the corresponding weight.

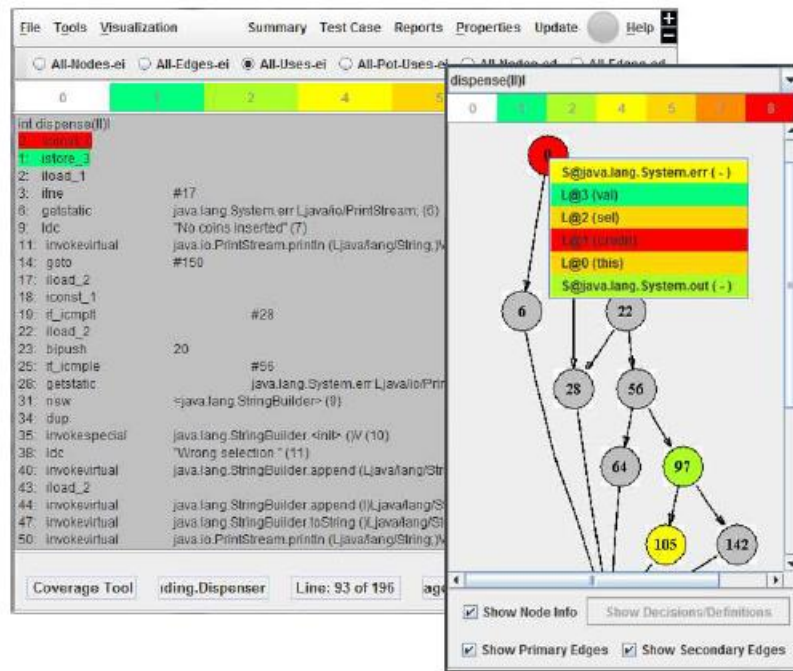


Figure 13: Color schema for All-Pri-Uses criterion: first layer.

It is important to observe that the weights provided by the tool may be seen as hints for the tester to facilitate the generation of test cases, such that a higher coverage can be obtained with fewer test cases. Since the prioritization does not consider the criticality of a given part of the code, if desired, the tester may choose a different part of the code to be tested first based on his/her knowledge about the application, even if such a part has a lower weight than other parts of the code. After the critical parts have to be tested, additional test cases can be developed considering the hints.

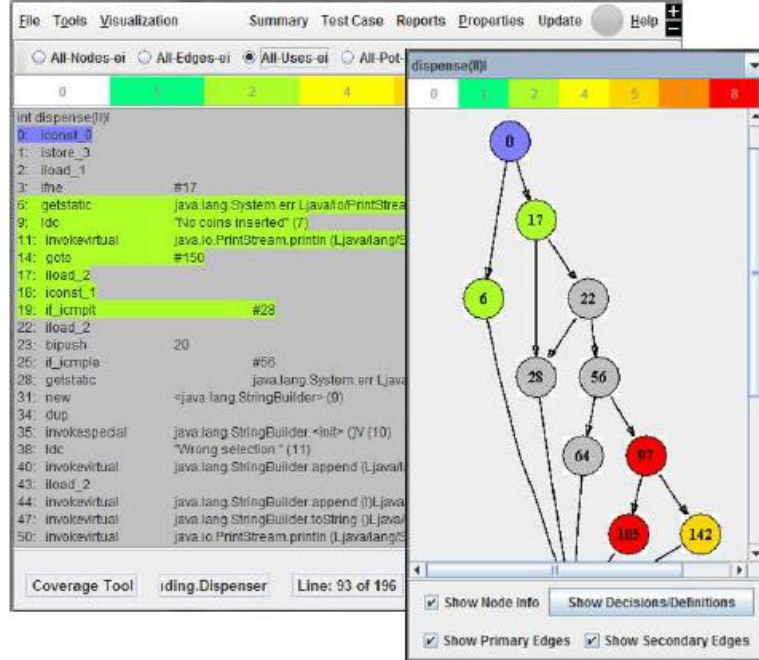


Figure 14: Color schema for All-Pri-Uses criterion: second layer.

### 3.2 How to generate testing reports

To evaluate the coverage obtained, the tool provides personalized tabled style testing reports that can be accessed from the Summary and Test Case menus. Any tabled style report can be saved as a HTML file by accessing Reports → Summary to HTML menu option. The tool provides reports w.r.t. each testing criterion (Summary → By Criterion), w.r.t. each class file (Summary → By Class) and w.r.t. each method (Summary → By Method). Figures 15, 16, and 17 show each one of these reports, respectively, considering that no test case have been executed.

| Testing Criterion | Coverage | Percentage |
|-------------------|----------|------------|
| All-Nodes-ei      | 0 of 29  | 0%         |
| All-Nodes-ed      | 0 of 1   | 0%         |
| All-Edges-ei      | 0 of 28  | 0%         |
| All-Edges-ed      | 0 of 5   | 0%         |
| All-Uses-ei       | 0 of 56  | 0%         |
| All-Uses-ed       | 0 of 0   | 0%         |
| All-Pot-Uses-ei   | 0 of 107 | 0%         |
| All-Pot-Uses-ed   | 0 of 4   | 0%         |

A mouse click over any column header causes the table data to be classified in an increasing order by such a column. If the SHIFT key is kept pressed during the click, the data is classified in a decreasing order.

JaBUTi: Coverage      Bytecode Files: 2 of 2      Active Test Cases: 0 of 0

Figure 15: Initial summary information per testing criterion.

For example, in Figures 15 it is possible to evaluate the number of required elements by criterion that were generated for the entire project, i.e., classes `VendingMachine` and `Dispenser`. The individual information per class file can be obtained by generating the summary report by class (Figures 16).

When showing the summary by class or by method, the tester can choose, among the available testing criteria, which one he/she wants to evaluate. Figures 16(a), 16(b), and 16(c) illustrate the summary per individual class w.r.t. the `All-Nodes-ei`, `All-Edges-ei`, and `All-Uses-ei` criteria, respectively. Considering the summary by class and by method, by clicking on the class file name or in the method name, the corresponding bytecode is highlighted and displayed considering the weight w.r.t. the selected criterion.

As commented in Figures 15, double-clicking on the desired column header, any tabled report generated by JaBUTi is classified in an increasing order, considering the clicked column header. A double-click with the `SHIFT` key pressed causes the table data to be sorted in a decreasing order.

Using these summary reports the tester can evaluate the current coverage of each class under testing with different granularity and decides whether a given class requires additional testing w.r.t. six different intra-method testing criteria. Section 3.4 illustrates how to include test cases and how to generate testing reports by test case and by test case paths.

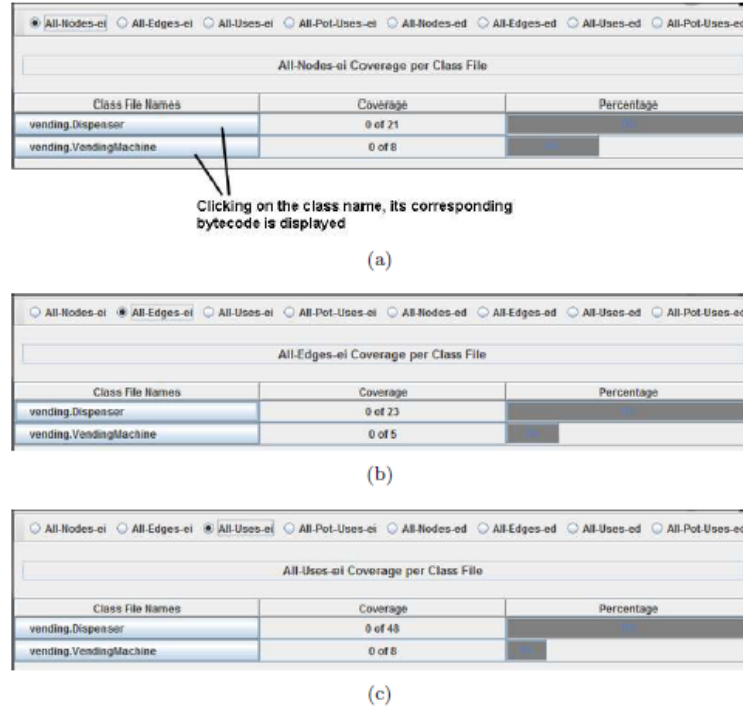


Figure 16: Initial summary per class file: (a) All-Pri-Nodes criterion, (b) All-Pri-Edges criterion, and (c) All-Pri-Uses criterion.

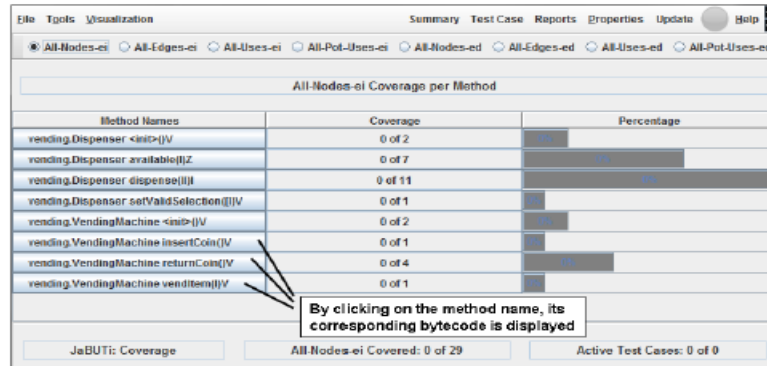


Figure 17: Initial summary information per method: All-Pri-Nodes criterion.

### 3.3 How to generate an HTML version of a JaBUTi report

As mentioned above, any kind of tabled report presented in JaBUTi's graphical interface can be saved as an HTML file through the Reports → Summary to HTML menu option. For example, Figure 18(a) shows how to create a summary-by-criterion.html file, corresponding to the current JaBUTi screen. Figure 18(b) illustrates the generated HTML file in a browser window. In this way the tester can collect and save different testing report showing the evolution of the testing activity.

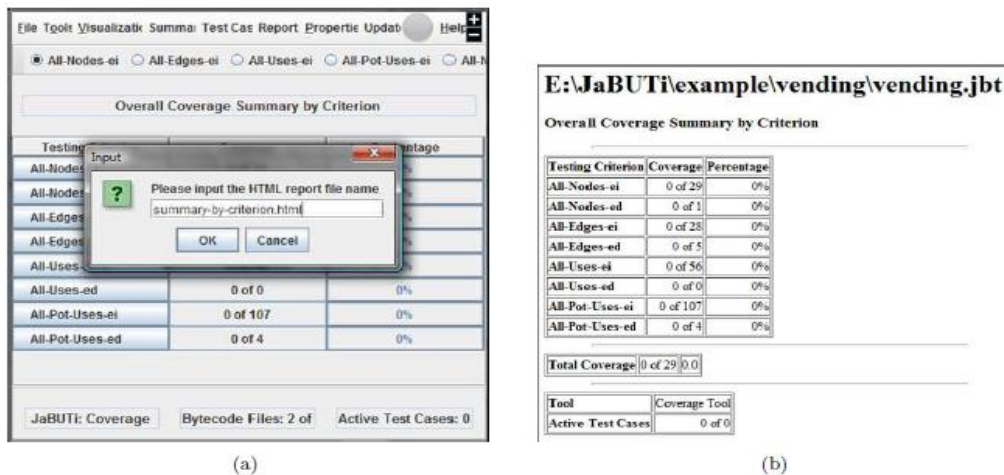


Figure 18: Example of an HTML report generated by JaBUTi.

A different kind of report can be generated through the Reports → Custom Report menu option. By selecting this option, a dialog windows, as illustrated in Figure 19(a), appears and the tester can choose the level of information that will be saved in a HTML file. Observe that to save information at method level it is also required to save information about the class and project level, since the information is saved hierarchically. Part of a complete report for the entire project is presented in Figure 19(b).

☒ Project Info  
☒ Class Info  
☒ Method Info  
☒ Test Set Info  
☒ Test Case Info  
☒ Test Case Paths Info

Output File

(a)

### Project Report: Complete

**General Info**

|                  |                                       |
|------------------|---------------------------------------|
| Name             | E:\JaBUTi\example\wending\wending.jbt |
| Type             | Research                              |
| Mobility         | false                                 |
| CFG Option       | 1                                     |
| Base Class       | vending.TestDriver                    |
| Avoided Packages |                                       |

**Project Coverage**

| Criterion       | Number Of Covered Requirements | Percentage |
|-----------------|--------------------------------|------------|
| All-Nodes-ei    | 0 of 29                        | 0.0        |
| All-Nodes-ed    | 0 of 1                         | 0.0        |
| All-Edges-ei    | 0 of 28                        | 0.0        |
| All-Edges-ed    | 0 of 5                         | 0.0        |
| All-Uses-ei     | 0 of 56                        | 0.0        |
| All-Uses-ed     | 0 of 0                         | 0.0        |
| All-Pot-Uses-ei | 0 of 107                       | 0.0        |
| All-Pot-Uses-ed | 0 of 4                         | 0.0        |

(b)

Figure 19: Example of custom HTML report generated by JaBUTi for the entire project.

### 3.4 How to include a test case

JaBUTi is designed to support test case set adequacy evaluation and to provide guidance in test case selection. If there is a previously developed test set, e.g. a functional test set, such a test set can be evaluated against the structural testing criteria implemented by JaBUTi. This allows the tester to check, for example, whether all instructions/statements are executed by a particular test set. If they are not yet executed, additional test cases can be developed to improve the quality of the previous test set.

On the other hand, if there is no previous test set available, the tester can use the hints provided by the tool to create test cases aiming at to cover the different testing requirements generated by JaBUTi based on its structural criteria. Once the test criteria can be applied incrementally, the tester can start by developing an adequate test set w.r.t. the **All-Nodes-ei** criterion, then, if necessary, to develop additional test cases to evolve the **All-Nodes-ei**-adequate test set to a **All-Edges-ei**-adequate test set, and later, if necessary, to develop additional test cases to evolve it to an **All-Uses-ei**-adequate test set. All these criteria, prefixed by **All-Pri-**, are related with the normal program execution. If desired, the tester can check the coverage of the exception-handling mechanism, by using the **All-Nodes-ed**, **All-Edges-ed** and **All-Uses-ed** criteria. The idea is to use these criteria incrementally to reduce their complexity and also to allow the tester to deal with the cost and time constraints.

JaBUTi allows to include test cases in two different ways: 1) using the JaBUTi's class loader; or 2) importing from a JUnit test set. The first is done by a command line application (**probe.ProberLoader**, the JaBUTi's class loader). This command line application extends the default Java class loader [11] such that, from a given testing project, it identifies which classes should be instrumented before to be loaded and executed. By detecting that a given class belongs to the set of classes under testing, JaBUTi's class loader inserts the probes to collect the execution trace and then loads the instrumented class file to be executed. The trace information w.r.t. the current execution is appended in a trace file with the same name of the testing project but with the extension **.trc** instead of **.jbt**.

For example, the test case file **input1**, presented in Figure 20, touches the testing requirement (considering the **All-Nodes-ei** criterion) with the highest weight illustrated in Figure 7 and also reveals a fault contained in the **Dispenser** component. Such a test case will be used in the next section to show how to use JaBUTi to ease the fault localization.

```
auri@AURIMRV ~/example
$ cat input1
insertCoin
vendItem 3
```

Figure 20: Example of test case file.

The command line in Figure 21(a) shows the output of the **vending.TestDriver** when executed by the default class loader. Figure 21(b) shows the resultant output when **vending.TestDriver** is executed using the JaBUTi's class loader. The execution of the command as shown in Figure 21(b) causes the **vending.trc** to be generated/updated with the execution path of test case file **input1**.

Considering Figure 21(b), observe that the **CLASSPATH** variable is set containing all the paths necessary to run the JaBUTi's class loader and also the vending machine example. The next parameter, **probe.ProberLoader**, is the name of the JaBUTi's class loader. It demands two parameters to execute: the name of the project (**-P vending.jbt**), and the name of the base class to be executed (**vending.TestDriver**). In our example, since **vending.TestDriver** requires one parameter to be executed,

|                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> auri@AURIMRV ~/example \$ java -cp "." vending.TestDriver input1 VendingMachine ON Current value = 25 Enter 25 coins Current value = -25 VendingMachine OFF </pre> | <pre> auri@AURIMRV ~/example \$ java -cp ".;..\Tools\jabuti;\ &gt;..\Tools\jabuti\lib\BCEL.jar;\ &gt;..\Tools\jabuti\lib\crimson.jar;\ &gt; probe.ProberLoader -P vending.jbt \ &gt; vending.TestDriver input1 Project Name: vending.jbt Trace File Name: vending.trc Processing File vending.jbt VendingMachine ON Current value = 25 Enter 25 coins Current value = -25 VendingMachine OFF </pre> |
| (a)                                                                                                                                                                      | (b)                                                                                                                                                                                                                                                                                                                                                                                                 |

Figure 21: vending.TestDriver output: (a) default class loader, and (b) JaBUTi class loader.

this parameter is also provided (input1). During the execution of vending.TestDriver, VendingMachine and Dispenser classes are required to be loaded. From the project file (vending.jbt) the JaBUTi's class loader detects that these class files have to be instrumented before to be executed. The instrumentation is performed on-the-fly every time the probe.ProberLoader is invoked.

The resultant execution trace information, corresponding to the execution of the test case input1, is appended in the end of the trace file vending.trc. Every time the size of the trace file increase, the Update button in the JaBUTi's graphical interface becomes red, indicating that the coverage information can be updated considering the new test case(s). Figure 22 illustrates this event.

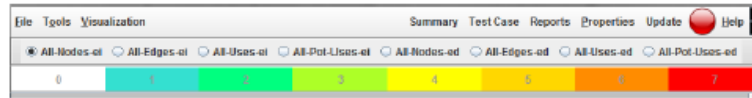


Figure 22: Update button with a different background color: new test case(s) available.

By clicking on the Update button, JaBUTi imports the new test case(s), empties the trace file vending.trc and updates the coverage information and the weights for each testing criterion. For example, after the importation of test case input1, the weight of the Java source code w.r.t. the All-Nodes-ei criterion changed as illustrated in Figure 23.

Comparing the updated source of Figure 23 (front) with the one presented in Figure 23 (back) it can be observed that the requirement with the highest changed from source lines 22 and 24 (now covered – painted in white) to source lines 16, 18, 20 and 27. Considering the entire project, Figure 24(b) shows the updated coverage for each method in the Dispenser and VendingMachine classes w.r.t. the All-Nodes-ei criterion. Observe that input1 covered 7 primary nodes (63%) of Dispenser.dispense method and 19 of 29 primary nodes (65%) considering the entire project. By accessing the Test Case → Report by Test Case menu option, the tester can visualize the coverage of the entire project by test case, as illustrated in Figure 24(a).

If desired, the tester can develop additional test cases to improve the coverage w.r.t. All-Nodes-ei criterion. For example, besides input1, Table 1 shows four additional test cases developed to improve the coverage of such a criterion.

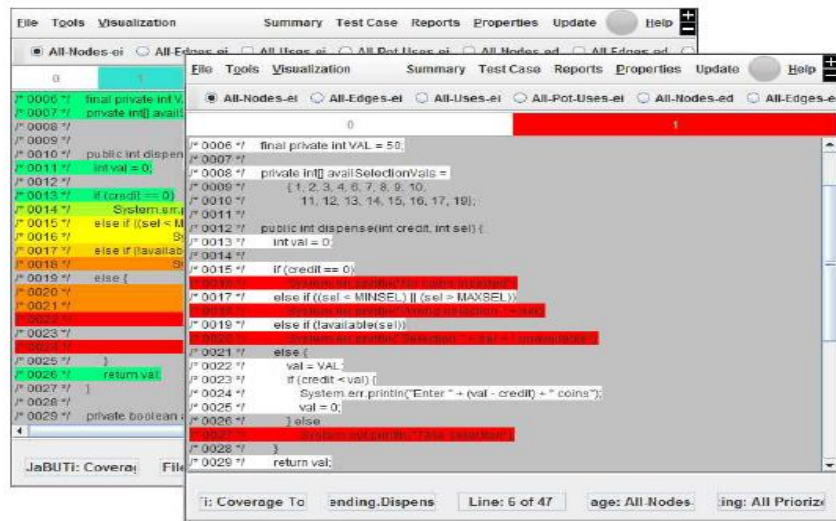


Figure 23: Dispenser.dispenser() method before and after the weight's updating w.r.t. All-Pri-Nodes criterion.

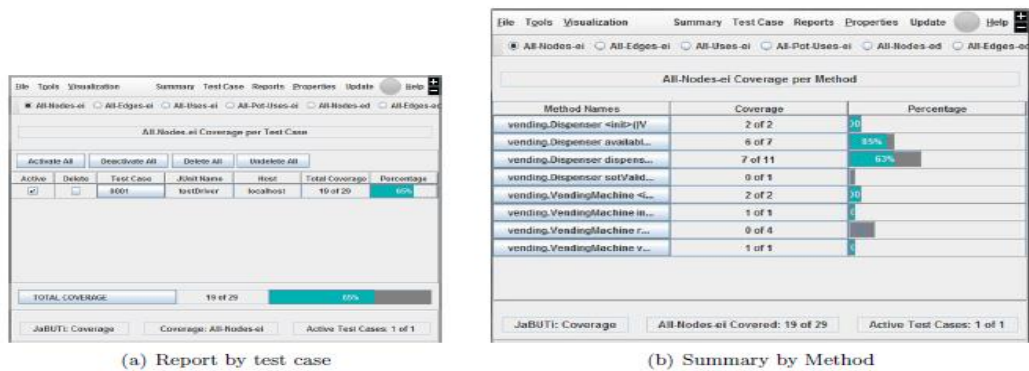


Figure 24: Updated coverage after test case input1.

Table 1: Adequate test set to the Dispenser component.

| Name   | Content                                 | Correct output |
|--------|-----------------------------------------|----------------|
| input1 | insertCoin<br>vendItem 3                | no             |
| input2 | insertCoin<br>vendItem 18               | yes            |
| input3 | insertCoin<br>insertCoin<br>vendItem 25 | yes            |
| input4 | vendItem 3                              | yes            |
| input5 | insertCoin<br>insertCoin<br>vendItem 3  | yes            |

### 3.5 How to import test cases from JUnit framework

Observe that after five test cases, the coverage of almost all methods, w.r.t. the All-Nodes-ei criterion, is 100%. The only two methods uncovered are `Dispenser.setValidSelection()` and `VendingMachine.returnCoin()`. To obtain a adequate test set w.r.t. the All-Nodes-ei, additional test cases are required. Figure 25 shows one additional test case, specified according to the JUnit framework, to cover 100% of required elements of the All-Nodes-ei for the `Dispenser` class. More information about how to develop test set using the JUnit framework can be found elsewhere [3, 18].

```
import org.junit.Before;
import org.junit.Test;

import br.jabuti.probe.DefaultProber;

public class DispenserTestCase {
    protected Dispenser d;

    @Before
    protected void setUp() throws Exception {
        d = new Dispenser();
    }

    @Test
    public void testDispenserException() {
        int val;

        d.setValidSelection( null );
        val = d.dispense( 50, 10 );
        assertEquals( 0, val );
    }

    @After
    public void tearDown() {
        DefaultProber.dump();
    }
}
```

Figure 25: Example of a test set using the JUnit framework.

As can be observed in Figure 25, JaBUTi requires that a static method (`DefaultProber.dump()`) to be called at the end of each test case. Such a method is responsible to write the execution trace in the trace file. Observe that to call such a static method it is also necessary to include the import statement for the class. Once compiled, such a JUnit test case can be imported by JaBUTi from the Test Case → Import from JUnit menu option. Figure 26 shows the screen to import a JUnit's test case.

The fields to be entered are:

- **Path to application:** automatically filled by JaBUTi.
- **Path to JUnit test suite source/binary code:** if you followed the manual it should be your current path; otherwise it's the path to your application's source and binary codes.
- **Test suite full qualified name:** the full name of the JUnit test which will be used.
- **JaBUTi's library:** the .jar used to run JaBUTi.
- **Other application specific libraries:** other libraries your application may need.
- **javac:** full path to your java compiler.
- **Trace file name:** automatically filled by JaBUTi.

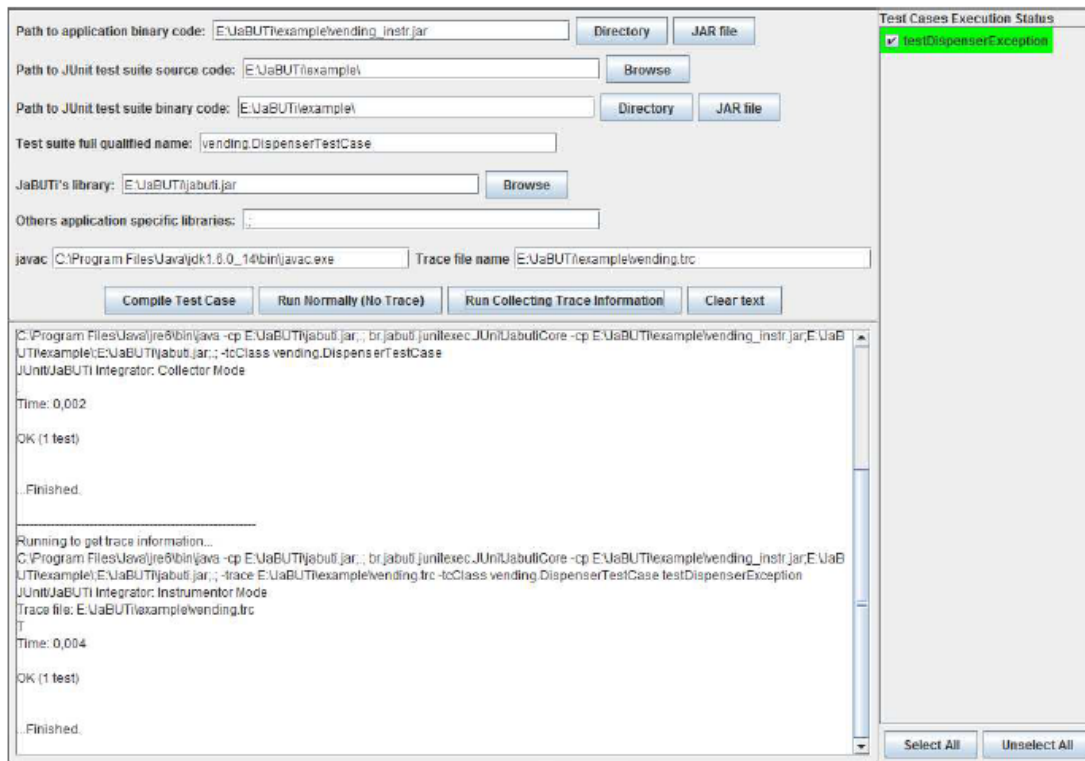


Figure 26: Importing a JUnit Test Case.

If everything is correct, you should be able to (1) compile the test case, (2) run it normally (to make sure it works) and (3) run it collecting trace information.

In our example (Figure 3.5), there is only one test case to be imported. Observe that the name of such a test case `testDispenserException` is highlighted in green. This is because JUnit framework provides several assertions statements, like the one at line 22 of Figure 25, that allows to verify if the test case's output is according to its specification or not. We use a green background color to represent test cases that behave accordantly to their specifications and a red background color to represent the ones that do not.

As soon as JaBUTi detects a new test case execution trace is appended in the end of the trace file, the **Update** button becomes red indicating that the coverage needs to be updated. Updated the coverage information, Figure 27 shows the summary report by methods and the corresponding coverage w.r.t. the **All-Nodes-ei** criterion. The summary report by criterion can be seen in Figure 28.

After the execution of these six test cases, all methods of the `Dispenser` component are 100% covered w.r.t. the **All-Nodes-ei** criterion, but this test set is adequate to the `VendingMachine` class. More specifically, the `VendingMachine.returnCoin` method is not covered yet. So, after six test cases, the coverage of the entire project w.r.t. **All-Nodes-ei** and **All-Nodes-ed** are 86% and 100%, respectively. The coverage for the **All-Edges-ei** and the **All-Edges-ed** are 82% and 20%, respectively, and the coverage for the **All-Uses-ei** and **All-Uses-ed** are 82% and 0%, respectively.

Independently of the number of test cases available, if desired, the tester can enable/disable any combination of test cases by accessing the **Test Case → Report By Test Case** menu option and choosing

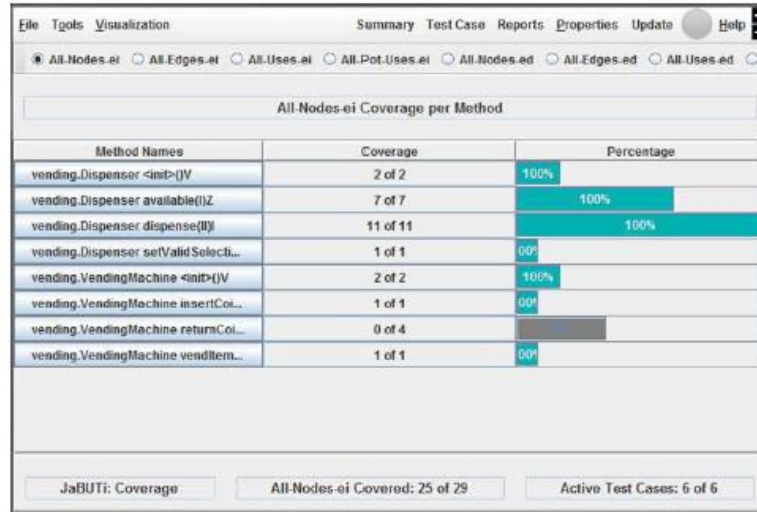


Figure 27: Updated coverage after six test cases w.r.t. the All-Pri-Nodes criterion.

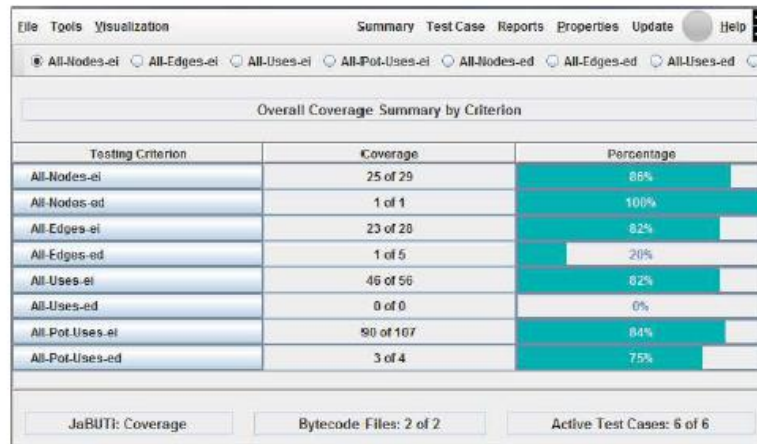


Figure 28: Updated coverage after six test cases w.r.t. all criteria.

which test case should be enabled/disabled. Once a different subset of test cases are enabled/disabled all the coverage information should be updated by clicking on the Update button. Figure 29(a) shows the complete test set with all test cases enabled. Figure 29(b) shows the updated coverage considering only the test cases number 2 and 5.

Using the resource of activate/deactivate test cases, the tester can evaluate how the coverage changes, considering different combinations of test case. Moreover, besides activate/deactivate test cases, the tester can also mark a test case to be deleted. Once a test case is marked to be deleted it is only logically deleted. Such a test case is physically deleted when the project is saved and closed. While the project is not closed, a test case marked as deleted can be undeleted.



Figure 29: Summary by test case: (a) all enabled, and (b) 2 and 5 enabled.

**Experiment 3.** A program written in java to compute the factorial of a given integer number study its test cases import the test cases from Junit .  
Consider the following method which should compute the factorial of a given integer number.

```
public class Factorial {

    public long compute(int x) throws NegativeNumberException {
        if (x < 0) {
            long r = 1;
            for (int k = 2 ; k <= x ; k++) {
                r += k ;
            }
            return r ;
        } else {
            throw new NegativeNumberException() ;
        }
    }

    public long computeRecursive(int x)
    {
        if (x <= 2) {
            if (x == 0) {
```

```

        return 1;
    } else {
        return x;
    }
} else {
    return x * compute(x - 1);
}
}
}

```

The first fault located at line 4 (the correct should be  $x \geq 0$ ) is responsible for a domain fault because, regardless of the value of  $x$ , a wrong set of statements is executed. The second fault (supposing the first one is corrected) located at line 7 (the correct should be  $r *= k$ ) is responsible for a computational fault since, except for negative values and for  $x = \{0, 1, 3\}$ , the method computes the factorial function incorrectly. Observe that this is also an example of data sensible fault since for  $x = 3$  the fault is activated but the correct value of  $3! = 6$  is computed. Therefore, in order to detect the fault, a value of  $x$  different from  $\{0, 1, 3\}$  should be used.

### Factorial test cases

```

factorial package ;

import junit.framework.* ;

public class FactorialComputeTestCase extends TestCase
{
    protected Factorial fat ;

    public FactorialComputeTestCase () {
        this ( new String ());
    }
}

```

```

    }

    public FactorialComputeTestCase ( String str ) {

        super ( str );

        fat = new Factual ();

    }

    public void testCompute1 () {

        long result = fat . compute ( 0 );

        assertEquals ( result , 1 );

    }

    public void testCompute2 () {

        long result = fat . compute ( 2 );

        assertEquals ( result , 2 );

    }

    public void testCompute3 () {

        long result = fat . compute ( 11 );

        assertEquals ( result , 39916800 );

    }

}

```

**Experiment4.** Write the program and study of their possible test cases

programs to find  
a)FindLast  
b)LastZero  
c)CountPositive  
d)NumZero

e)OddOrPos

**i) FindLast**

```
/**
 * If x == null throw NullPointerException, else return the index
 * of the last element in x that equals y. If no such element
 * exists, return -1.
 */
public int findLast(int[] x, int y)
{
    for (int i=x.length-1; i > 0; i--) {
        if (x[i] == y) {
            return i;
        }
    }
    return -1;
}
```

1. Identify the fault for the test case {Input: x = [2, 3, 5], y = 2; Expected output = 0}.
2. If possible, identify a test case that does not execute the fault.
3. If possible, identify a test case that executes the fault, but does not result in an error state.
4. If possible identify a test case that results in an error, but not a failure. Hint: Don't forget about the program counter.
5. For the given test case, identify the first error state. Be sure to describe the complete state.
6. Fix the fault and verify that the given test now produces the expected output.

**ii) LastZero**

```
/**
 * If x == null throw NullPointerException, else return the index of the
 * last 0 in x. Return -1 if 0 does not occur in x.
```

```

*/
public static int lastZero(int[] x)
{
    for (int i = 0; i < x.length; i++) {
        if (x[i] == 0) {
            return i;
        }
    }
    return -1;
}

```

1. Identify the fault for the test case {Input: x = [0, 1, 0]; Expected output = 2}.
2. If possible, identify a test case that does not execute the fault.
3. If possible, identify a test case that executes the fault, but does not result in an error state.
4. If possible identify a test case that results in an error, but not a failure. Hint: Don't forget about the program counter.
5. For the given test case, identify the first error state. Be sure to describe the complete state.
6. Fix the fault and verify that the given test now produces the expected output.

### iii)CountPositive

```

/**
 * If x == null throw NullPointerException, else return the number
 * of positive elements in x.
 */
public int countPositive(int[] x) {
    int count = 0;
    for (int i=0; i < x.length; i++) {
        if (x[i] >= 0) {
            count++;
        }
    }
    return count;
}

```

```

        }
    }
    return count;
}

```

1. Identify the fault for the test case {Input: x = [-4, 2, 0, 2]; Expected output = 2}.
2. If possible, identify a test case that does not execute the fault.
3. If possible, identify a test case that executes the fault, but does not result in an error state.
4. If possible identify a test case that results in an error, but not a failure. Hint: Don't forget about the program counter.
5. For the given test case, identify the first error state. Be sure to describe the complete state.
6. Fix the fault and verify that the given test now produces the expected output.

#### iv)NumZero

```

class numZero
{
    /**
     * If arr is null throw NullPointerException, else return the
     * number of occurrences of zero in arr
     */
    public static int numZero (int[] arr)
    {
        int count = 0;
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] == 0) {
                count++;
            }
        }
        return count;
    }
}

```

```

    }

    /**
     * Driver method for numZero.
     */
    public static void main(String[] argv)
    {
        // Read an array from standard input, call numZero()
        int[] inArr = new int[argv.length];
        if (argv.length == 0) {
            System.out.println("Usage: java numZero v1 [v2] [v3] ... ");
            return;
        }

        for (int i = 1; i < argv.length; i++) {
            try {
                inArr[i] = Integer.parseInt(argv[i]);
            } catch (NumberFormatException e) {
                System.out.println ("Entry must be a integer, using 1.");
                inArr[i] = 1;
            }
        }

        System.out.println("Number of zeros is: " + numZero(inArr));
    }
}

```

The fault in this program is that it starts looking for zeroes at index 1 instead of index 0, as is necessary for arrays in Java. For example, `numZero([2, 7, 0])` correctly evaluates to 1, while `numZero([0, 7, 2])` incorrectly evaluates to 0. In both of these cases the fault is executed. Although both of these cases result in an error, only the second case results in failure. To

understand the error states, we need to identify the state for the program. The state for numZero consists of values for the variables x, count, i, and the program counter (denoted PC). For the first example given above, the state at the if statement on the very first iteration of the loop is (x = [2, 7, 0], count = 0, i = 1, PC = if). Notice that this state is in error precisely because the value of i should be zero on the first iteration. However, since the value of count is coincidentally correct, the error state does not propagate to the output, and hence the software does not fail. In other words, a state is in error simply if it is not the expected state, even if all of the values in the state, considered in isolation, are acceptable. More generally, if the required sequence of states is s0 , s1 , s2 , ..., and the actual sequence of states is s0 , s2 , s3 , ..., then state s2 is in error in the second sequence.

In the second case the corresponding error state is (x = [0, 7, 2], count = 0, i = 1, PC = if). In this case, the error propagates to the variable count and is present in the return value of the method. Hence a failure results.

Regarding the numZero method, some test cases that could be designed for this code are:

- ([2, 7, 0], 1)
- ([0, 7, 2], 1)
- ([], 0)
- ([0, 0, 0], 3)

## v)OddOrPos

```
/**
 * If x == null throw NullPointerException, else return the number of
 * elements in x that are either odd or positive (or both).
 */
public static int oddOrPos(int[] x)
{
    int count = 0;
    for (int i = 0; i < x.length; i++) {
        if (x[i] % 2 == 1 || x[i] > 0) {
            count++;
        }
    }
}
```

```
        return count;
    }
```

1. Identify the fault for the test case {Input: x = [-3, -2, 0, 1, 4; Expected output = 3]}.
2. If possible, identify a test case that does not execute the fault.
3. If possible, identify a test case that executes the fault, but does not result in an error state.
4. If possible identify a test case that results in an error, but not a failure. Hint: Don't forget about the program counter.
5. For the given test case, identify the first error state. Be sure to describe the complete state.
6. Fix the fault and verify that the given test now produces the expected output.

#### Experiment 5. Write a program to study the sorting and draw a control flow graph

Consider a sort implementation for a array of integers. The input of the sort application, the array, is set as the argument of the application, and the result is printed to the console. For example, for the input array '3 1 7 4', the result would be:

```
$ sort 3 1 7 4 1 3 7 4
```

Thus, a possible set of test cases is:

- <[3, 1, 7, 4], [1, 3, 7, 4]>
- <[1, 2, 3, 4], [1, 2, 3, 4]>
- <[0, 1, 0, 2], [0, 0, 1, 2]>
- <[], []>
- <[a, b, 0, 1], error message printed to console>

#### Bubble sort

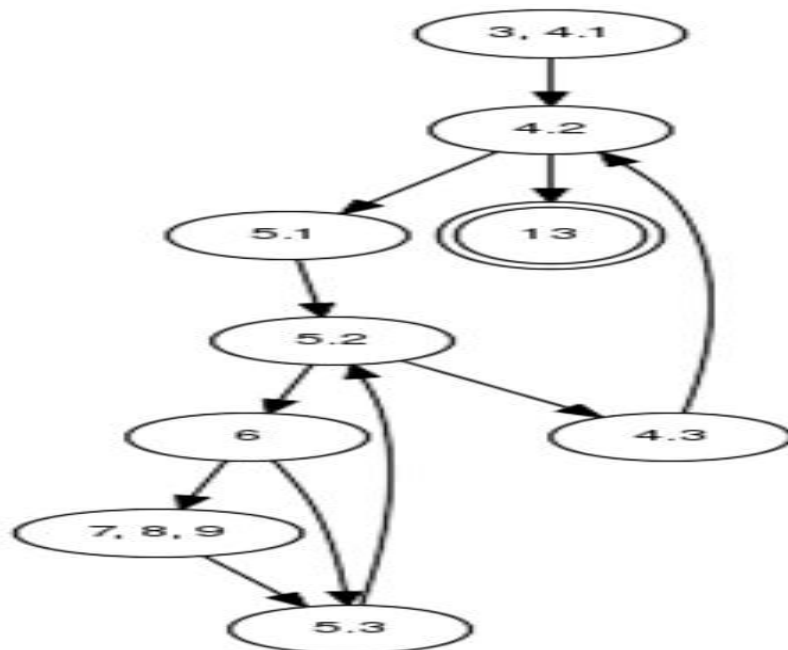
```
public void bubble ( int [ ] a , int size ) {
    int i , j , aux ;
    for ( i = 0 ; i < size ; i ++ ) {
        for ( j = size - 1 ; j > i ; j - ) {
```

```

    if ( a [ j - 1 ] > a [ j ] ) {
        aux = a [ j - 1 ];
        a [ j - 1 ] = a [ j ];
        a [ j ] = aux ;
    }
}
}
}
}

```

### Control flow graph



### Viva Questions

Q1. What is System Testing?

Testing that attempts to discover defects that are properties of the entire system rather than of its individual components.

Q2. What is Testability?

The degree to which a system or component facilitates the establishment of test criteria and the performance of tests to determine whether those criteria have been met.

Q3. What is Test Bed?

An execution environment configured for testing. May consist of specific hardware, OS, network topology, configuration of the product under test, other application or system software, etc. The Test Plan for a project should enumerate the test beds(s) to be used.

Q4. What is Test Driver?

A program or test tool used to execute a tests. Also known as a Test Harness.

What is Test Environment?

The hardware and software environment in which tests will be run, and any other software with which the software under test interacts when under test including stubs and test drivers.

Q5. What is Test First Design?

Test-first design is one of the mandatory practices of Extreme Programming (XP). It requires that programmers do not write any production code until they have first written a unit test.

Q6. What is Test Plan?

A document describing the scope, approach, resources, and schedule of intended testing activities. It identifies test items, the features to be tested, the testing tasks, who will do each task, and any risks requiring contingency planning. Ref IEEE Std 829.

Q7. What is Test Procedure?

A document providing detailed instructions for the execution of one or more test cases.

Q8. What is Test Script?

Commonly used to refer to the instructions for a particular test that will be carried out by an automated test tool.

Q9. What is Test Specification?

A document specifying the test approach for a software feature or combination of features and the inputs, predicted results and execution conditions for the associated tests.

Q10. What is Test Suite?

A collection of tests used to validate the behavior of a product. The scope of a Test Suite varies from organization to organization. There may be several Test Suites for a particular product for example. In most cases however a Test Suite is a high level concept, grouping together hundreds or thousands of tests related by what they are intended to test.

Q11. What is Test Tools?

Computer programs used in the testing of a system, a component of the system, or its documentation.

Q12. What is Thread Testing?

A variation of top-down testing where the progressive integration of components follows the implementation of subsets of the requirements, as opposed to the integration of components by successively lower levels.

## Experiment 6.

---

Create a test set for the following application:

---

- The software read three integer values that represents the edges of a triangle. For those values, the software should write if the triangle is isosceles, scalene, or equilateral. The input values should obey a condition: the sum of two edges must be bigger than the third edge.

Hints:

- There is a test case for a valid scalene triangle?
- There is a test case for a valid isosceles triangle?
- There is a test case for a valid equilateral triangle?
- There is at least three test cases for valid isosceles triangles (with the permutation of the same values)?
- There is a test case with a value of zero?
- There is a test case with a negative value?
- There is a test case which the sum of two edges is equal to the third edge?
- There is a test case which the sum of two edges is equal to the third edge for each values permutation?
- There is a test case which the sum of two edges is less than the third edge?
- There is a test case which the sum of two edges is less than the third edge for each values permutation?
- There is a test case which all the values are zero?
- There is a test case for non-integer values?
- There is a test case with insufficient values (e.g., 2 instead of 3)?
- For every test case, have you specified the expected output for the specified input?

## Identifier test requirements

---

### Equivalence partition

| Input conditions               | Valid classes                       | Invalid classes |
|--------------------------------|-------------------------------------|-----------------|
| Identifier's size t            | 1 <= t <= 6 (1)t < 1 (2), t > 6 (3) |                 |
| First character c is a letter  | Yes (4)                             | No (5)          |
| Only contains valid characters | Yes (6)                             | No (7)          |

### All-Nodes

- 6

### **All-Edges**

- (8, 12)

### **Boundary-interior**

- (4, 5, 6, 7, 4)

### **All-Paths**

- (1, 2, 3, 4, 8, 12)

### **All-Defs**

- valid\_id = false

### **All-P-Uses**

- while (i < s.length() - 1)

### **All-C-Uses**

- i++

### **Data flow testing**

- {(s, 1, 3), (valid\_id, 1, (8, 9,)), (valid\_id, 1, (8, 12,))
- (valid\_id, 1, (8, 9,)) is infeasible.

### **All-Pot-Uses**

- (s, 1, 6), (achar, 3, (8, 9,)), and (achar, 3, (8, 12,))

---

### **Identifier test sets**

#### **Equivalence partition adequate**

#### **Input      Output**

a1      Valid (1, 4, 6)

Invalid (2)

A1b2C3dInvalid (3)

2B3 Invalid (5)

Z#12 Invalid (7)

### **Boundary analysis adequate**

#### **IdentifierResultDescription**

|         |         |                                                |
|---------|---------|------------------------------------------------|
| a       | Valid   | First valid lower case, min. size              |
| b3      | Valid   | Second valid lower case, above min. size       |
| Xkl     | Valid   | Last but one valid upper case, valid size      |
| Z9      | Valid   | Last valid upper case, above min. size         |
| xkl     | Valid   | Last but one valid lower case, valid size      |
| zAaZ1   | Valid   | Last but one valid lower case, below max. size |
| AaZz91  | Valid   | First valid upper case, max. size              |
| BaZz91  | Valid   | Second valid upper case, max. size             |
| abcdefg | Invalid | Valid characters, above max. size              |
| @       | Invalid | First char. below min. (A), min. size          |
| [       | Invalid | First char. above max. (Z), valid size         |
| `       | Invalid | First char. below min. (a), min. size          |
| {       | Invalid | First char. above max. (z), max. size          |
| A-&\$#  | Invalid | Invalid characters, max. size                  |

### **All-nodes adequate**

#### **InputOutput**

1#@ Invalid (1, 2, 3, 4, 5, 6, 7, 8 12)

i Valid (1, 2, 8, 9, 10, 11)

### All-edges adequate

Edge (9, 12) is infeasible.

#### Input    Output

Invalid (1, 8)

A1b2C3dInvalid {(5, 7), (10, 12)}

### All-uses adequate

For the Java version:

- (i, 3, 5)
- (i, 3, 7)
- (valid\_id, 1, (8,9))
- (valid\_id, 1, (8,12))
- ..

### All-Potential-Uses

For the Java version:

- (s, 1, 6)
- (achar, 3, (8, 9))
- (achar, 3, (8, 12))
- ...

## II. JMETER

**Objective:** Analyse the performance of following website using JMeter.

|             |                    |               |
|-------------|--------------------|---------------|
| Site Amazon | Website Amazon.com | Type shopping |
| Flip kart   | Flipkart.com       | shopping      |

|                     |             |                     |
|---------------------|-------------|---------------------|
| Railway reservation | Irctc.co.in | Ticket booking site |
| Train searching     | Erail.in    | Train searching     |

**JMeter:** jMeter is designed to cover various categories of tests such as load testing, functional testing, performance testing, regression testing, etc.

- **Performance Test** – This test sets the best possible performance expectation under a given configuration of infrastructure. It also highlights early in the testing process if any changes need to be made before the application goes into production.
- **Load Test** – This test is basically used for testing the system under the top load it was designed to operate under.
- **Stress Test** – This test is an attempt to break the system by overwhelming its resources.

### What is JMeter?

JMeter is software that can perform load test, performance-oriented business (functional) test, regression test, etc., on different protocols or technologies.

JMeter is a Java desktop application with a graphical interface that uses the Swing graphical API. It can therefore run on any environment / workstation that accepts a Java virtual machine, for example – Windows, Linux, Mac, etc.

The protocols supported by JMeter are –

- Web – HTTP, HTTPS sites 'web 1.0' web 2.0 (ajax, flex and flex-ws-amf)
- Web Services – SOAP / XML-RPC
- Database via JDBC drivers
- Directory – LDAP
- Messaging Oriented service via JMS
- Service – POP3, IMAP, SMTP
- FTP Service

### System Requirement

|            |               |
|------------|---------------|
| <b>JDK</b> | 1.6 or above. |
|------------|---------------|

|                         |                         |
|-------------------------|-------------------------|
| <b>Memory</b>           | No minimum requirement. |
| <b>Disk Space</b>       | No minimum requirement. |
| <b>Operating System</b> | No minimum requirement. |

## **Elements of JMeter**

**Test Plan-** A test plan describes a series of steps JMeter will execute when run. A complete test plan will consist of one or more Thread Groups, logic controllers, sample generating controllers, listeners, timers, assertions, and configuration elements

**Thread group-** Thread group elements are the beginning points of any test plan. All controllers and samplers must be under a thread group. Other elements, e.g. Listeners, may be placed directly under the test plan, in which case they will apply to all the thread groups. As the name implies, the thread group element controls the number of threads JMeter will use to execute your test. Set the ramp-up period. Set the number of times to execute the test

**Controller-** JMeter has two types of Controllers Samplers and Logical Controllers. These drive the processing of a test. Samplers tell JMeter to send requests to a server. For example, add an HTTP Request Sampler if you want JMeter to send an HTTP request. Logical Controllers let you customize the logic that JMeter uses to decide when to send requests.

**Sampler-** Samplers tell JMeter to send requests to a server and wait for a response. They are processed in the order they appear in the tree. Controllers can be used to modify the number of repetitions of a sampler, JMeter samplers include FTP Request ,HTTP Request ,JDBC Request . ,Java object request ,LDAP Request ,SOAP/XML-RPC Request, WebService (SOAP) Request . Each sampler has several properties you can set.

**Listeners** - Listeners provide access to the information JMeter gathers about the test cases while JMeter runs. The Graph Results listener plots the response times on a graph. The "View Results Tree" Listener shows details of sampler requests and responses, and can display basic HTML and XML representations of the response. Other listeners provide summary or aggregation information.

**Configuration tool-** A configuration element works closely with a Sampler. Although it does not send requests (except for HTTP Proxy Server ), it can add to or modify requests.

## **Installation of JMeter Tool**

### **Step 1: Verify Java Installation**

First of all, verify whether you have Java installed in your system. Open your console and execute one of the following **java** commands based on the operating system you are working on.

| OS      | Task                 | Command            |
|---------|----------------------|--------------------|
| Windows | Open Command Console | c:\> java -version |

### **Step 2: Set Java Environment**

Set the **JAVA\_HOME** environment variable to point to the base directory location, where Java is installed on your machine. For example –

| OS      | Output                                                                      |
|---------|-----------------------------------------------------------------------------|
| Windows | Set the environment variable JAVA_HOME to C:\Program Files\Java\jdk1.7.0_25 |

### **Step 3: Download JMeter**

Download the latest version of JMeter from [https://jmeter.apache.org/download\\_jmeter.cgi](https://jmeter.apache.org/download_jmeter.cgi). For this tutorial, we downloaded *apache-jmeter-2.9* and copied it into C:\>JMeter folder.

The directory structure should look like as shown below –

- apache-jmeter-2.9
- apache-jmeter-2.9\bin
- apache-jmeter-2.9\docs
- apache-jmeter-2.9\extras
- apache-jmeter-2.9\lib\
- apache-jmeter-2.9\lib\ext
- apache-jmeter-2.9\lib\junit

- apache-jmeter-2.9\printable\_docs

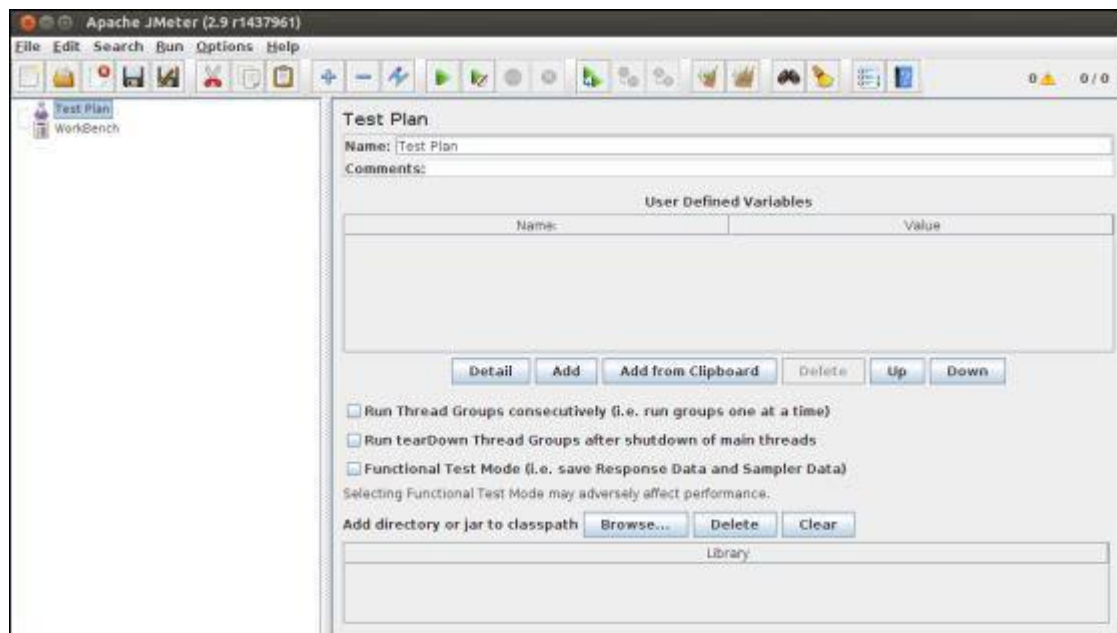
You can rename the parent directory (i.e. apache-jmeter-2.9) if you want, but do not change any of the sub-directory names.

#### Step 4: Run JMeter

After downloading JMeter, go to the *bin* directory. In this case, it is **/home/manisha/apache-jmeter-2.9/bin**. Now click on the following –

| OS      | Output     |
|---------|------------|
| Windows | jmeter.bat |

After a short pause, the JMeter GUI should appear, which is a Swing application, as seen in the following screenshot –



### Experiment-7

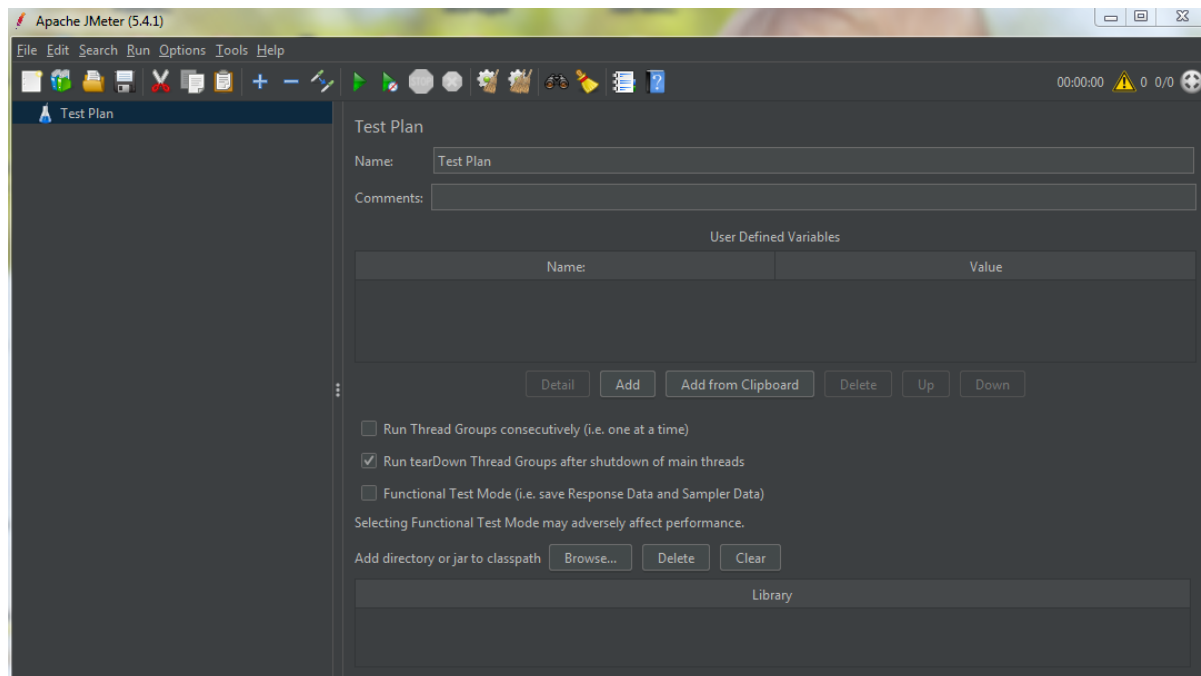
**Objective-** Analyse the performance of amazon website using JMeter.

## **Procedure**

1. Start JMeter
2. Rename the test plan and change the name test plan as *Amazon*
3. Add thread group
  - (a) Right-click the element where you want to add a child element node.
  - (b) Choose the appropriate option to add.
  - (c) Right-click on the Amazon (our Test Plan) → Add → Threads (Users) → Thread Group. Thus, the Thread Group gets added under the Test Plan (Amazon) node.
4. Add sampler
  - (a) Right-clicking and we will add HTTP Request Sampler by choosing Add → Sampler → HTTP request option.
5. Add listener
  - (a) Open the context menu
  - (b) Right-click the Thread Group (Users)
  - (c) Choose Add → Listener → View Results Tree option
6. Run the test plan
7. Save the test plan
8. View the output

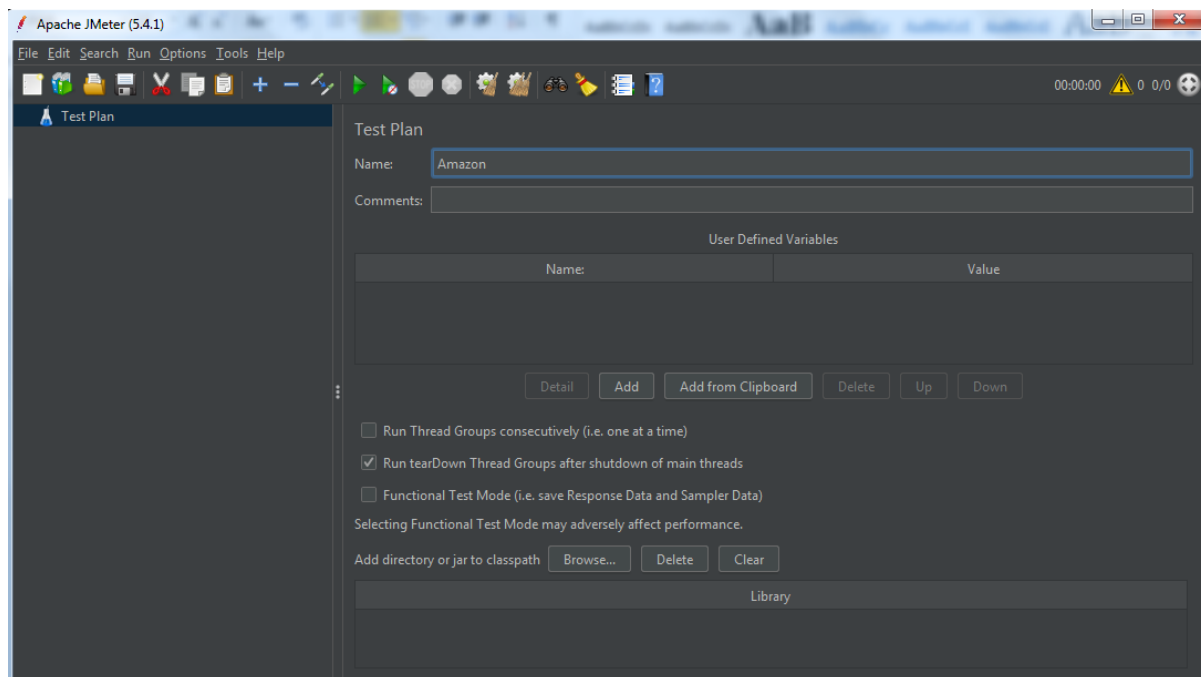
### **Step 1: Start JMeter**

Open the JMeter window



## Step 2: Rename the Test Plan

Change the name of test plan node to *Amazon* in the *Name* text box. You need to change the focus to workbench node and back to the Test Plan node to see the name getting reflected.

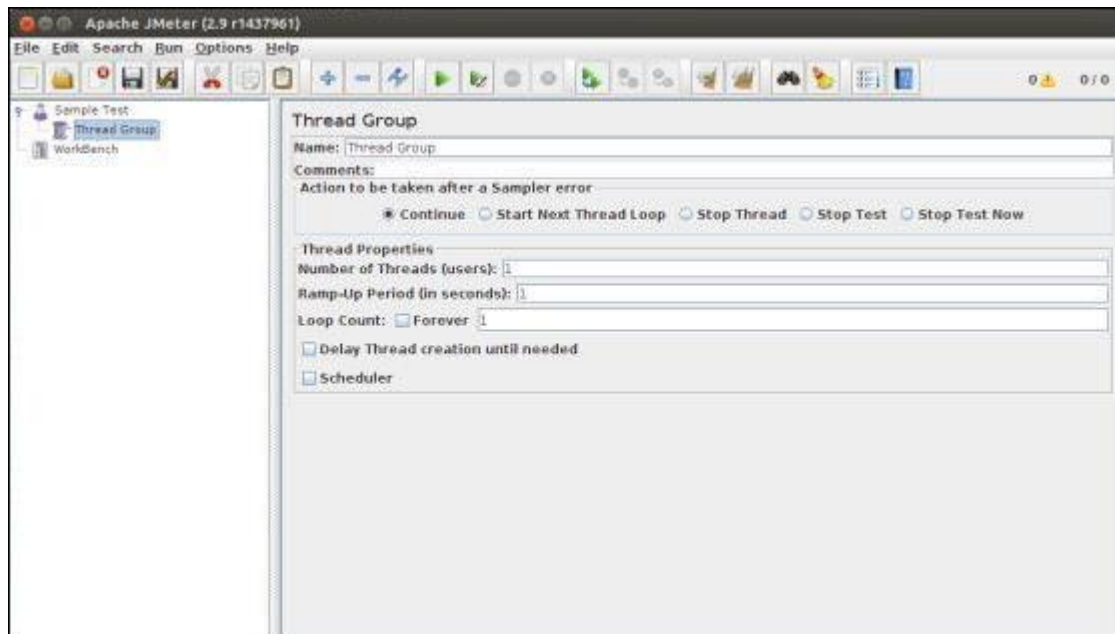


## Step 3: Add Thread Group

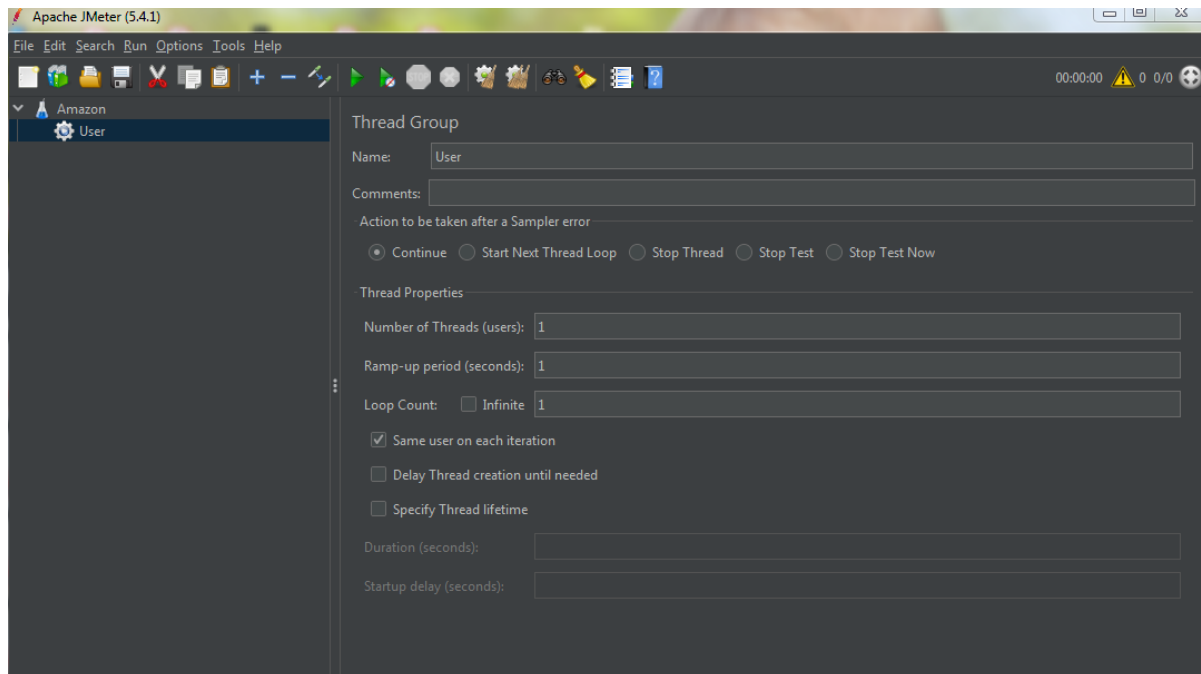
Now we add our first element in the window. We add one Thread Group, which is a placeholder for all other elements like Samplers, Controllers, and Listeners. We need one so we can configure number of users to simulate.

In JMeter, all the node elements are added by using the context menu.

- Right-click the element where you want to add a child element node.
- Choose the appropriate option to add.
- Right-click on the Amazon (our Test Plan) → Add → Threads (Users) → Thread Group. Thus, the Thread Group gets added under the Test Plan (Amazon) node.

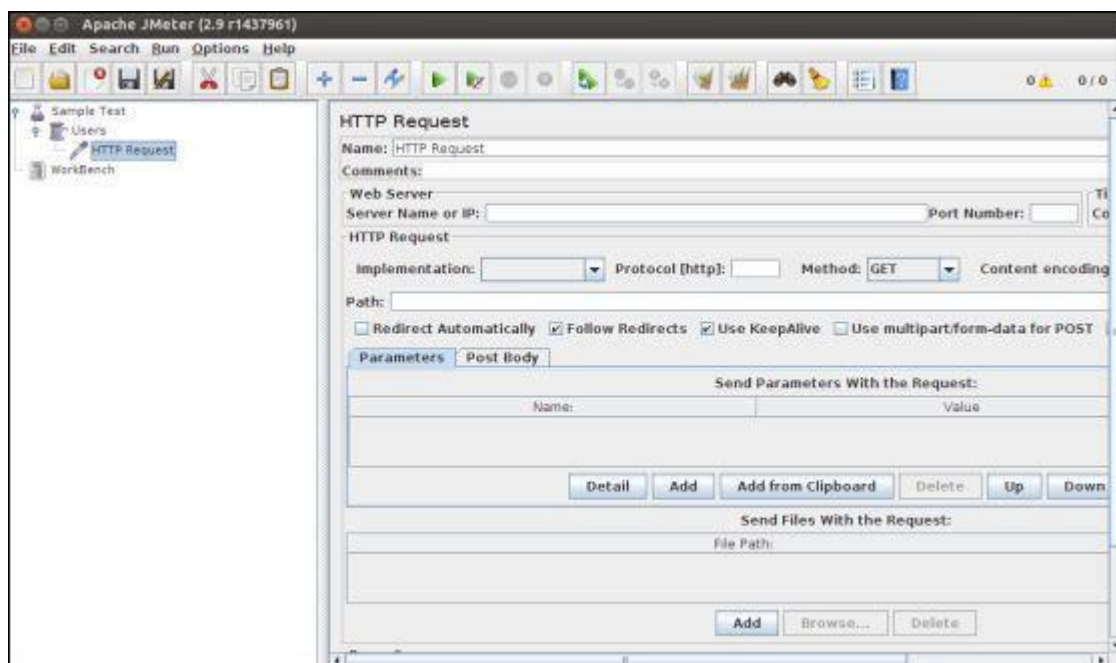


- Name the Thread Group as *Users*. For us, this element means users visiting the Amazon Home Page.

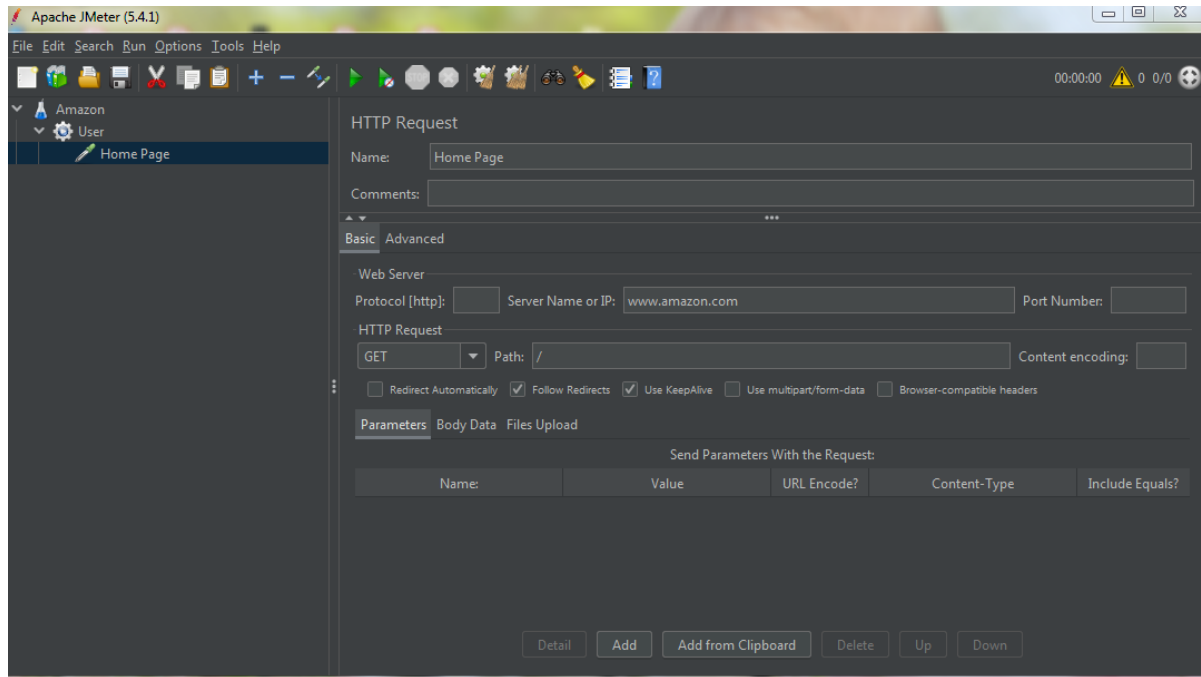


#### Step 4: Add Sampler

We need to add one Sampler in our Thread Group (Users). As done earlier for adding Thread group, this time we will open the context menu of the Thread Group (Users) node by right-clicking and we will add HTTP Request Sampler by choosing Add → Sampler → HTTP request option.



It will add one empty HTTP Request Sampler under the Thread Group (Users) node. Let us configure this node element –



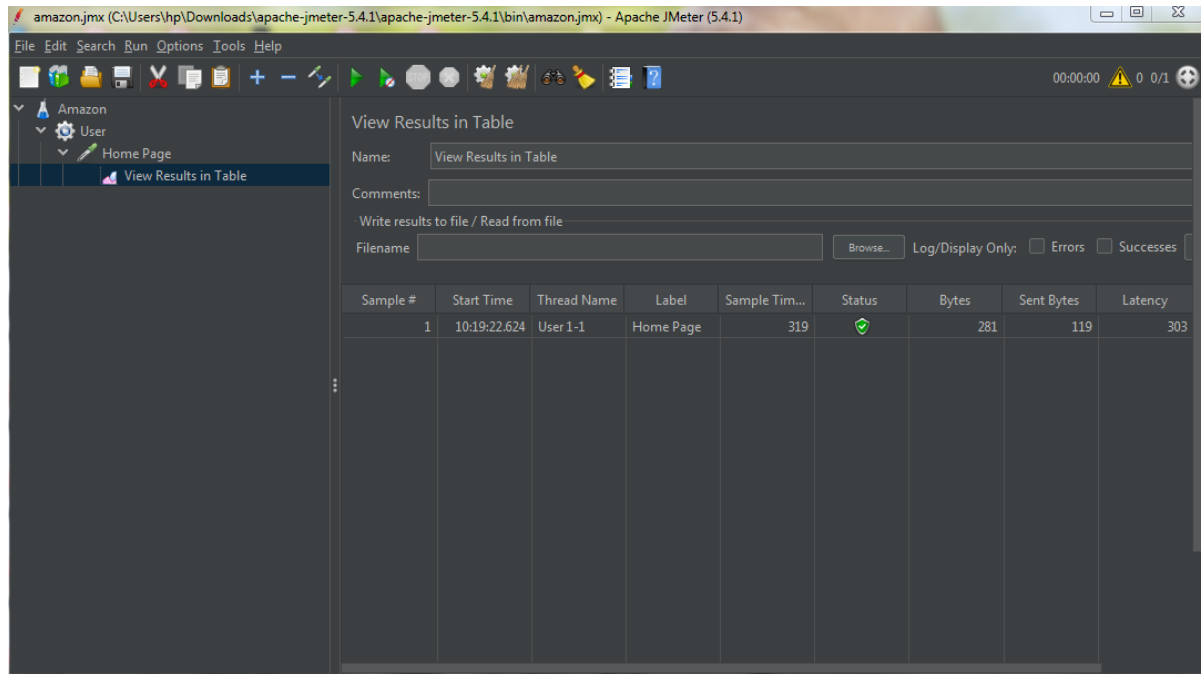
- **Name** – We will change the name to reflect the action what we want to achieve. We will name it as for example **Visit Amazon Home Page**
- **Server Name or IP** – Here, we have to type the web server name. In our case it is **www.amazon.com**. (http:// part is not written this is only the name of the server or its IP)
- **Protocol** – We will keep this blank, which means we want HTTP as the protocol.
- **Path** – We will type path as / (slash). It means we want the root page of the server.

### Step 5: Add Listener

We will now add a listener. Let us add View Results Table Listener under the Thread Group (User) node. It will ensure that the results of the Sampler will be available to view in this Listener node element.

To add a listener –

- Open the context menu
- Right-click the Thread Group (Users)
- Choose Add → Listener → View Results Table option

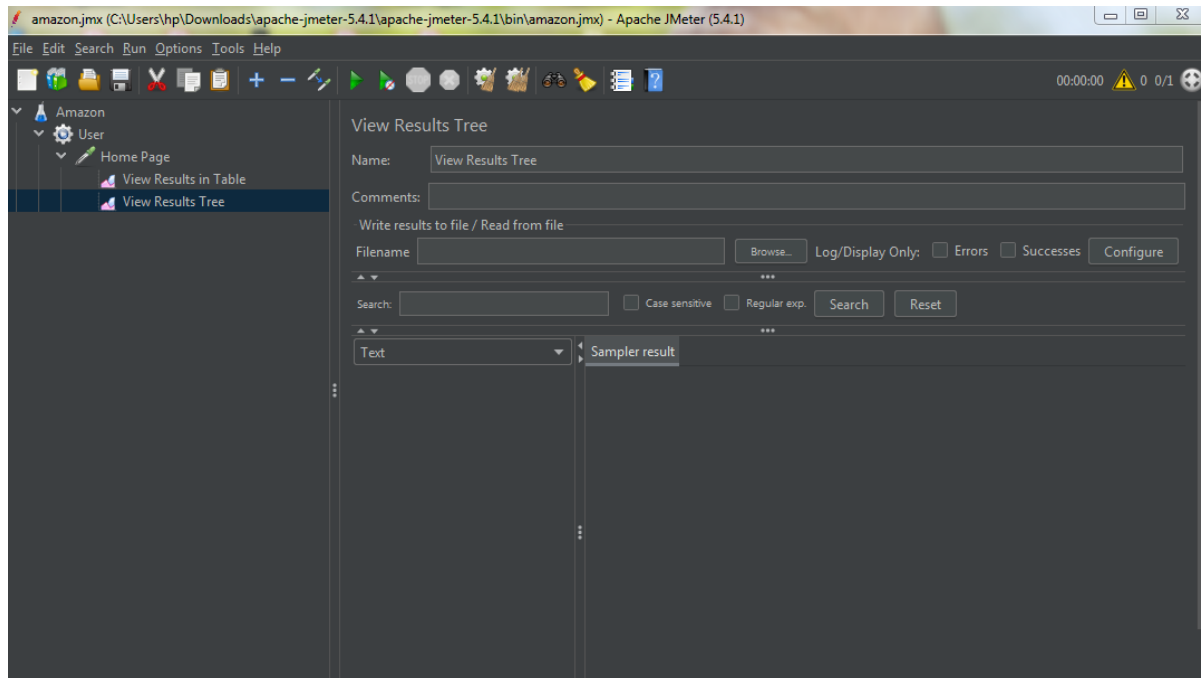


## Step 6: Run the Test Plan

Now with all the setup, let us execute the test plan. With the configuration of the Thread Group (Users), we keep all the default values. It means JMeter will execute the sampler only once. It is similar to a single user, only once.

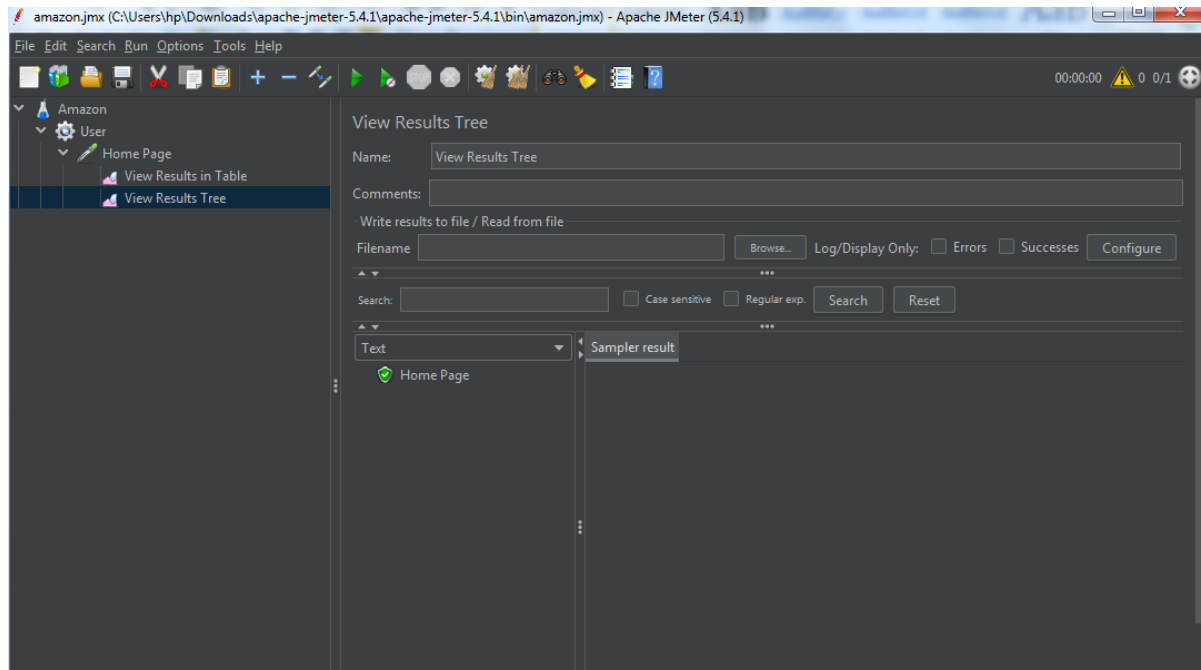
This is similar to a user visiting a web page through browser, with JMeter sampler. To execute the test plan, Select Run from the menu and select Start option.

Apache JMeter asks us to save the test plan in a disk file before actually starting the test. This is important if you want to run the test plan multiple times. You can opt for running it without saving too.



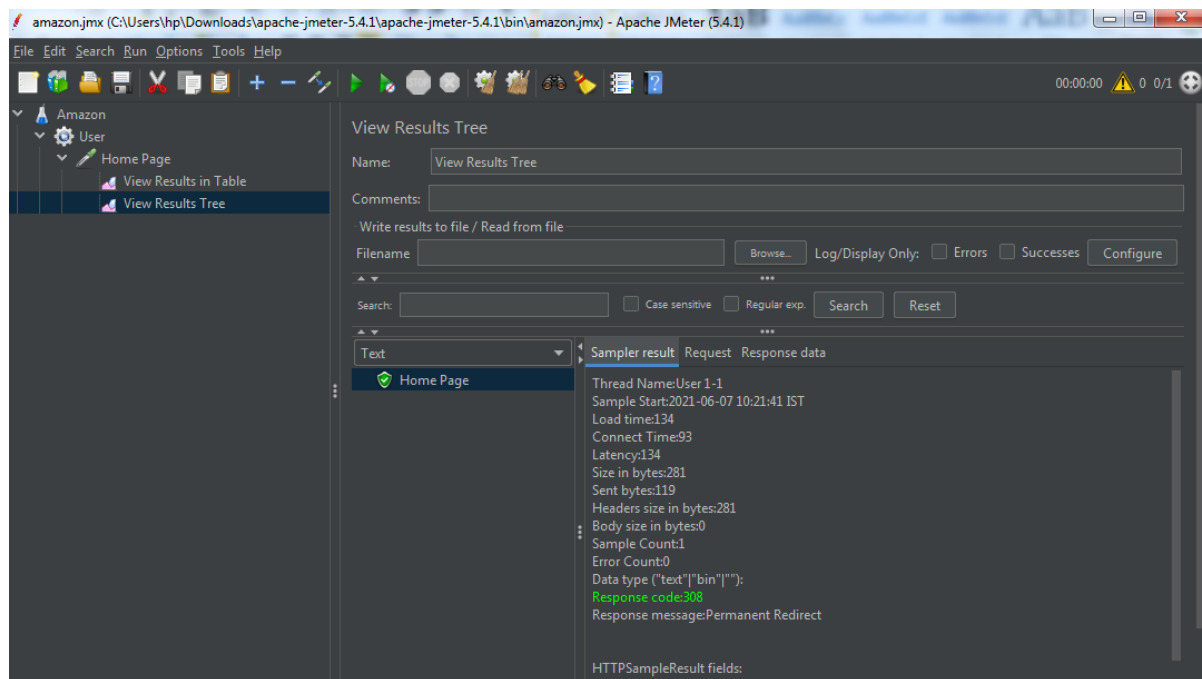
## Step 7: View the Output

We have kept the setting of the thread group as single thread (one user only) and loop for 1 time (run only one time), hence we will get the result of one single transaction in the View Result Tree Listener.

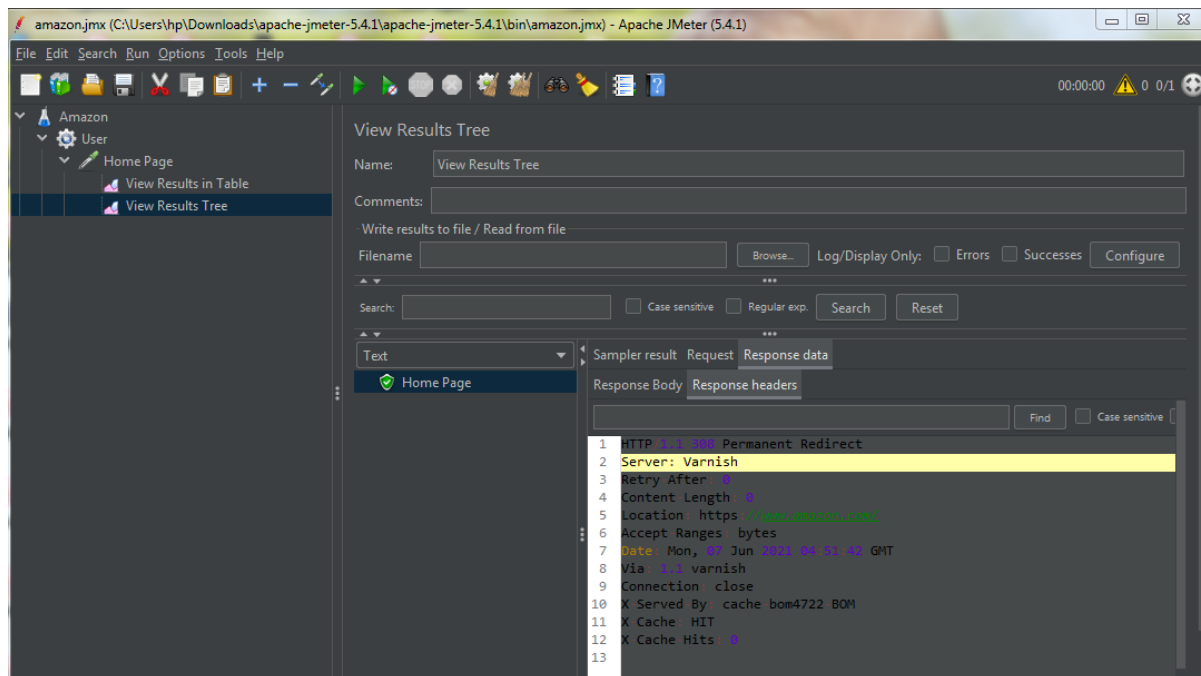


Details of the above result are –

- Green color against the name *Visit amazon Home Page* indicates success.
- JMeter has stored all the headers and the responses sent by the web server and ready to show us the result in many ways.
- The first tab is Sampler Results. It shows JMeter data as well as data returned by the web server.
- The second tab is Request, which shows all the data sent to the web server as part of the request.



The last tab is Response data. In this tab, the listener shows the data received from server in text format.



This is just a simple test plan which executes only one request. But JMeter's real strength is in sending the same request, as if many users are sending it. To test the web servers with multiple users, we need to change the Thread Group (Users) settings.

## Experiment-8

**Objective-** Analyse the performance of railway reservation website using JMeter.

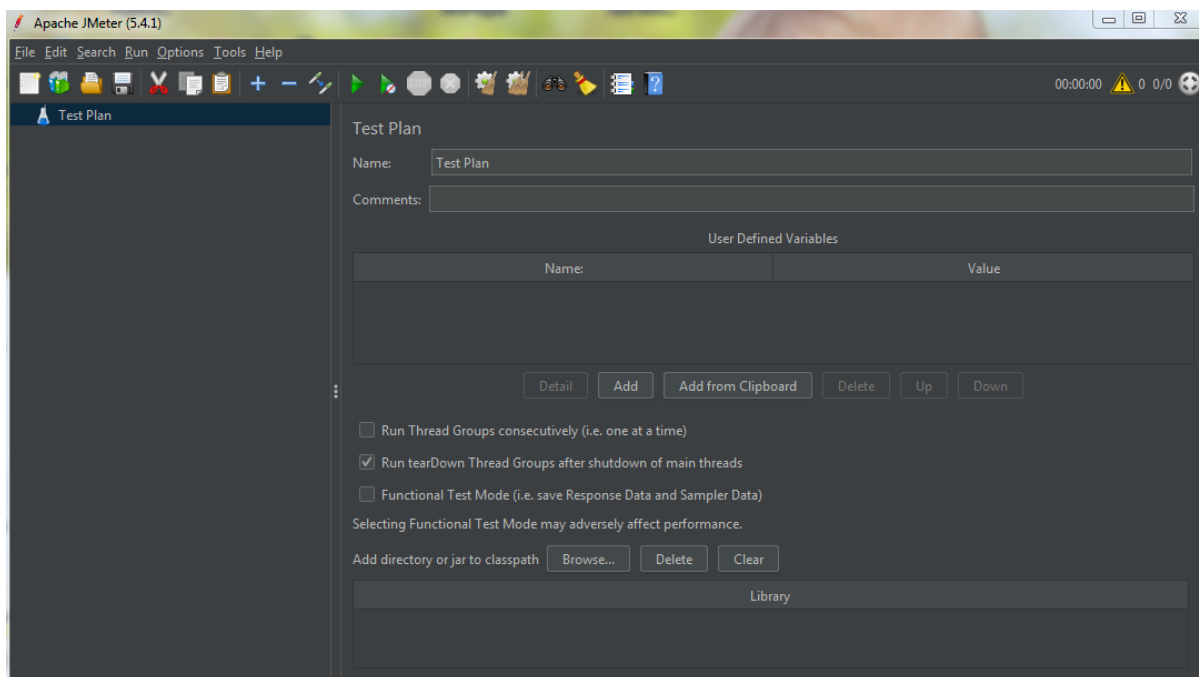
### **Procedure**

- Start JMeter
- Rename the test plan and change the name test plan as *irctc*
- Add thread group
  - (d) Right-click the element where you want to add a child element node.
  - (e) Choose the appropriate option to add.
  - (f) Right-click on the *irctc* (our Test Plan) → Add → Threads (Users) → Thread Group.  
Thus, the Thread Group gets added under the Test Plan (Amazon) node.
- Add sampler
  - (b) Right-clicking and we will add HTTP Request Sampler by choosing Add → Sampler → HTTP request option.
- Add listener
  - (d) Open the context menu

- (e) Right-click the Thread Group (Users)
- (f) Choose Add → Listener → View Results Tree option
- Run the test plan
- Save the test plan
- View the output

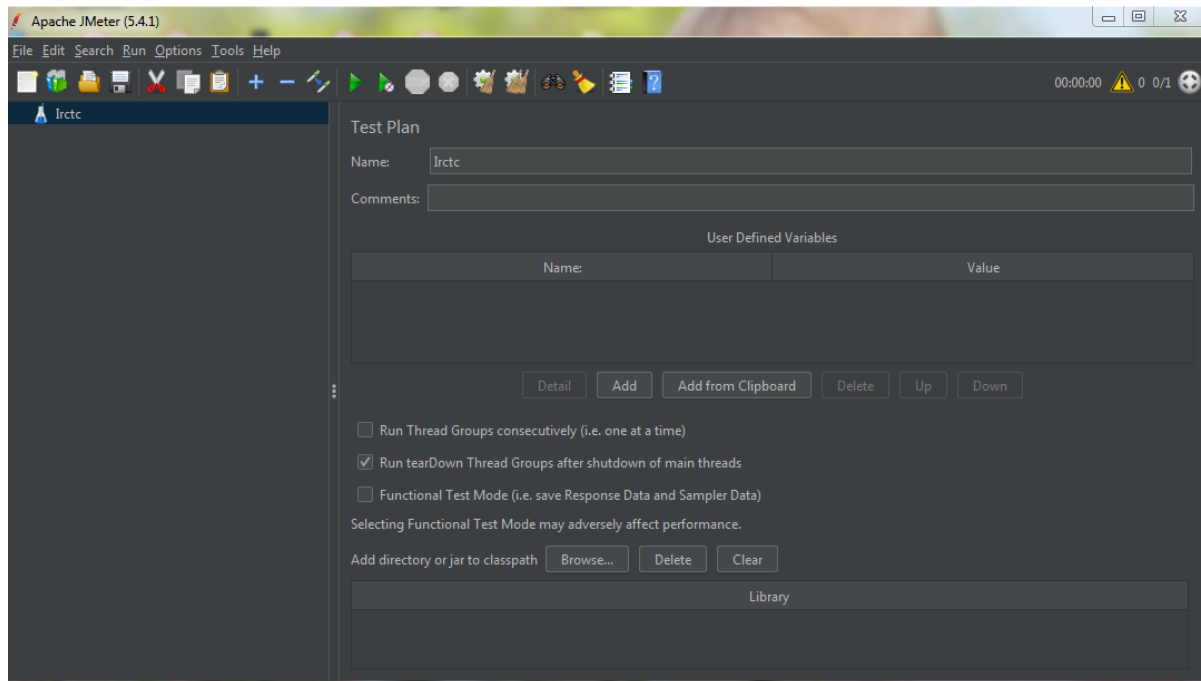
## Step 1: Start JMeter

Open the JMeter window



## Step 2: Rename the Test Plan

Change the name of test plan node to *Irctc* in the *Name* text box. You need to change the focus to workbench node and back to the Test Plan node to see the name getting reflected.

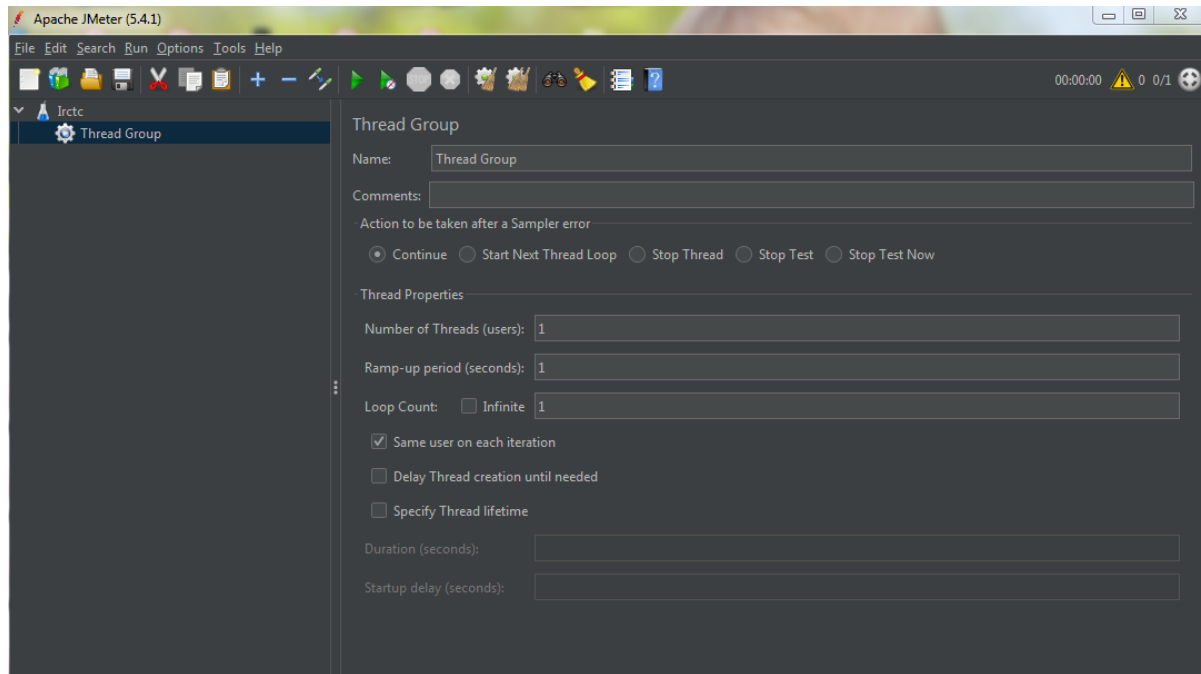


### Step 3: Add Thread Group

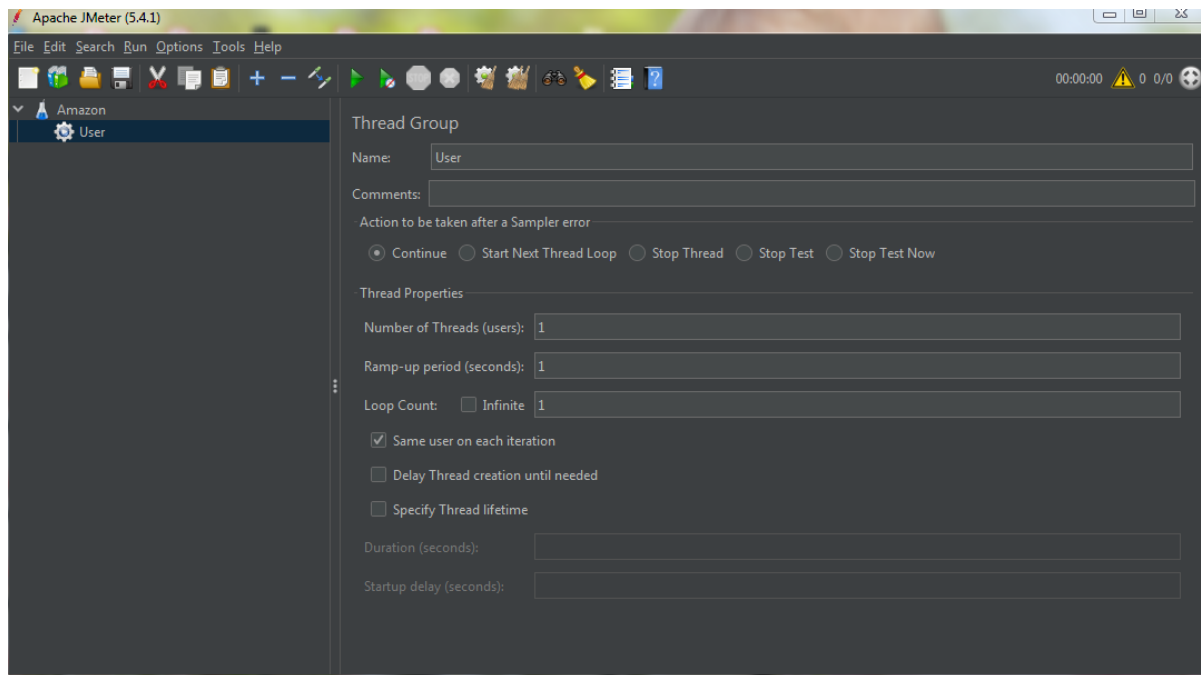
Now we add our first element in the window. We add one Thread Group, which is a placeholder for all other elements like Samplers, Controllers, and Listeners. We need one so we can configure number of users to simulate.

In JMeter, all the node elements are added by using the context menu.

- Right-click the element where you want to add a child element node.
- Choose the appropriate option to add.
- Right-click on the Irttc (our Test Plan) → Add → Threads (Users) → Thread Group. Thus, the Thread Group gets added under the Test Plan (Irttc) node.



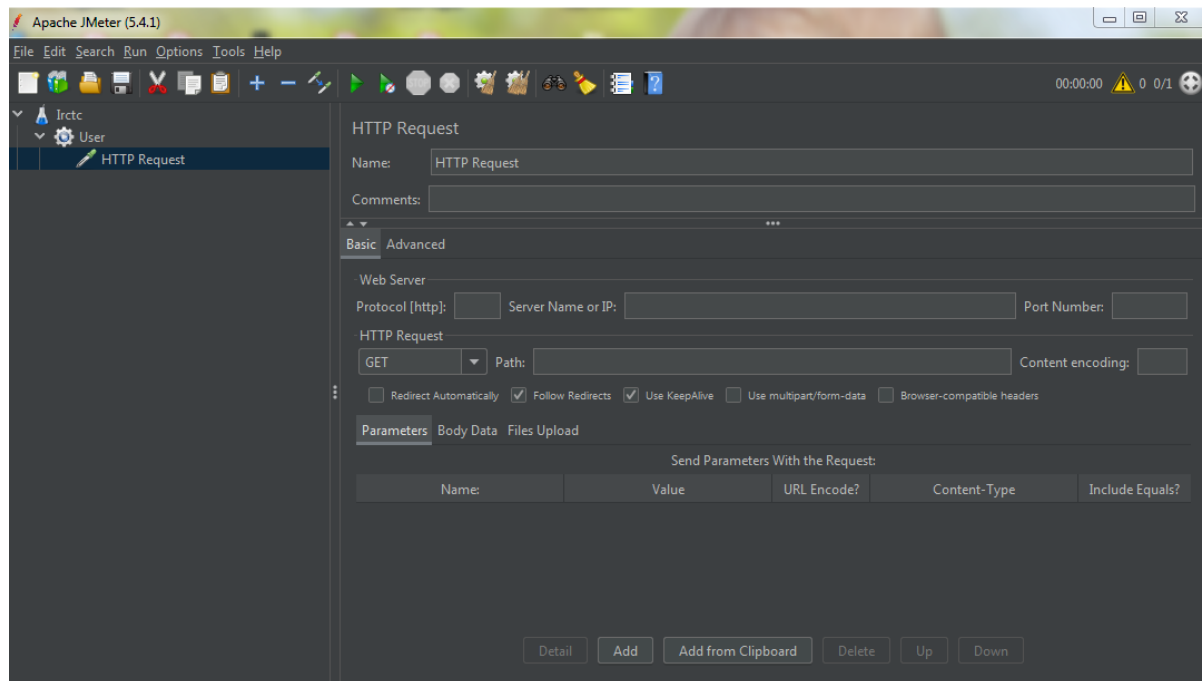
- Name the Thread Group as *Users*. For us, this element means users visiting the Ircrc Home Page.



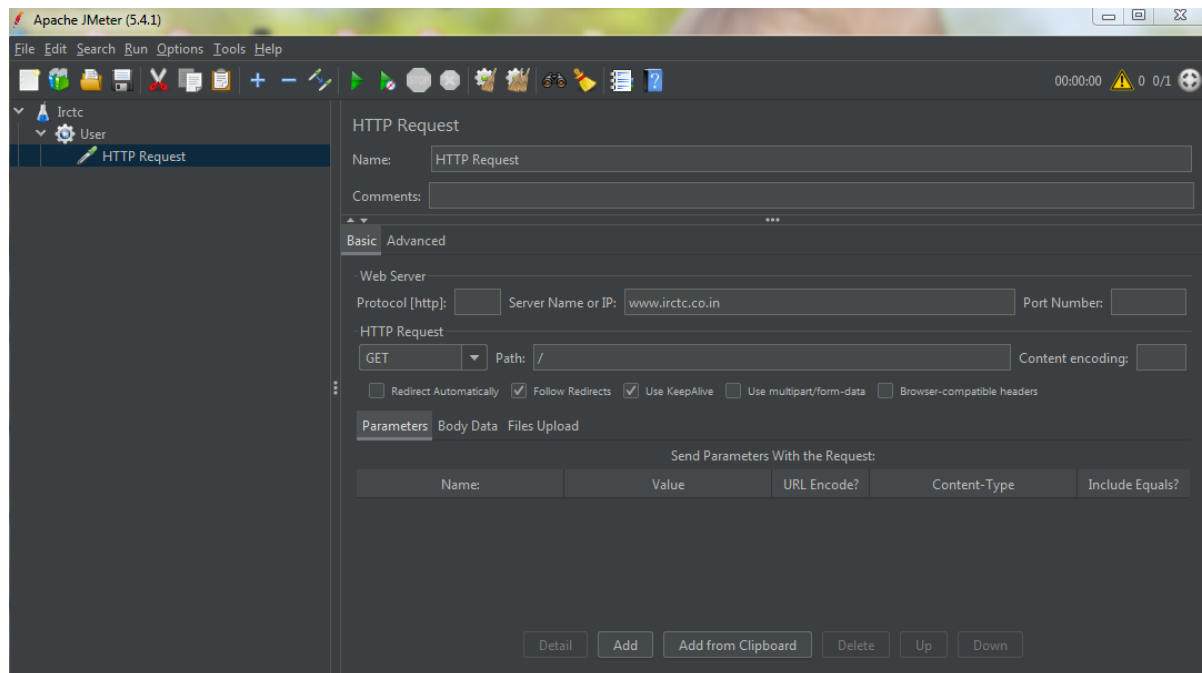
#### Step 4: Add Sampler

We need to add one Sampler in our Thread Group (Users). As done earlier for adding Thread group, this time we will open the context menu of the Thread Group (Users) node by right-

clicking and we will add HTTP Request Sampler by choosing Add → Sampler → HTTP request option.



It will add one empty HTTP Request Sampler under the Thread Group (Users) node. Let us configure this node element –



- **Name** – We will change the name to reflect the action what we want to achieve. We will name it as for example **Visit Ircrc Home Page**

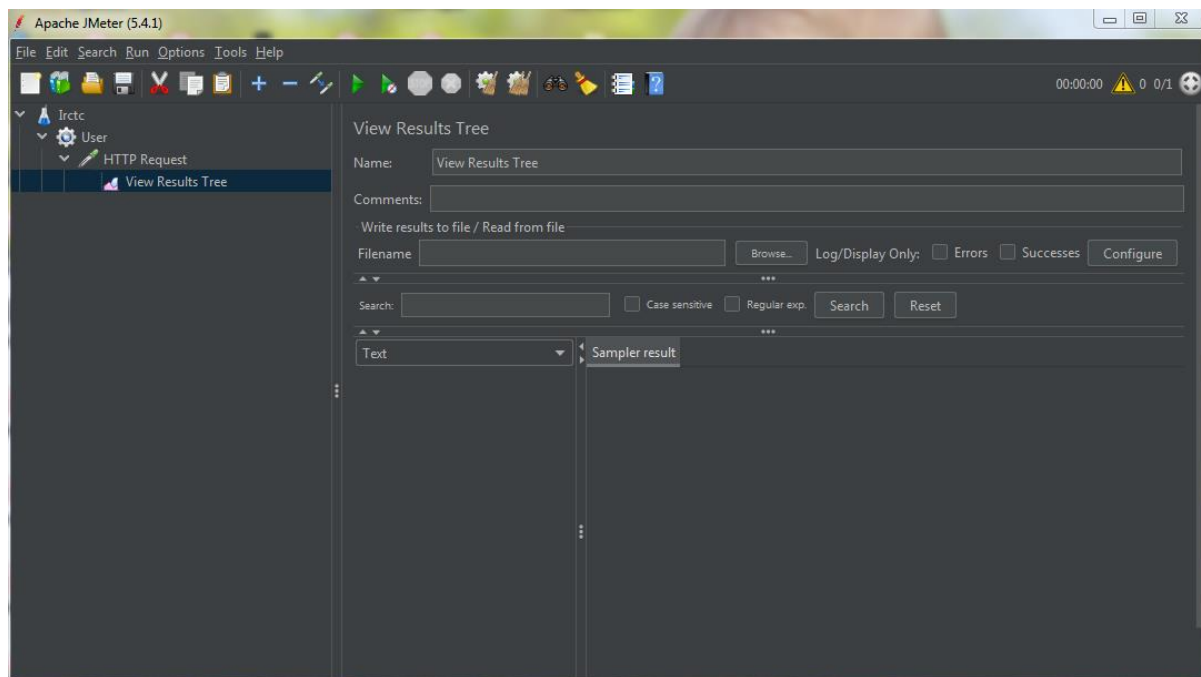
- **Server Name or IP** – Here, we have to type the web server name. In our case it is **www.irctc.co.in**. (http:// part is not written this is only the name of the server or its IP)
- **Protocol** – We will keep this blank, which means we want HTTP as the protocol.
- **Path** – We will type path as / (slash). It means we want the root page of the server.

### Step 5: Add Listener

We will now add a listener. Let us add View Results Table Listener under the Thread Group (User) node. It will ensure that the results of the Sampler will be available to view in this Listener node element.

To add a listener –

- Open the context menu
- Right-click the Thread Group (Users)
- Choose Add → Listener → View Results Table option

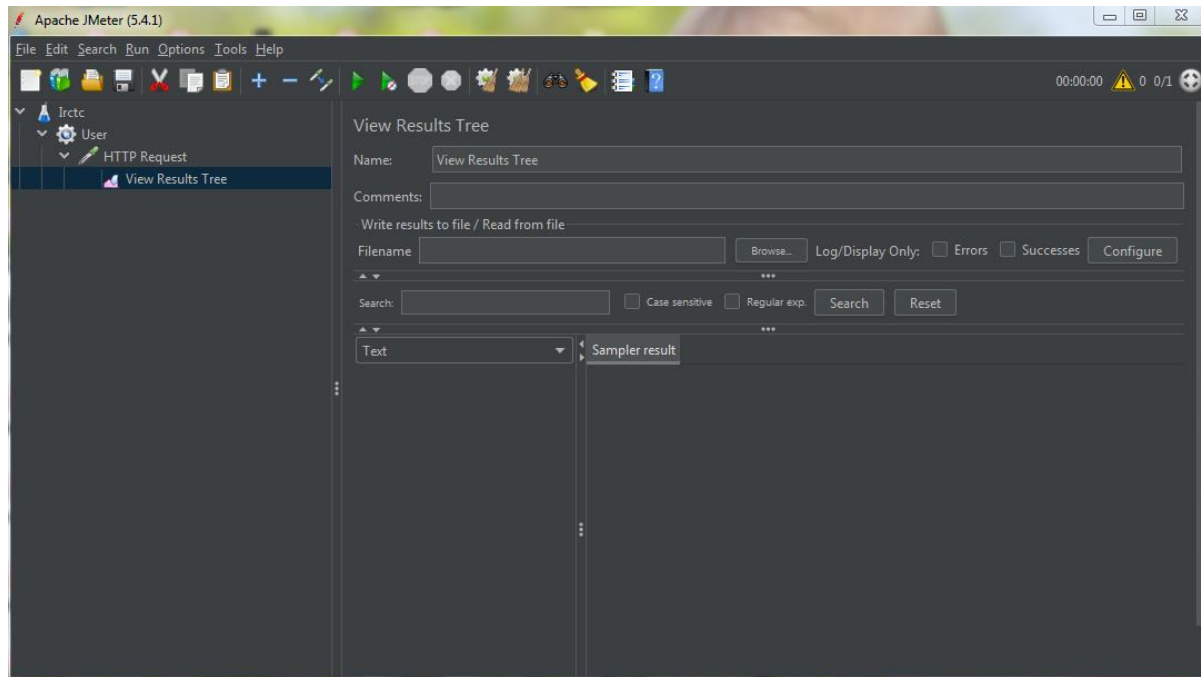


### Step 6: Run the Test Plan

Now with all the setup, let us execute the test plan. With the configuration of the Thread Group (Users), we keep all the default values. It means JMeter will execute the sampler only once. It is similar to a single user, only once.

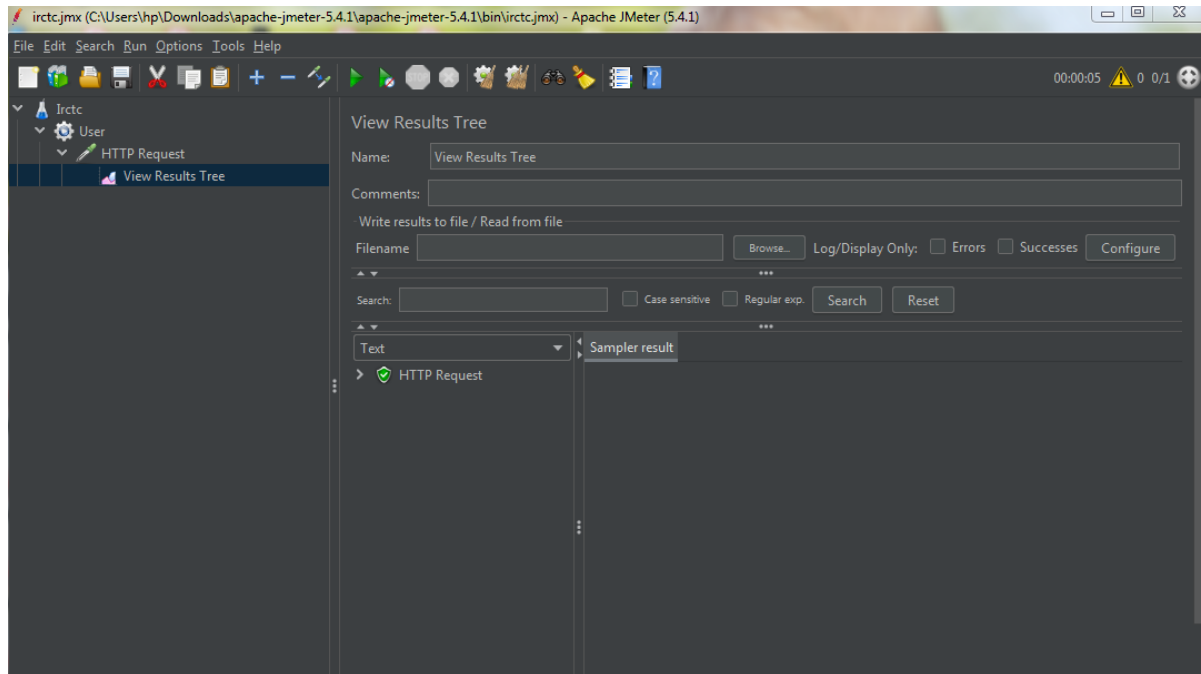
This is similar to a user visiting a web page through browser, with JMeter sampler. To execute the test plan, Select Run from the menu and select Start option.

Apache JMeter asks us to save the test plan in a disk file before actually starting the test. This is important if you want to run the test plan multiple times. You can opt for running it without saving too.



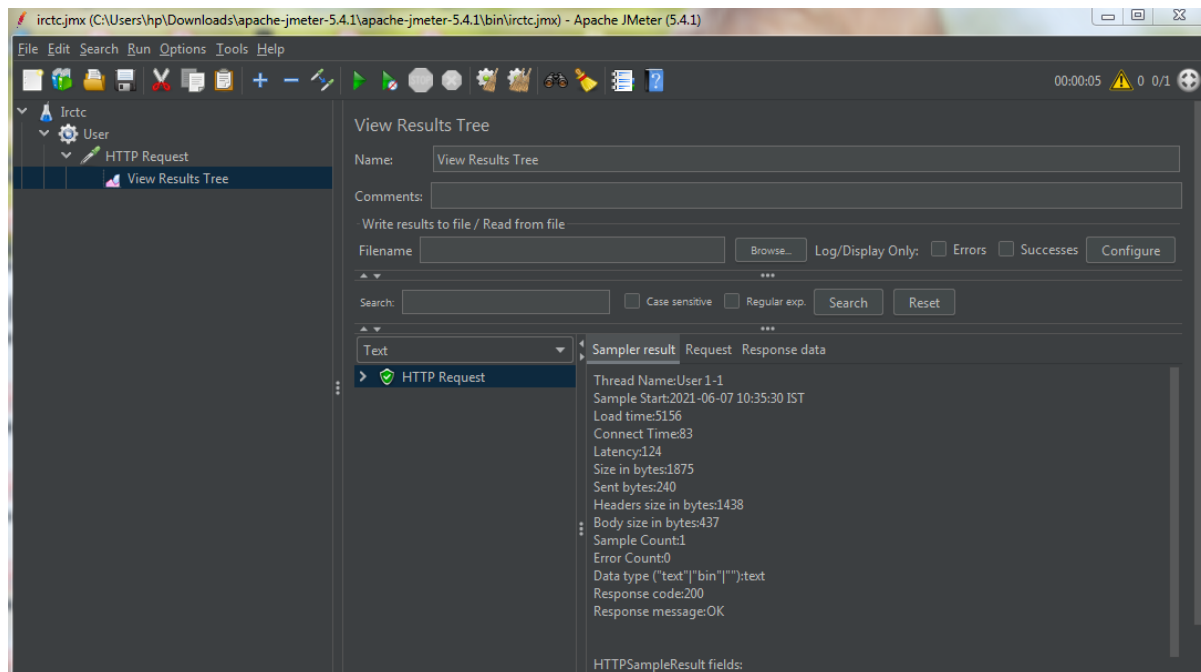
### Step 7: View the Output

We have kept the setting of the thread group as single thread (one user only) and loop for 1 time (run only one time), hence we will get the result of one single transaction in the View Result Tree Listener.

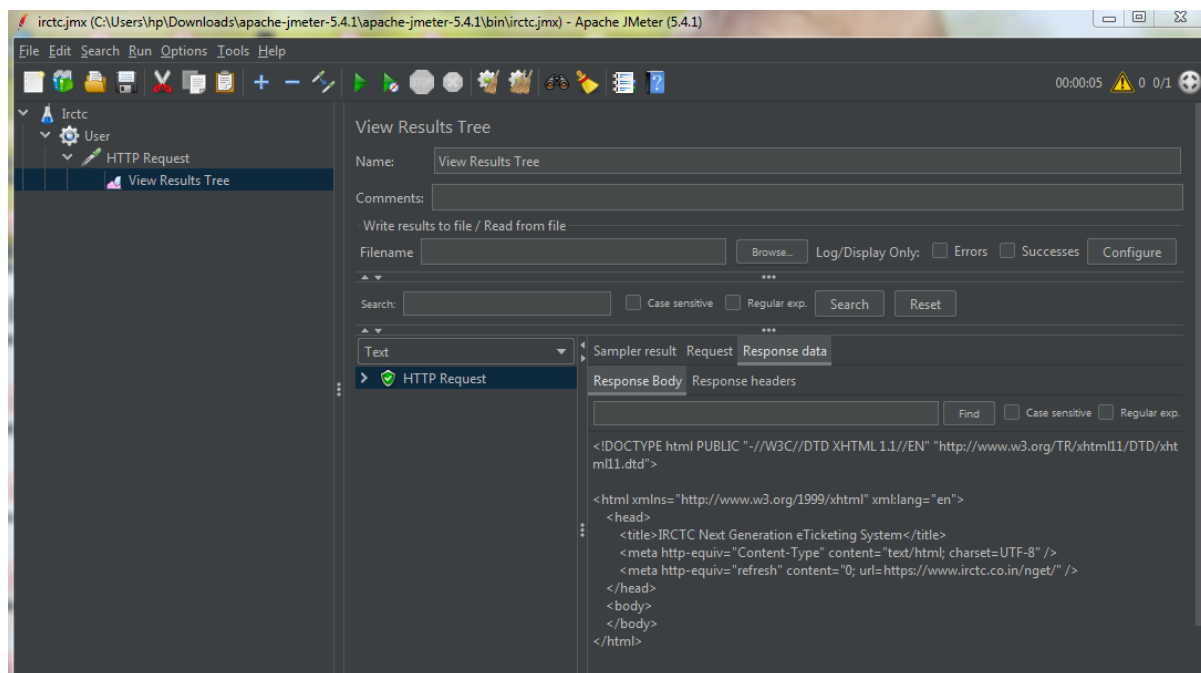


Details of the above result are –

- Green color against the name *Visit irctcHome Page* indicates success.
- JMeter has stored all the headers and the responses sent by the web server and ready to show us the result in many ways.
- The first tab is Sampler Results. It shows JMeter data as well as data returned by the web server.
- The second tab is Request, which shows all the data sent to the web server as part of the request.



The last tab is Response data. In this tab, the listener shows the data received from server in text format.



This is just a simple test plan which executes only one request. But JMeter's real strength is in sending the same request, as if many users are sending it. To test the web servers with multiple users, we need to change the Thread Group (Users) settings.

## VIVA QUESTIONS

### 1. What is performance testing?

It is designed to test the run-time performance of software within the context of an integrated system. It is used to test the speed and effectiveness of the program. It is also called load testing. In it we check, what the performance of the system is in the given load.

Example:

Checking number of processor cycles.

### 2. How do listeners provide access to the information?

Listeners provide access to the information JMeter gathers about the test cases while JMeter runs. The results or information gathered by listeners can be shown in the form of –

- tree
- tables
- graphs
- log file

### 3. What are the elements of JMeter?

- Thread groups
- Sampler
- Listener
- Configuration

### 4. What are the features of JMeter?

Following are some of the features of JMeter –

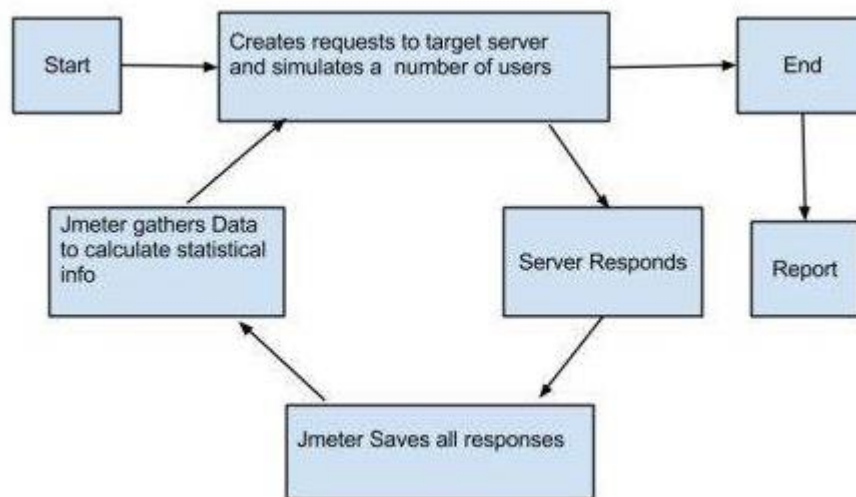
- Being an open source software, it is freely available.
- It has a simple and intuitive GUI.
- JMeter can conduct load and performance test for many different server types – Web - HTTP, HTTPS, SOAP, Database via JDBC, LDAP, JMS, Mail - POP3, etc.
- It is a platform-independent tool. On Linux/Unix, JMeter can be invoked by clicking on JMeter shell script. On Windows, it can be invoked by starting the jmeter.bat file.
- It has full Swing and lightweight component support (precompiled JAR uses packages javax.swing.\* ).
- JMeter store its test plans in XML format. This means you can generate a test plan using a text editor.

- Its full multi-threading framework allows concurrent sampling by many threads and simultaneous sampling of different functions by separate thread groups.
- It is highly extensible.
- It can also be used to perform automated and functional testing of the applications.

## 5. How JMeter Works?

JMeter simulates a group of users sending requests to a target server, and returns statistics that show the performance/functionality of the target server/application via tables, graphs, etc.

Take a look at the following figure that depicts how JMeter works –



## III JUMBLE

Jumble is a class level mutation testing tool that works in conjunction with JUnit. The purpose of mutation testing is to provide a measure of the effectiveness of test cases. A single mutation is performed on the code to be tested, the corresponding test cases are then executed. If the modified code fails the tests, then this increases confidence in the tests. Conversely, if the modified code passes the tests this indicates a testing deficiency.

## Installation and Usage

Release 1.3.0 of Jumble is available as an Eclipse plugin. Just add <http://jumble.sourceforge.net/update/> to Eclipse as an update site, and you will be able to install Jumble. Then you should be able to right-click on any class and do **Jumble Mutation Tester / Analyse tests of this class** to mutate the class and analyse its tests. Note that you click on your source code class, NOT on the JUnit tests.

For command line usage, just download `jumble_1.x.y.jar` via the Sourceforge Files release system, and run it with `java -jar jumble_1.x.y.jar --help` to see the help message.

## Mutation Testing in Java (Using Jumble)

Mutation testing is a white-box testing technique, which changes certain portions of the code to reveal possible faults. On a very high level, it is the process of rewriting the source code with some known or possible bugs or differences to the actual code in small ways in order to remove the redundancies in the source code. The goal of mutation testing is to evaluate the coverage of your tests. There are many mutation testing tool for Java.

1. Jumble
2. PIT
3.  $\mu$ Java
4. Jester

### Jumble

As Jumble is a mutation testing tool which changes the Java code at **byte code** level. It runs a unit test case and applies certain set of mutations.

- **Passed Mutations:** Changes in code which do not affect the final output
- **Failed Mutations:** Changes in code which change the final output

**Score :** It is the percentage of passed mutations out of the total mutations. It is used to assess the validity of our test suite.

### Installation

Download Eclipse IDE and install Jumble to Eclipse.

## Experiment 9. Simple Code Snippet in Java

```
int index = 0;
while(true)
{
    index++;
    if (index == 10)
        break;
}
```

### Mutated Code in JAVA

```
int index = 0;
while (true)
{
    index++;
    if (index >= 10)
        break;
}
```

**Explanation:** The mutant code above will **pass the Jumble test** because change in == to >= does not affect the output of the code. Execution will stop when index == 10 and since we are increasing value by 1 and index starts from 0 so the output will remain same.

### Try with Jumble

The code written below has been tested with Jumble plugin in Eclipse. The code detects the first occurrence of a duplicate and returns the value to the calling function. The program is flawed in many ways which you can try figuring out.

```
// Java program to illustrate mutation Testing

// The code detects the first occurrence of a

// duplicate and returns the value to the calling

// function

package testPackage;
```

```
import java.util.Arrays;

import java.util.List;

public class SampProg

{

    protected int repeatedNumber(final List a)

    {

        int len = a.size(),i,dup = -1;

        int[] arr = new int[len];

        for (i=0; i<len; i++)

        {

            arr[i] = a.get(i);

        }

        Arrays.sort(arr);

        try

        {

            for (i=1; i<len; i++)

            {
```

```

        if(arr[i] == arr[i-1])

        {

            dup = arr[i];

            break;

        }

    }

}

catch(Exception e)

{

    System.out.println(e.getMessage());

}

return dup;

}

}

```

After writing the program we create test cases using JUnit in Java. The output obtained on performing Jumble Analysis is given below:

Mutating testPackage.SampProg

Tests: testPackage.SampProgTest

Mutation points = 11, unit test time limit 2.94s

M FAIL: (testPackage.SampProg.java:8): -1 -> 1

M FAIL: (testPackage.SampProg.java:10): 0 -> 1

.M FAIL: (testPackage.SampProg.java:10): negated conditional

M FAIL: (testPackage.SampProg.java:16): 1 -> 0

M FAIL: (testPackage.SampProg.java:18): 1 -> 0  
M FAIL: (testPackage.SampProg.java:18): - -> +  
M FAIL: (testPackage.SampProg.java:18): negated conditional  
M FAIL: (testPackage.SampProg.java:16): += -> -=  
M FAIL: (testPackage.SampProg.java:16): negated conditional.  
Jumbling took 7.595s  
Score: 18%

### **Interpreting the Result**

1. In the first line of M FAIL in the output -1 was changed to 1. In the main program, if there are no duplicates we expect the value to be returned as -1. but instead we get the value 1. We can fix the mutation by enabling checks for value mutation.
2. Next, we can see “negated conditional” on the 3rd, 7th and 9th line where condition has been negated. Here however we can’t fix the error for this program.

Other cases where the mutation failed to give proper output is when operators are changed to other operators. In this and many algorithmic codes, mutations related to operator changes cannot be fixed. However mutation testing gives brief idea in some mutations as to where the test case is weak or there is an error in the source code.

## **IV EcEmma**

**Objective:** - Calculate the coverage analysis of programs given in 1 (a) to 1 (f) using Eclemma Free open source Tool.

**EcEmma** records which parts of your Java code are executed during a particular program launch. This technique is called code coverage analysis and typically used with automated testing like JUnit unit tests. It helps to identify untested parts of a code base and improve the corresponding tests.

### **Procedure of Installation**

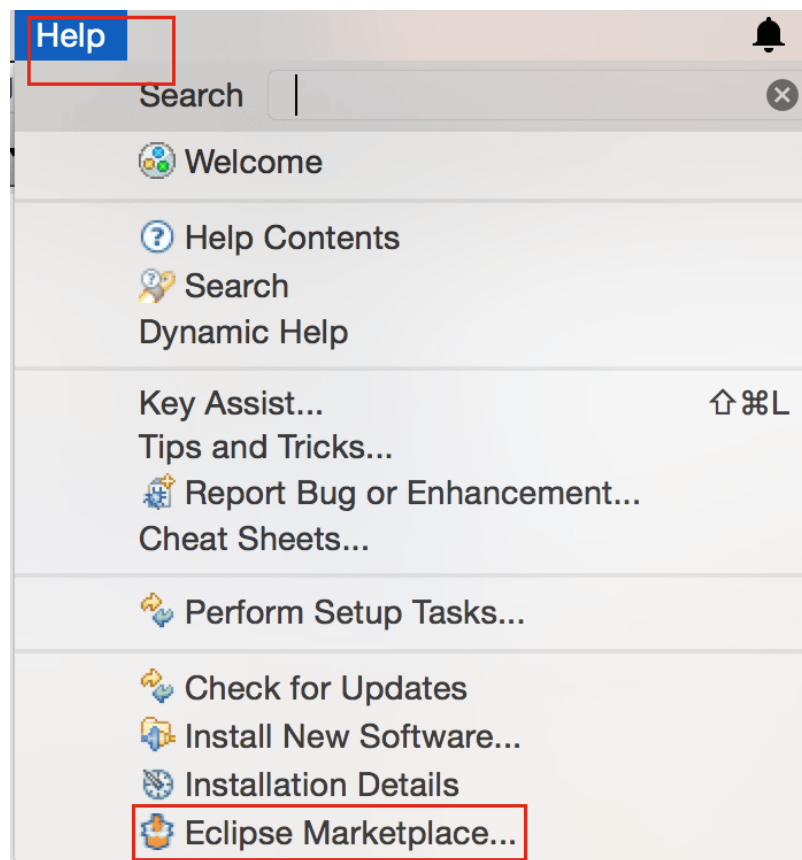
1. Install EcEmma Java Code Coverage Eclipse Plugin
2. Type Code Coverage and press Enter.
3. Click on install button next to ElcEmma Code Coverage
4. Create JUnit test case and checkout Code Coverage
5. Create java test-case in class CrunchifyFibonacciSeriesTest.java
6. Right click on file and run as Coverage As -> JUnit Test.
7. Checkout the results under Coverage Tab for output.

## Experiment-10

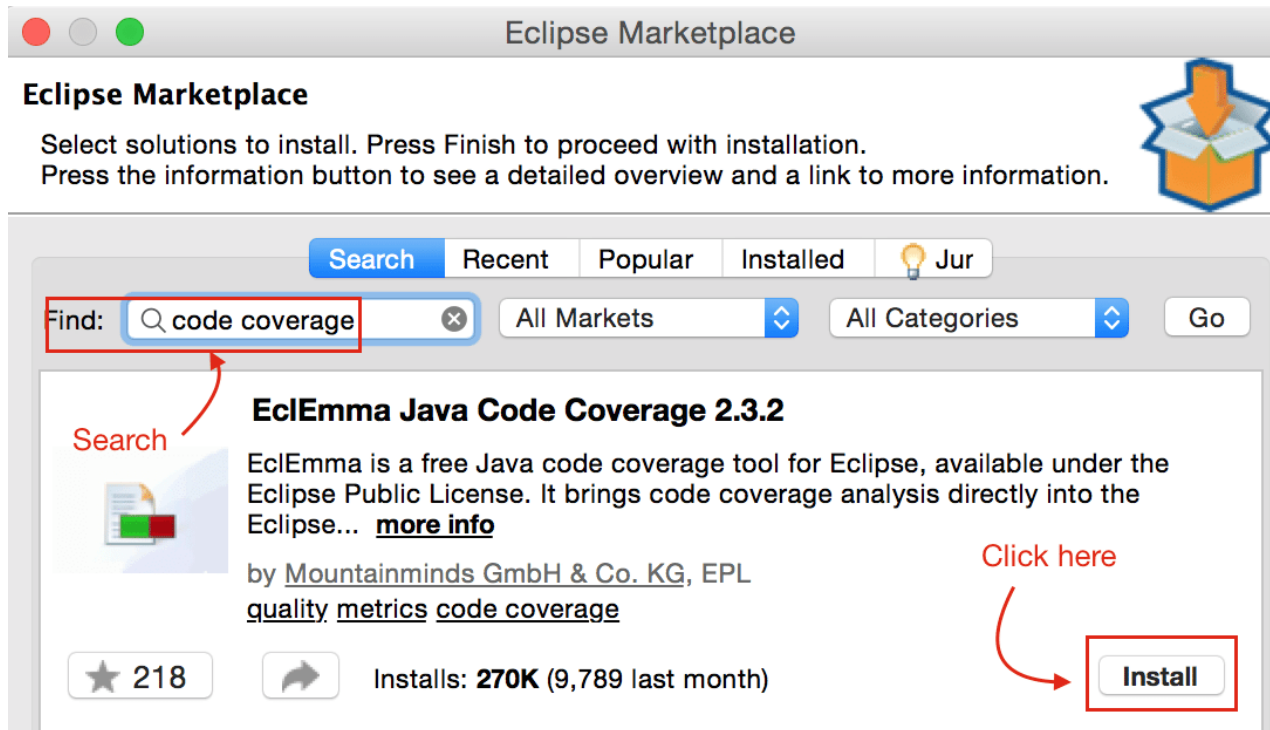
**Objective-** Calculate the coverage analysis of fabonacci series using eclemma tool

**Step-1:** Install EcEmma Java Code Coverage Eclipse Plugin

Click on Help -> Eclipse Marketplace...



**Step-2:** Type Code Coverage and press Enter.



### Step-3

- Click on install button next to ElcEmma Code Coverage
- Click continue and install plugin successfully
- Restart Eclipse

### Step-4 Create JUnit test case and checkout Code Coverage

We are going to create JUnit test case for Fibonacci Series program. Make sure you copy and run it in your Eclipse environment before proceeding below.

If you have maven project then include below dependency. Follow steps here to convert your eclipse project to maven project.

```
<dependency>
```

```
    <groupId>junit</groupId>
```

```
    <artifactId>junit</artifactId>
```

```
    <version>4.12</version>
```

```
</dependency>
```

or non-maven project, you could also download junit.jar file and put it under your project's /lib folder.

#### Step-5: Create java test-case in class CrunchifyFibonacciSeriesTest.java

```
package crunchify.com.tests;
```

```
import org.junit.Test;
```

```
import crunchify.com.tutorial.CrunchifyFibonacci;
```

```
/**
```

```
 * @author Crunchify.com
```

```
 */
```

```
public class CrunchifyFibonacciSeriesTest {
```

```
int totalNumber = 5;
```

```
@Test
```

```
public void testCheckFibonacciRecursion() {
```

```
    CrunchifyFibonacci.fibonacciRecursion(totalNumber);
```

```
}
```

```
@Test
```

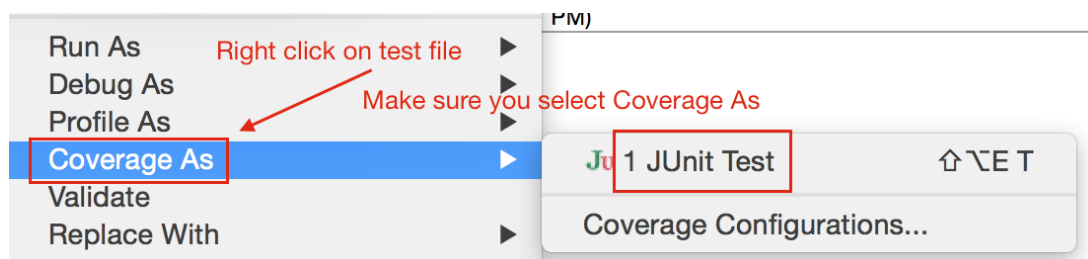
```
public void testCheckFibonacciLoop() {
```

```
    CrunchifyFibonacci.fibonacciLoop(totalNumber);
```

```
}
```

```
}
```

**Step-6: Right click on file and run as Coverage As -> JUnit Test.**



**Step-7: Checkout the results under Coverage Tab for output.**

| Crunchify FibonacciSeriesTest (Jul 9, 2015 3:59:54 PM) |          |                      |                     |                    |
|--------------------------------------------------------|----------|----------------------|---------------------|--------------------|
| Element                                                | Coverage | Covered Instructions | Missed Instructions | Total Instructions |
| CrunchifyFibonacci                                     | 51.6 %   | 47                   | 44                  | 91                 |
| main(String[])                                         | 0.0 %    | 0                    | 39                  | 39                 |
| fibonacciLoop(int)                                     | 93.5 %   | 29                   | 2                   | 31                 |
| fibonacciRecursion(int)                                | 100.0 %  | 18                   | 0                   | 18                 |

Good coverage for two methods

Crunchify Tips

```

// Printing Fibonacci series upto number
for(int i=1; i<=number; i++){
    System.out.print(fibonacciRecursion(i) + " ");
}

// Java program for Fibonacci number using recursion.
public static int fibonacciRecursion(int number){
    if(number == 1 || number == 2){
        return 1;
    }

    return fibonacciRecursion(number-1) + fibonacciRecursion(number-2); //tail recursion
}

// Java program for Fibonacci number using Loop.
public static int fibonacciLoop(int number){
    if(number == 1 || number == 2){
        return 1;
    }
    int fibo1=1, fibo2=1, fibonacci=1;
    for(int i= 3; i<= number; i++){
        fibonacci = fibo1 + fibo2; //Fibonacci number is sum of previous two Fibonacci number
        fibo1 = fibo2;
        fibo2 = fibonacci;
    }
    return fibonacci; //Fibonacci number
}

```

Red Color - we haven't touch main method - so all RED

Green Color - Code coverage is good for these lines

Crunchify

## VIVA QUESTIONS

### 1. What is code Inspection?

Code Inspection is the most formal type of review, which is a kind of static testing to avoid the defect multiplication at a later stage.

- The main purpose of code inspection is to find defects and it can also spot any process improvement if any.
- An inspection report lists the findings, which include metrics that can be used to aid improvements to the process as well as correcting defects in the document under review.
- Preparation before the meeting is essential, which includes reading of any source documents to ensure consistency.
- Inspections are often led by a trained moderator, who is not the author of the code.
- The inspection process is the most formal type of review based on rules and checklists and makes use of entry and exit criteria.
- It usually involves peer examination of the code and each one has a defined set of roles.
- After the meeting, a formal follow-up process is used to ensure that corrective action is completed in a timely manner.
- 

### 2. What is Code-Based Testing?

Code-based testing corresponds to the testing that is carried out on code development, code inspection, unit testing in software development process.

The Code-based testing consists of following testing:

- Dynamic Testing - Statement coverage, Branch coverage, Path coverage
- Checking for Complexity of Code using techniques like Cyclomatic Complexity
- Static Testing - Code Inspection, Code Walkthrough, Code Review, Code Audit

### 3. What is Code Free Testing?

Code free testing, a testing technique performed without the execution of code. The static testing techniques such as review and static analysis are part of code free testing.

- Review - Typically used to find and eliminate errors or ambiguities in documents such as requirements, design, test cases, etc.
- Static analysis - The code written by developers are analysed (usually by tools) for structural defects that may lead to defects.

#### 4. What is JUnit ?

JUnit is a unit testing framework for Java programming language. It plays a crucial role test-driven development, and is a family of unit testing frameworks collectively known as xUnit.

JUnit promotes the idea of "first testing then coding", which emphasizes on setting up the test data for a piece of code that can be tested first and then implemented. This approach is like "test a little, code a little, test a little, code a little." It increases the productivity of the programmer and the stability of program code, which in turn reduces the stress on the programmer and the time spent on debugging.

#### 5. Differentiate between Manual and Automated testing?

| Manual Testing                                                                                                                       | Automated Testing                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Executing a test cases manually without any tool support is known as manual testing.                                                 | Taking tool support and executing the test cases by using an automation tool is known as automation testing.                                              |
| <b>Time-consuming and tedious</b> – Since test cases are executed by human resources, it is very slow and tedious.                   | <b>Fast</b> – Automation runs test cases significantly faster than human resources.                                                                       |
| <b>Huge investment in human resources</b> – As test cases need to be executed manually, more testers are required in manual testing. | <b>Less investment in human resources</b> – Test cases are executed using automation tools, so less number of testers are required in automation testing. |
| <b>Less reliable</b> – Manual testing is less reliable, as it has to account for human errors.                                       | <b>More reliable</b> – Automation tests are precise and reliable.                                                                                         |
| <b>Non-programmable</b> – No programming can be done to write sophisticated tests to fetch hidden information.                       | <b>Programmable</b> – Testers can program sophisticated tests to bring out hidden information.                                                            |

## V selenium

**Objective:** - Generate Test sequences and validate using Selenium tool for given websites below:

|                     |                    |                     |
|---------------------|--------------------|---------------------|
| Site Amazon         | Website Amazon.com | Type shopping       |
| Flip kart           | Flipkart.com       | shopping            |
| Railway reservation | Irctc.co.in        | Ticket booking site |
| Train searching     | Erail.in           | Train searching     |

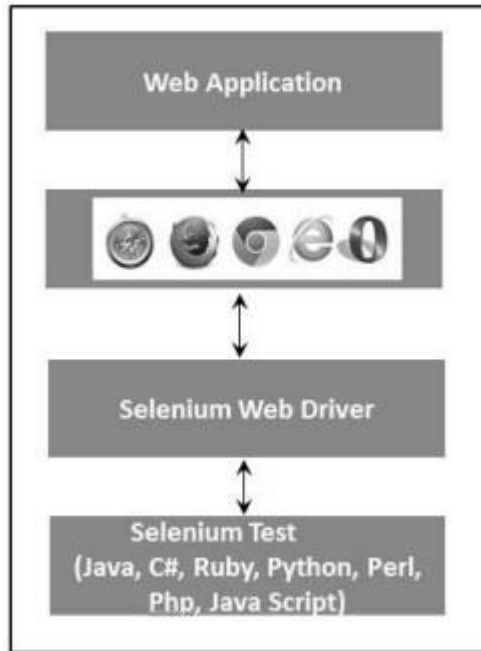
**Selenium tool:** Selenium is an open-source and a portable automated software testing tool for testing web applications. It has capabilities to operate across different browsers and operating systems. Selenium is not just a single tool but a set of tools that helps testers to automate web-based applications more efficiently.

WebDriver is a tool for automating testing web applications. It is popularly known as Selenium 2.0. WebDriver uses a different underlying framework, while Selenium RC uses JavaScript Selenium-Core embedded within the browser which has got some limitations. WebDriver interacts directly with the browser without any intermediary, unlike Selenium RC that depends on a server. It is used in the following context –

- Multi-browser testing including improved functionality for browsers which is not well-supported by Selenium RC (Selenium 1.0).
- Handling multiple frames, multiple browser windows, popups, and alerts.
- Complex page navigation.
- Advanced user navigation such as drag-and-drop.
- AJAX-based UI elements.

## Architecture

WebDriver is best explained with a simple architecture diagram as shown below.



## Installation of Java and Selenium Tool

Setting up the environment involves the following steps.

- Download and Install Java
- Download and Configure Eclipse
- Configure Selenium WebDriver

### Download and Install Java

We need to have JDK (Java Development Kit) installed in order to work with Selenium WebDriver/Selenium. Let us see how to download and install Java.

**Step 1** – Navigate to the UR.

<https://www.oracle.com/technetwork/java/javase/downloads/index.html>

**Step 2** – Go to "Downloads" section and select "JDK Download".

Overview **Downloads** Documentation Community Technologies Training

## Java SE Downloads

  
**DOWNLOAD** ↓  
Java Platform (JDK) 8u11

  
**DOWNLOAD** ↓  
JDK 8u11 & NetBeans 8.0

### Java Platform, Standard Edition

#### Java SE 8u11

This release includes important security fixes. Oracle strongly recommends that all Java SE 8 users upgrade to this release.  
[Learn more](#) ▶

- Installation Instructions
- Release Notes
- Oracle License
- Java SE Products
- Third Party Licenses
- Certified System Configurations
- Readme Files
  - JDK ReadMe
  - JRE ReadMe

**JDK**  
**DOWNLOAD** ↓

**Server JRE**  
**DOWNLOAD** ↓

**JRE**  
**DOWNLOAD** ↓

**Step 3** – Select "Accept License Agreement" radio button.

[Overview](#)
[Downloads](#)
[Documentation](#)
[Community](#)
[Technologies](#)
[Training](#)

## Java SE Development Kit 8 Downloads

Thank you for downloading this release of the Java™ Platform, Standard Edition Development Kit (JDK™). The JDK is a development environment for building applications, applets, and components using the Java programming language.

The JDK includes tools useful for developing and testing programs written in the Java programming language and running on the Java platform.

See also:

- Java Developer Newsletter (tick the checkbox under Subscription Center > Oracle Technology News)
- Java Developer Day hands-on workshops (free) and other events
- Java Magazine

JDK MD5 Checksum

**Looking for JDK 8 on ARM?**  
JDK 8 for ARM downloads have moved to the JDK 8 for ARM download page.

### Java SE Development Kit 8u11

You must accept the [Oracle Binary Code License Agreement for Java SE](#) to download this software.

☒ Accept License Agreement
 ☐ Decline License Agreement

| Product / File Description          | File Size | Download                                        |
|-------------------------------------|-----------|-------------------------------------------------|
| Linux x86                           | 133.58 MB | <a href="#">jdk-8u11-linux-i586.rpm</a>         |
| Linux x86                           | 152.55 MB | <a href="#">jdk-8u11-linux-i586.tar.gz</a>      |
| Linux x64                           | 133.89 MB | <a href="#">jdk-8u11-linux-x64.rpm</a>          |
| Linux x64                           | 151.65 MB | <a href="#">jdk-8u11-linux-x64.tar.gz</a>       |
| Mac OS X x64                        | 207.82 MB | <a href="#">jdk-8u11-macosx-x64.dmg</a>         |
| Solaris SPARC 64-bit (SVR4 package) | 135.66 MB | <a href="#">jdk-8u11-solaris-sparcv9.tar.Z</a>  |
| Solaris SPARC 64-bit                | 96.14 MB  | <a href="#">jdk-8u11-solaris-sparcv9.tar.gz</a> |
| Solaris x64 (SVR4 package)          | 135.7 MB  | <a href="#">jdk-8u11-solaris-x64.tar.Z</a>      |
| Solaris x64                         | 93.18 MB  | <a href="#">jdk-8u11-solaris-x64.tar.gz</a>     |
| Windows x86                         | 151.81 MB | <a href="#">jdk-8u11-windows-i586.exe</a>       |
| Windows x64                         | 155.29 MB | <a href="#">jdk-8u11-windows-x64.exe</a>        |

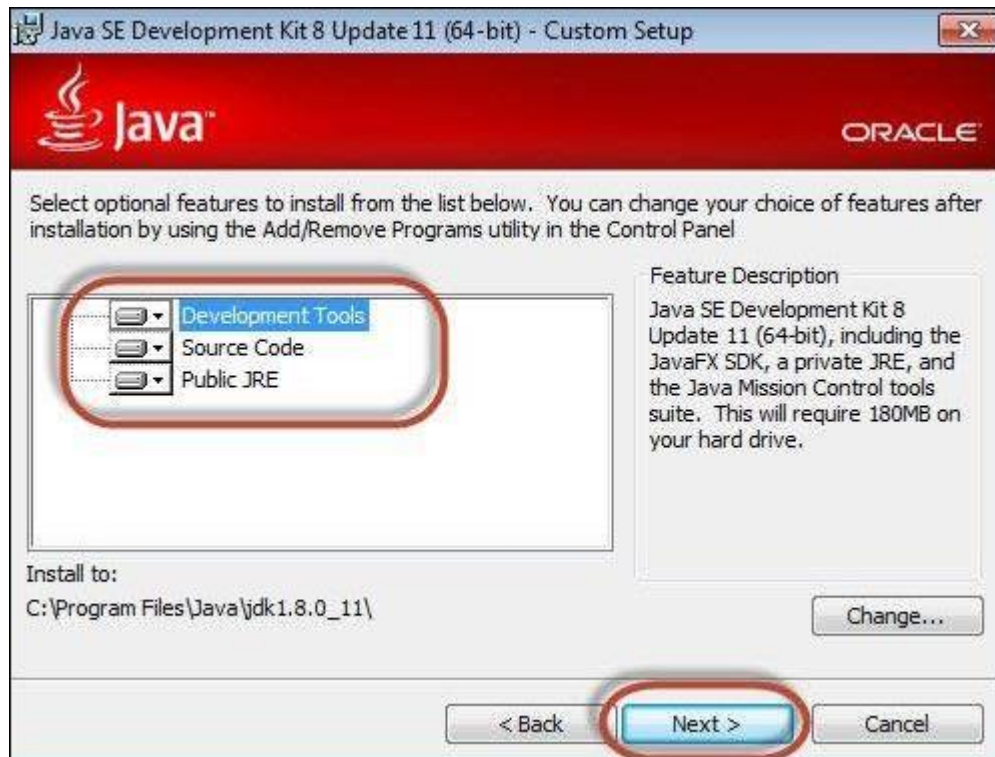
**Step 4** – Select the appropriate installation. In this case, it is 'Windows 7-64' bit. Click the appropriate link and save the .exe file to your disk.

| Java SE Development Kit 8u11                                                                                      |           |                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------|
| You must accept the Oracle Binary Code License Agreement for Java SE to download this software.                   |           |                                                                                                                                   |
| Thank you for accepting the Oracle Binary Code License Agreement for Java SE; you may now download this software. |           |                                                                                                                                   |
| Product / File Description                                                                                        | File Size | Download                                                                                                                          |
| Linux x86                                                                                                         | 133.58 MB |  <a href="#">jdk-8u11-linux-i586.rpm</a>         |
| Linux x86                                                                                                         | 152.55 MB |  <a href="#">jdk-8u11-linux-i586.tar.gz</a>      |
| Linux x64                                                                                                         | 133.89 MB |  <a href="#">jdk-8u11-linux-x64.rpm</a>          |
| Linux x64                                                                                                         | 151.65 MB |  <a href="#">jdk-8u11-linux-x64.tar.gz</a>       |
| Mac OS X x64                                                                                                      | 207.82 MB |  <a href="#">jdk-8u11-macosx-x64.dmg</a>         |
| Solaris SPARC 64-bit (SVR4 package)                                                                               | 135.66 MB |  <a href="#">jdk-8u11-solaris-sparcv9.tar.Z</a>  |
| Solaris SPARC 64-bit                                                                                              | 96.14 MB  |  <a href="#">jdk-8u11-solaris-sparcv9.tar.gz</a> |
| Solaris x64 (SVR4 package)                                                                                        | 135.7 MB  |  <a href="#">jdk-8u11-solaris-x64.tar.Z</a>      |
| Solaris x64                                                                                                       | 93.18 MB  |  <a href="#">jdk-8u11-solaris-x64.tar.gz</a>     |
| Windows x86                                                                                                       | 151.81 MB |  <a href="#">jdk-8u11-windows-i586.exe</a>       |
| Windows x64                                                                                                       | 155.29 MB |  <a href="#">jdk-8u11-windows-x64.exe</a>        |

**Step 5** – Run the downloaded exe file to launch the Installer wizard. Click 'Next' to continue.



**Step 6** – Select the features and click 'Next'.



**Step 7** – The installer is extracted and its progress is shown in the wizard.



**Step 8** – The user can choose the install location and click 'Next'.



**Step 9** – The installer installs the JDK and new files are copied across.



**Step 10** – The Installer installs successfully and displays the same to the user.



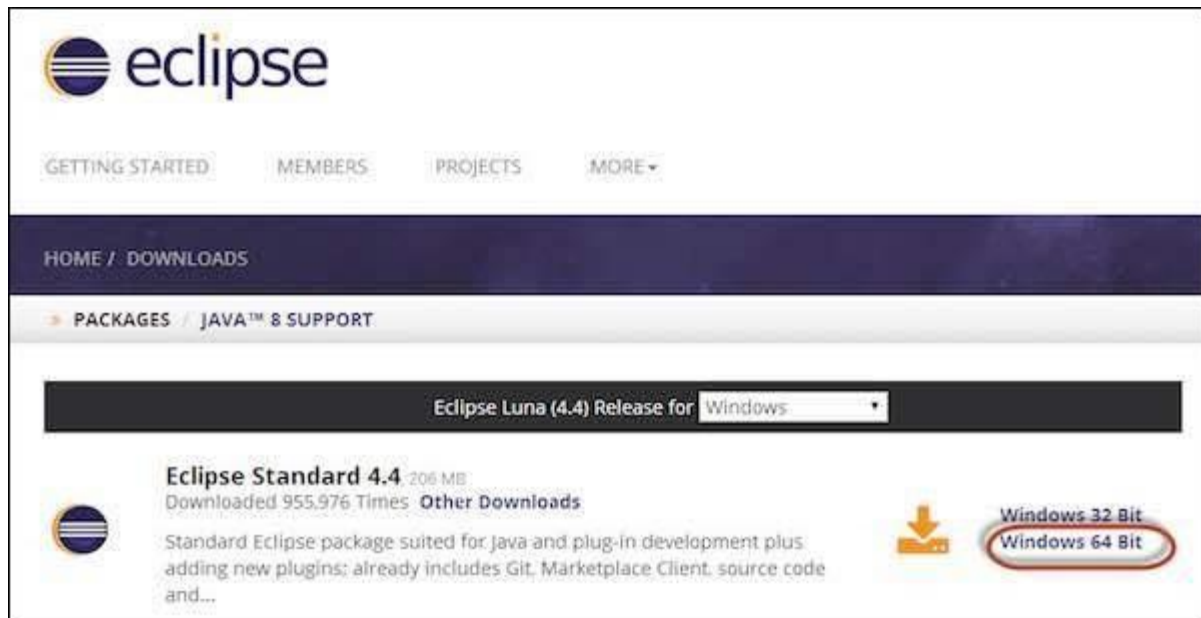
**Step 11** – To verify if the installation was successful, go to the command prompt and just type 'java' as a command. The output of the command is shown below. If the Java installation is unsuccessful or if it had NOT been installed, it would throw an "unknown command" error.

```
C:\Windows\system32\cmd.exe
G:\>java
Usage: java [-options] class [args...]
           (to execute a class)
or java [-options] -jar jarfile [args...]
           (to execute a jar file)
where options include:
    -d32          use a 32-bit data model if available
    -d64          use a 64-bit data model if available
    -server       to select the "server" VM
                  The default VM is server.

    -cp <class search path of directories and zip/jar files>
    -classpath <class search path of directories and zip/jar files>
                  A ; separated list of directories, JAR archives,
                  and ZIP archives to search for class files.
    -D<name>=<value>
                  set a system property
    -verbose[:<class|gc|jni>]
                  enable verbose output
    -version      print product version and exit
    -version:<value>
                  require the specified version to run
    -showversion  print product version and continue
    -jre-restrict-search | -no-jre-restrict-search
                  include/exclude user private JREs in the version search
    -? -help      print this help message
    -X            print help on non-standard options
    -ea[:<packagename>...![:<classname>]]
    -enableassertions[:<packagename>...![:<classname>]]
                  enable assertions with specified granularity
    -da[:<packagename>...![:<classname>]]
    -disableassertions[:<packagename>...![:<classname>]]
                  disable assertions with specified granularity
    -esa | -enablesystemassertions
                  enable system assertions
    -dsa | -disablesystemassertions
                  disable system assertions
    -agentlib:<libname>[=<options>]
                  load native agent library <libname>, e.g. -agentlib:hprof
                  see also, -agentlib:jdwp=help and -agentlib:hprof=help
    -agentpath:<pathname>[=<options>]
                  load native agent library by full pathname
    -javaagent:<jarpath>[=<options>]
                  load Java programming language agent, see java.lang.instrument
    -splash:<imagepath>
                  show splash screen with specified image
See http://www.oracle.com/technetwork/java/javase/documentation/index.html for more details.
G:\>
```

## Download and Configure Eclipse

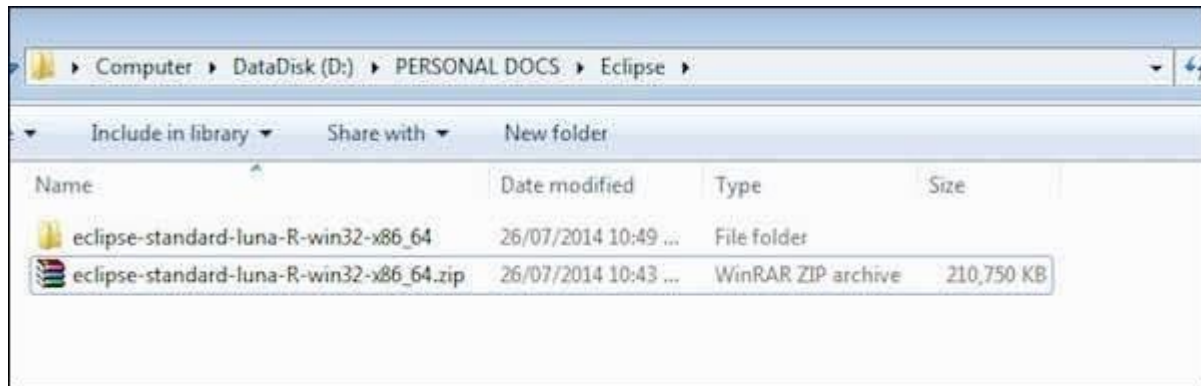
**Step 1** – Navigate to the URL: <https://www.eclipse.org/downloads/> and download the appropriate file based on your OS architecture.



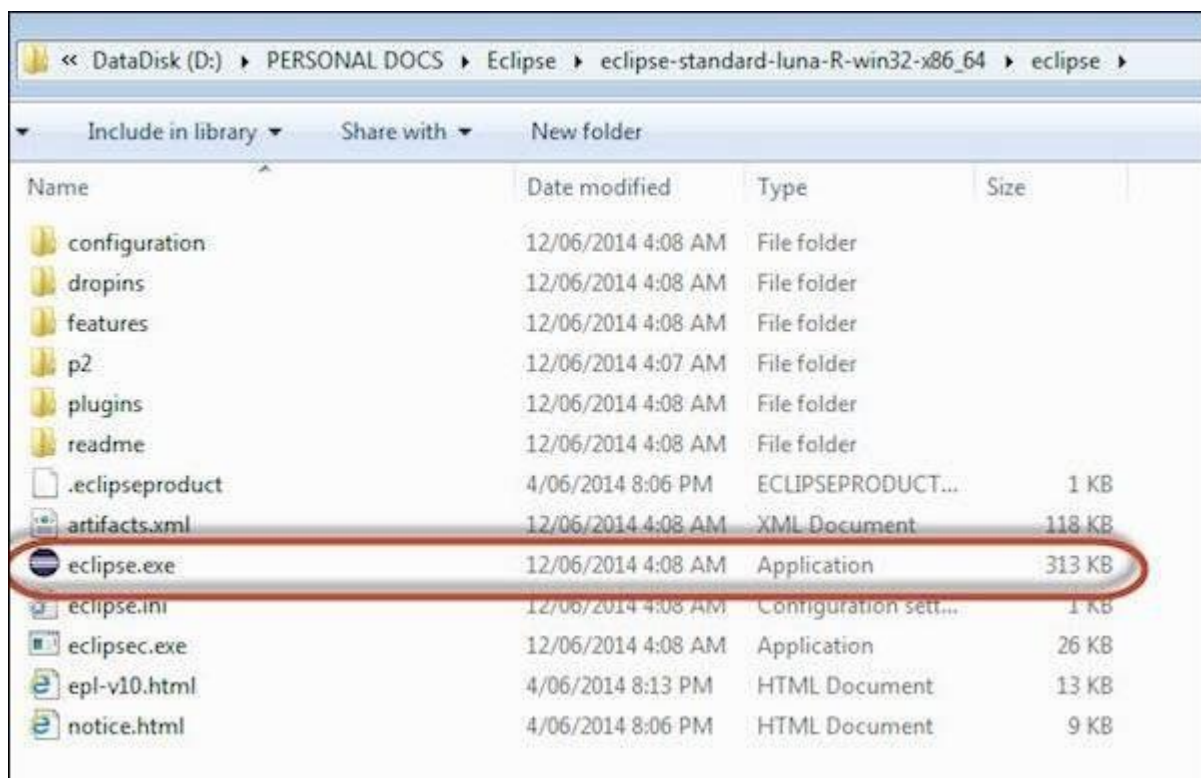
**Step 2** – Click the 'Download' button.



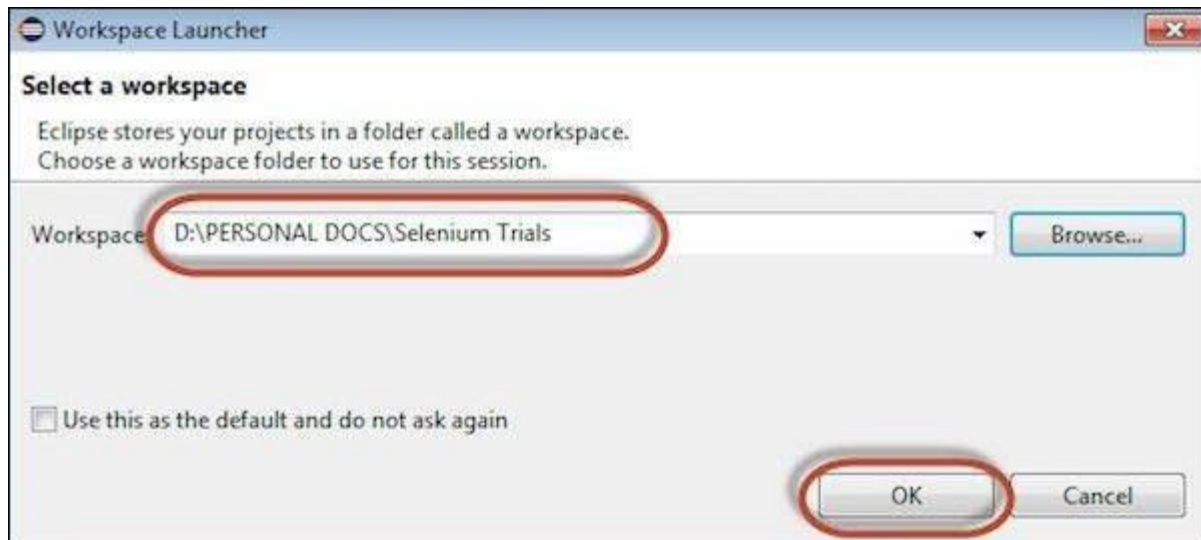
**Step 3** – The download would be in a Zipped format. Unzip the contents.



**Step 4** – Locate Eclipse.exe and double click on the file.



**Step 5** – To configure the workspace, select the location where the development has to take place.



**Step 6** – The Eclipse window opens as shown below.



### **Procedure:**

1. Launch Eclipse and select 'Install New Software'.
2. Enter the URL as 'http://beust.com/eclipse' and click 'Add'.
3. The dialog box 'Add Repository' opens. Enter the name as 'TestNG' and click 'OK'.
4. Click 'Select All' and 'TestNG' would be selected.
5. Click 'Next' to continue.
6. Review the items that are selected and click 'Next'.
7. "Accept the License Agreement" and click 'Finish'.

8. TestNG starts installing and the progress would be shown follows.
9. Security Warning pops up as the validity of the software cannot be established. Click 'Ok'.
10. The Installer prompts to restart Eclipse for the changes to take effect. Click 'Yes'.

### **TestNG-Eclipse Setup**

11. Launch Eclipse and create a 'New Java Project' as shown below.
12. Enter the project name and click 'Next'.
13. Navigate to "Libraries" Tab and Add the Selenium Remote Control Server JAR file by clicking on "Add External JAR's"
14. The added JAR file is shown here. Click 'Add Library'.
15. The 'Add Library' dialog opens. Select 'TestNG' and click 'Next' in the 'Add Library' dialog box.
16. The added 'TestNG' Library is added and it is displayed
17. Right-click on 'src' folder and select New >> Other.
18. Select 'TestNG' and click 'Next'.
19. Select the 'Source Folder' name and click 'Ok'.
20. Select the 'Package name', the 'class name', and click 'Finish'.
21. The Package explorer and the created class would be displayed
22. Write script using TestNG
23. To execute, right click on the created XML and select "Run As" >> "TestNG Suite"
24. The output is thrown to the console and it would appear

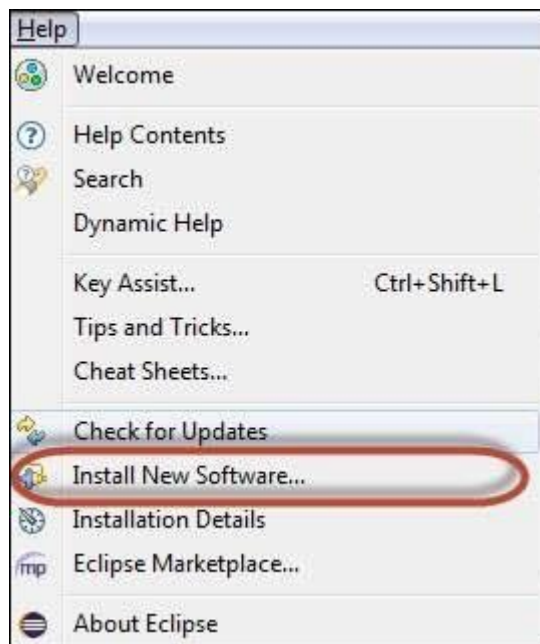
TestNG is a powerful testing framework, an enhanced version of JUnit which was in use for a long time before TestNG came into existence. NG stands for 'Next Generation'.

TestNG framework provides the following features –

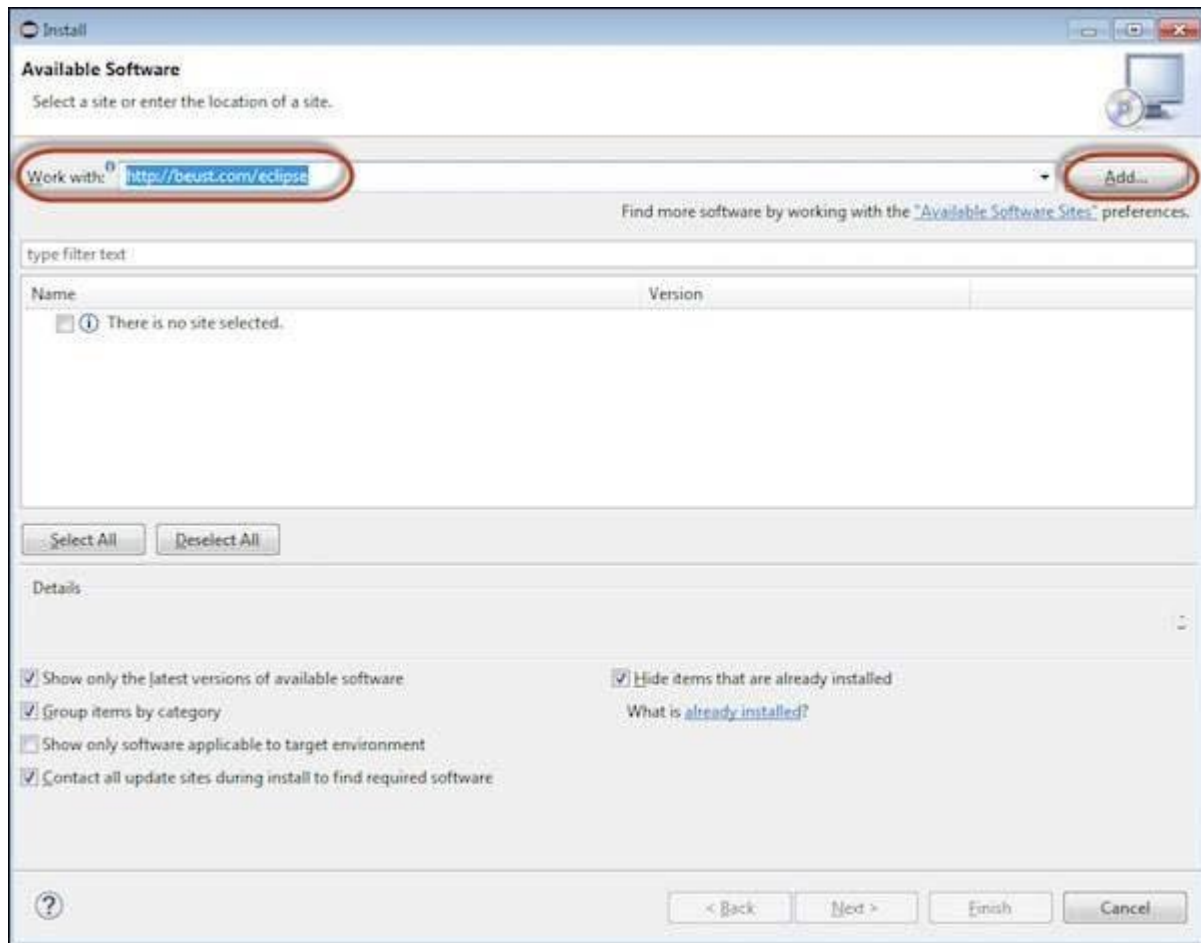
- Annotations help us organize the tests easily.
- Flexible test configuration.
- Test cases can be grouped more easily.
- Parallelization of tests can be achieved using TestNG.
- Support for data-driven testing.
- Inbuilt reporting.

### **Installing TestNG for Eclipse**

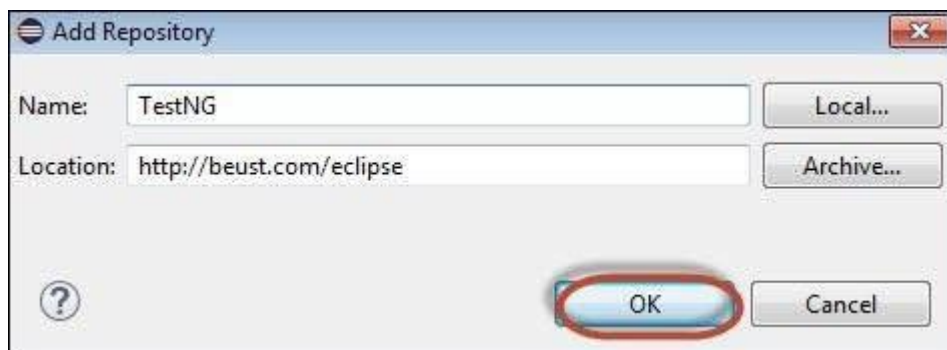
**Step 1** – Launch Eclipse and select 'Install New Software'.



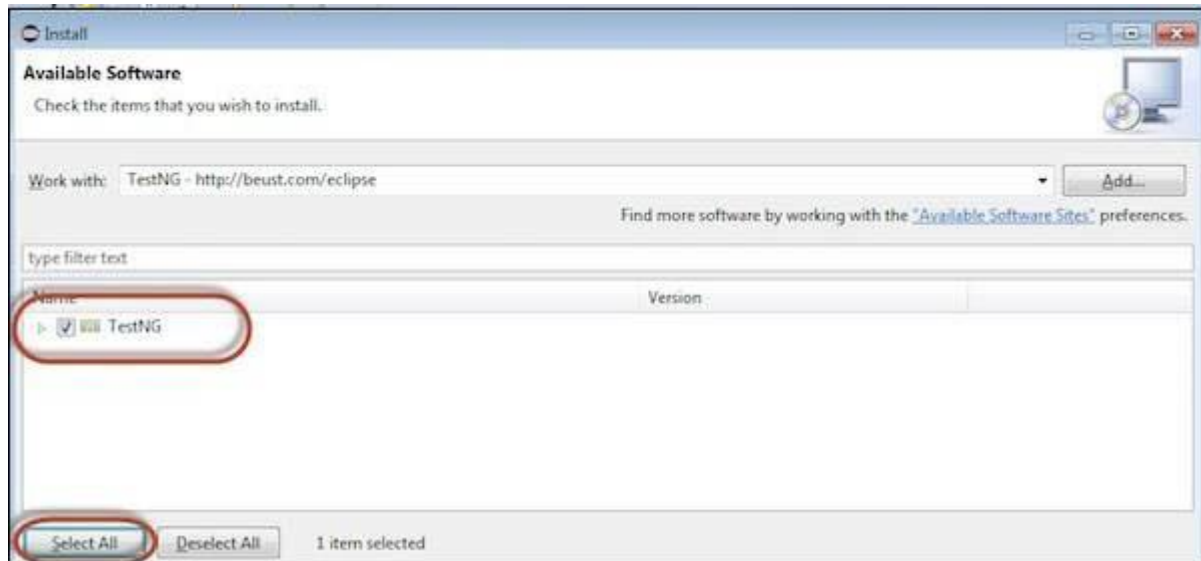
**Step 2** – Enter the URL as 'http://beust.com/eclipse' and click 'Add'.



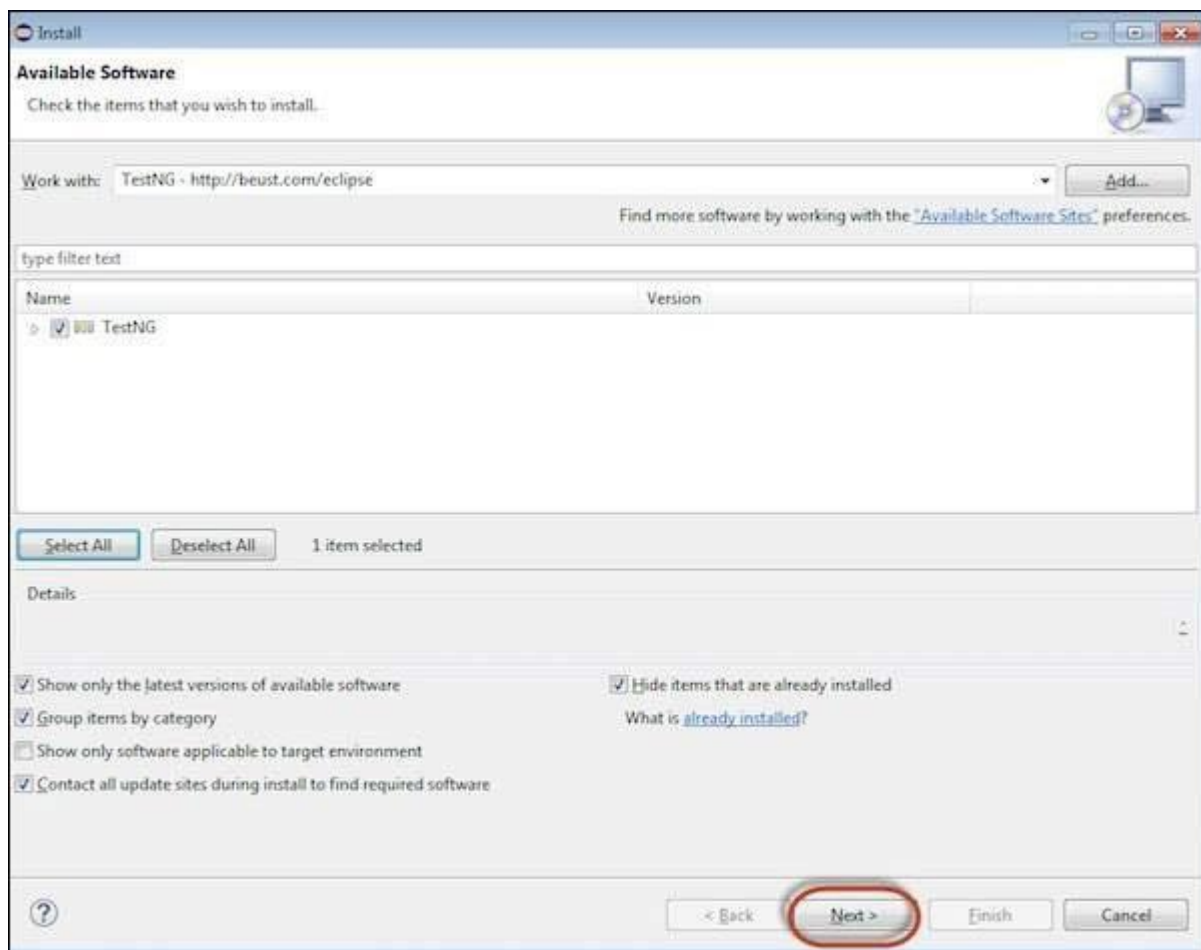
**Step 3** – The dialog box 'Add Repository' opens. Enter the name as 'TestNG' and click 'OK'



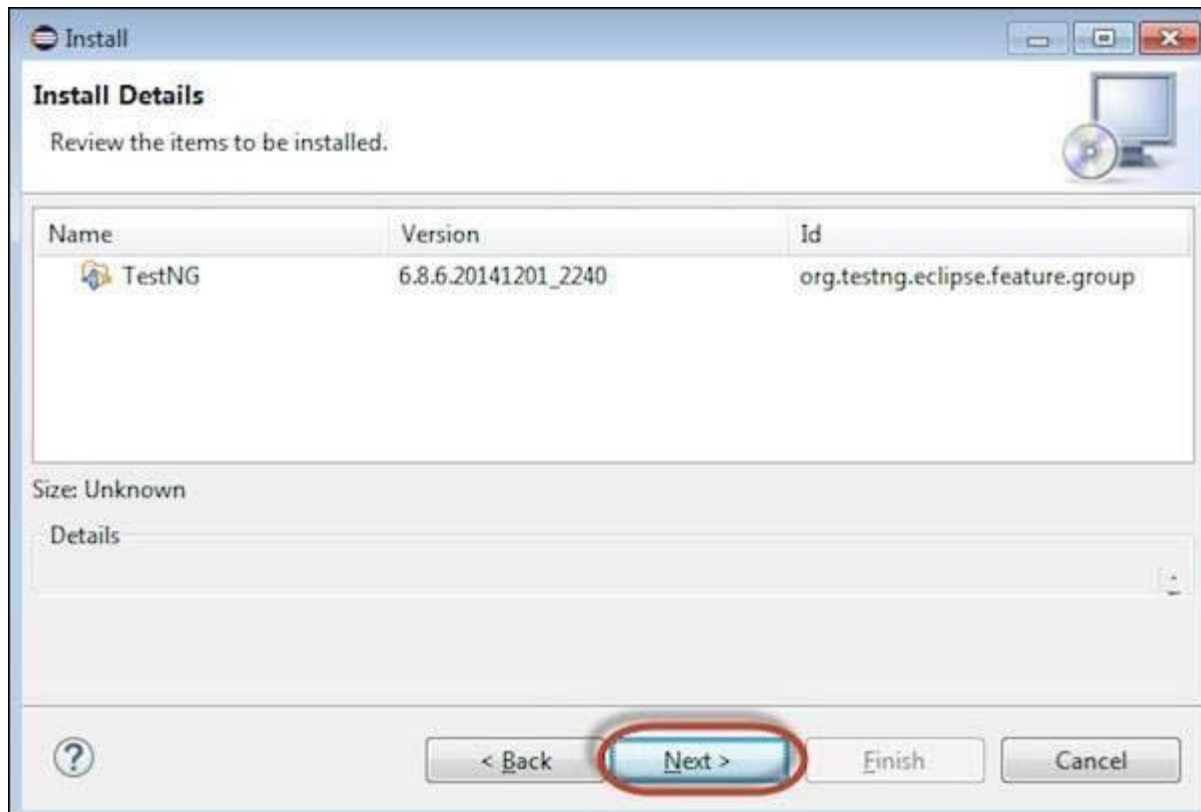
**Step 4** – Click 'Select All' and 'TestNG' would be selected as shown in the figure.



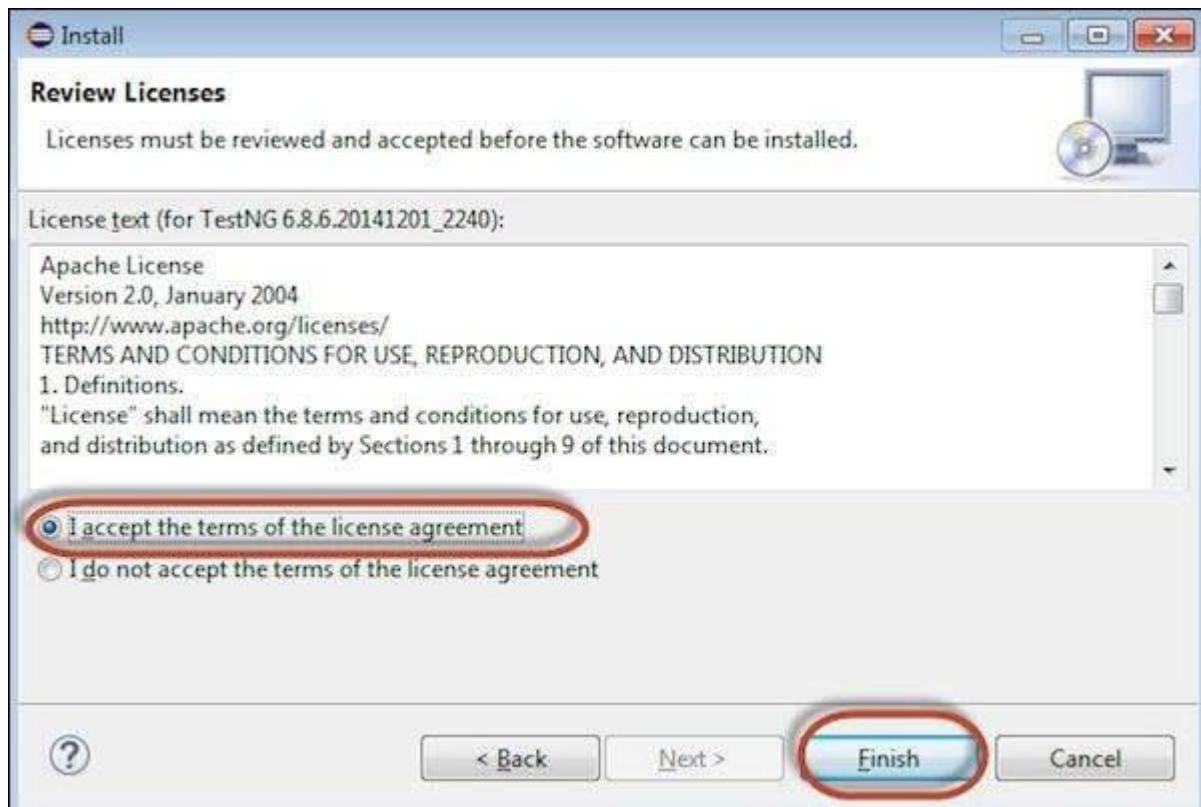
**Step 5** – Click 'Next' to continue.



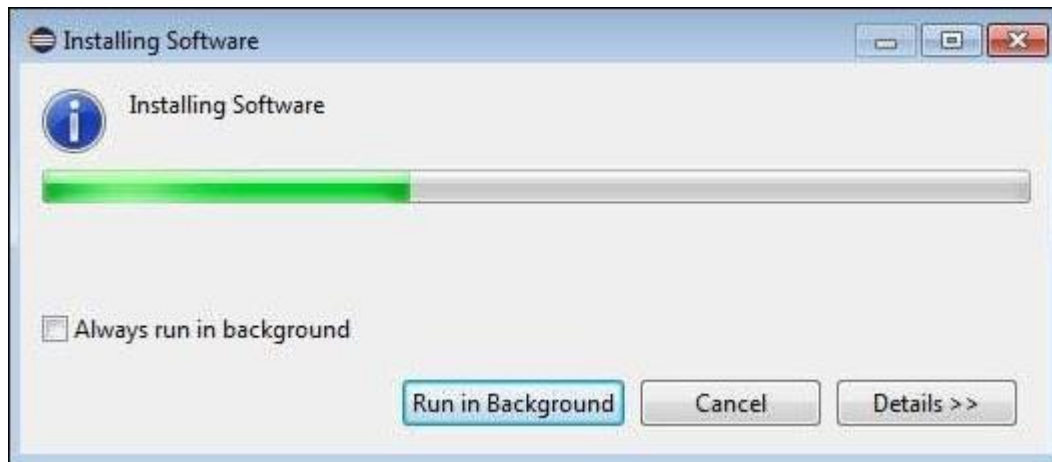
**Step 6** – Review the items that are selected and click 'Next'.



**Step 7** – "Accept the License Agreement" and click 'Finish'.



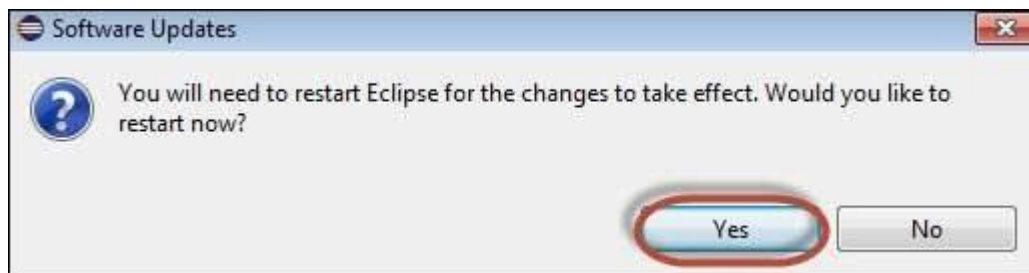
**Step 8** – TestNG starts installing and the progress would be shown follows.



**Step 9** – Security Warning pops up as the validity of the software cannot be established. Click 'Ok'.



**Step 10** – The Installer prompts to restart Eclipse for the changes to take effect. Click 'Yes'.



## Annotations in TestNG

Annotations were formally added to the Java language in JDK 5 and TestNG made the choice to use annotations to annotate test classes. Following are some of the benefits of using annotations.

- TestNG identifies the methods it is interested in by looking up annotations. Hence, method names are not restricted to any pattern or format.
- We can pass additional parameters to annotations.
- Annotations are strongly typed, so the compiler will flag any mistakes right away.

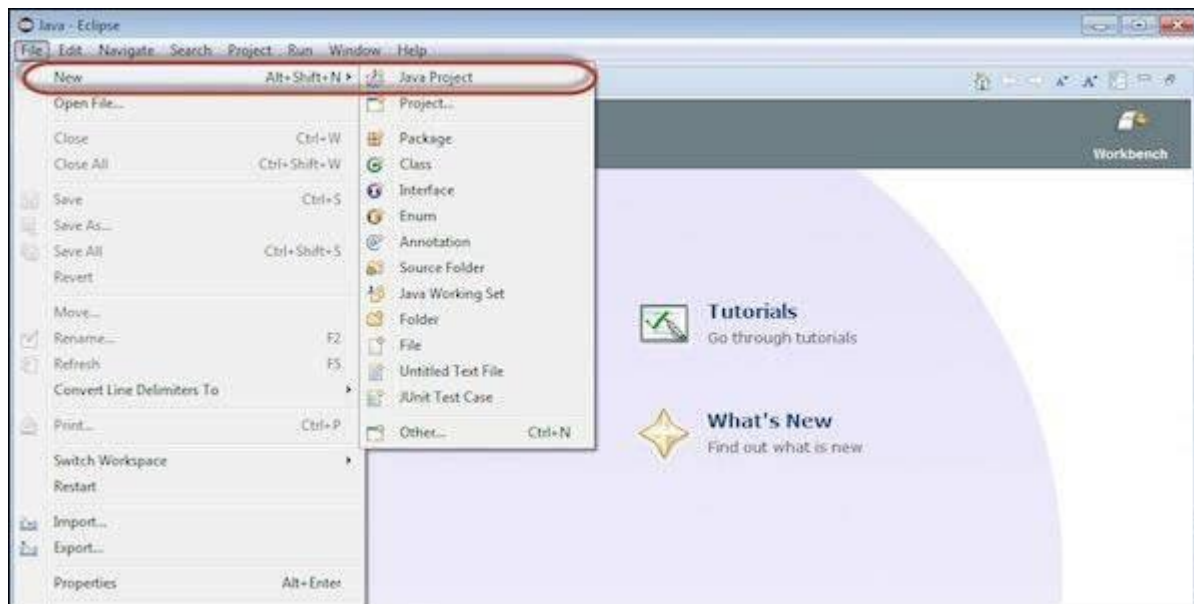
- Test classes no longer need to extend anything (such as TestCase, for JUnit 3).

### Experiment-11

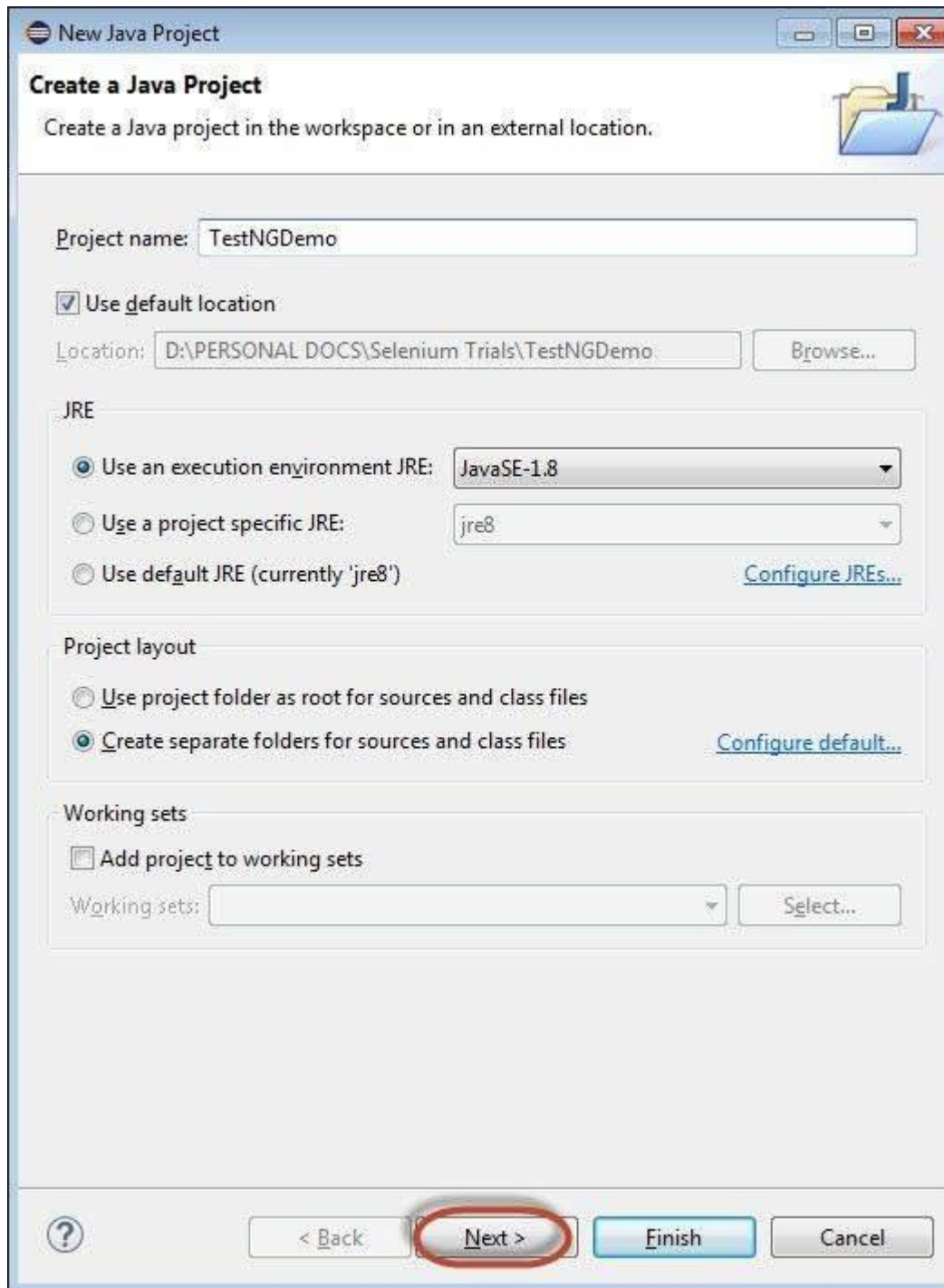
**Objective-** Generate Test sequences and validate using Selenium tool for calculator websites

#### TestNG-Eclipse Setup

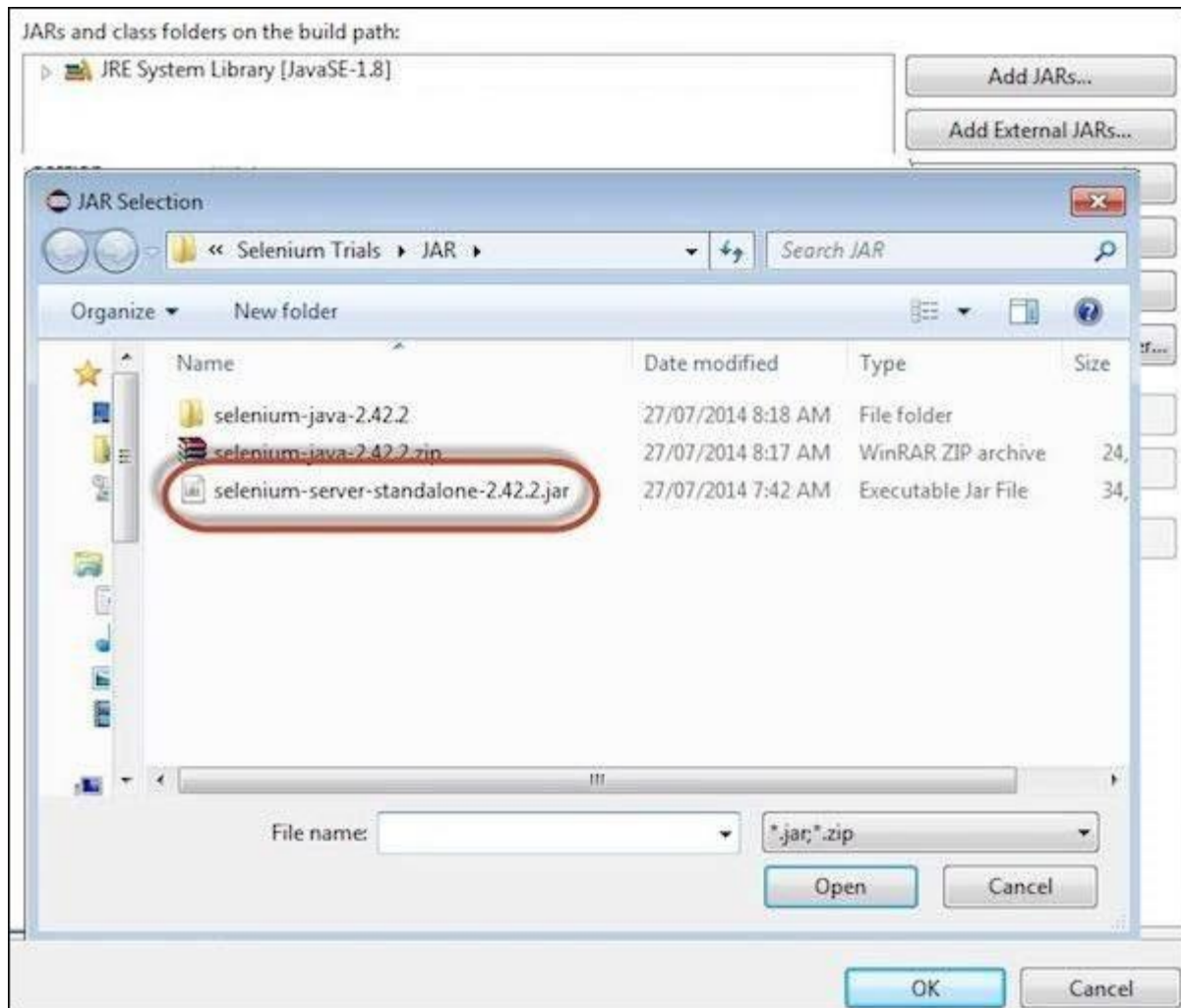
**Step 1** – Launch Eclipse and create a 'New Java Project' as shown below.



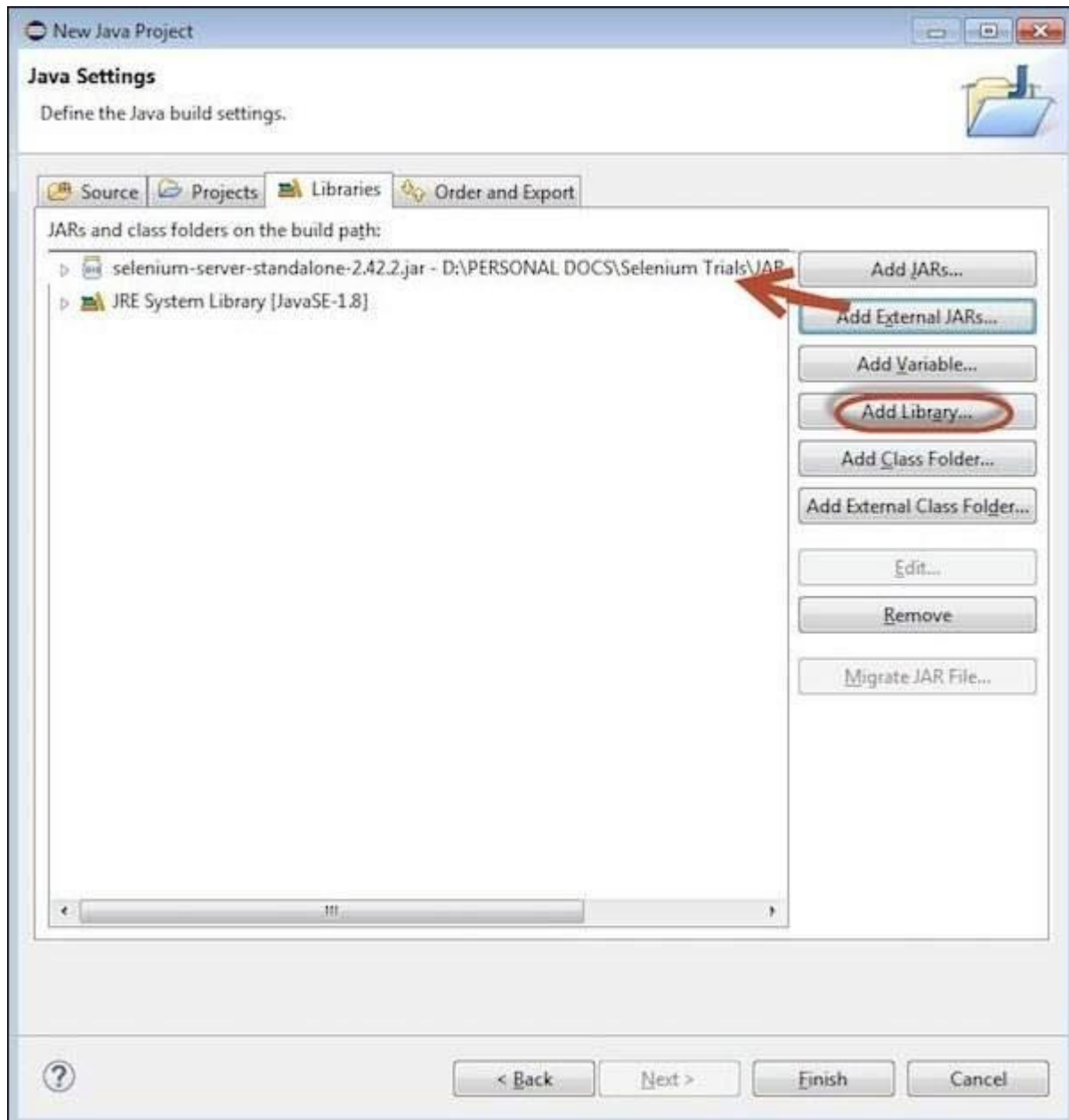
**Step 2** – Enter the project name and click 'Next'.



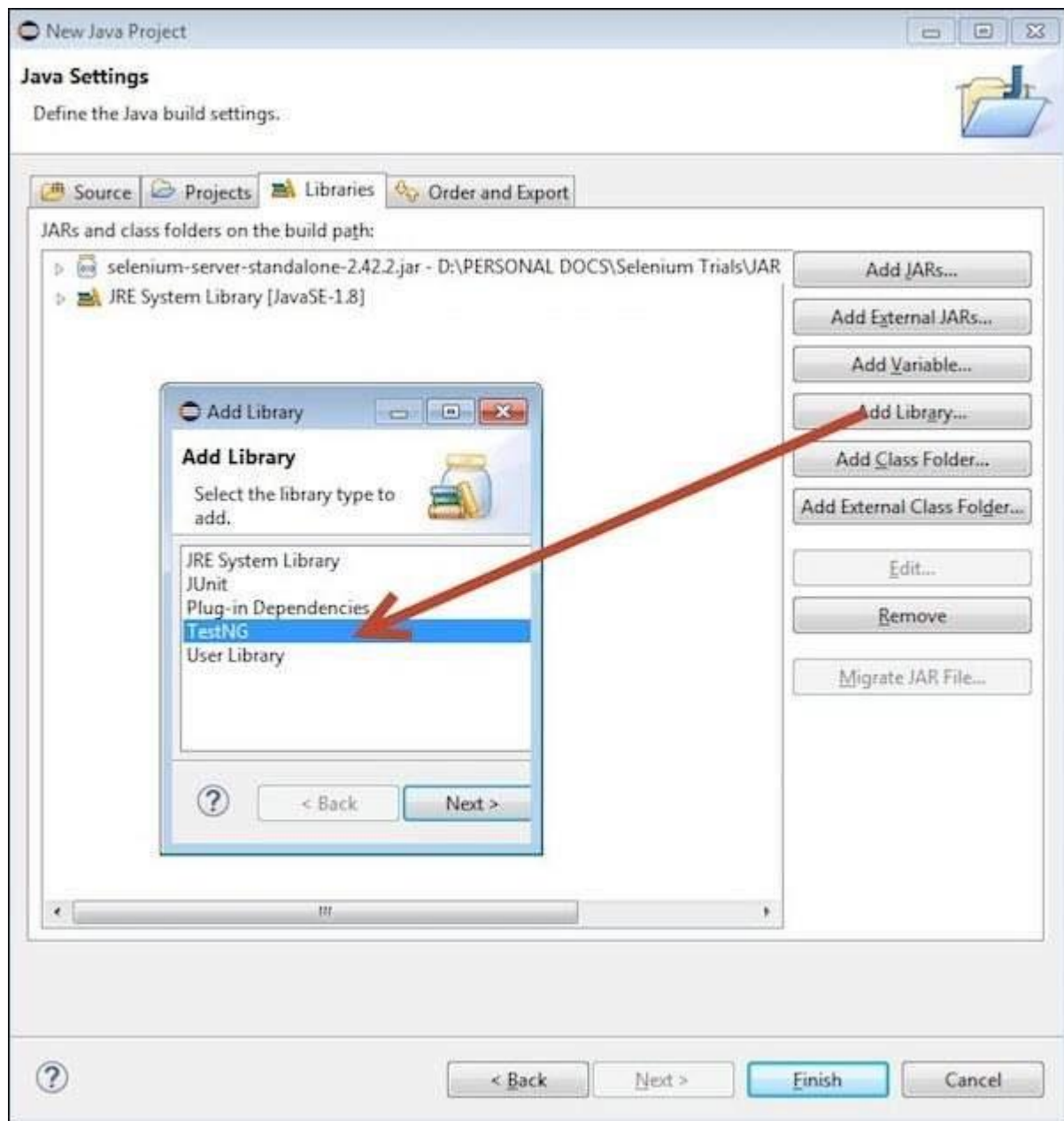
**Step 3** – Navigate to "Libraries" Tab and Add the Selenium Remote Control Server JAR file by clicking on "Add External JAR's" as shown below.



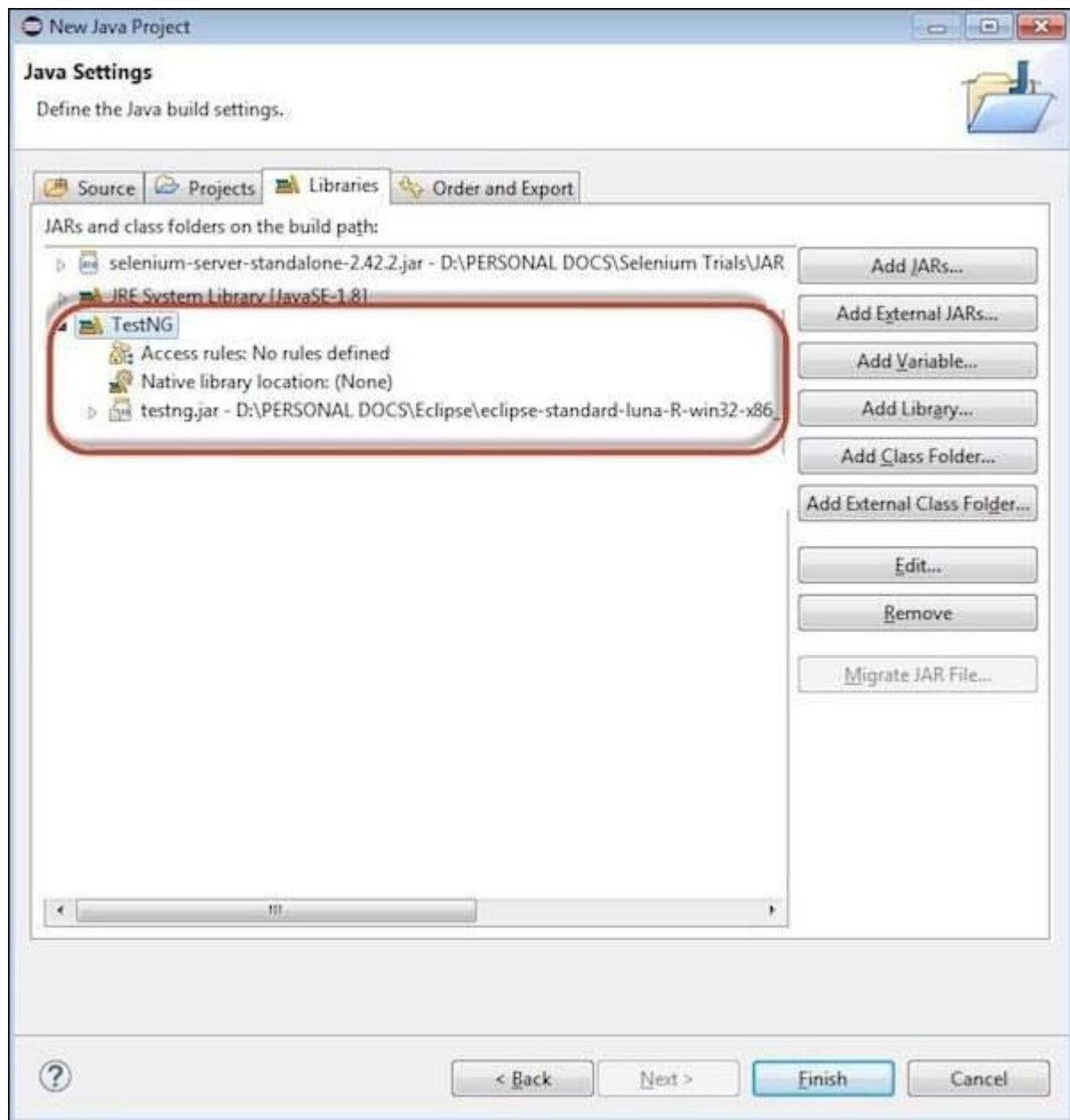
**Step 4** – The added JAR file is shown here. Click 'Add Library'.



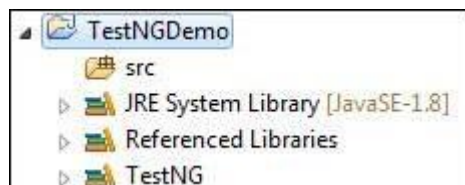
**Step 5** – The 'Add Library' dialog opens. Select 'TestNG' and click 'Next' in the 'Add Library' dialog box.



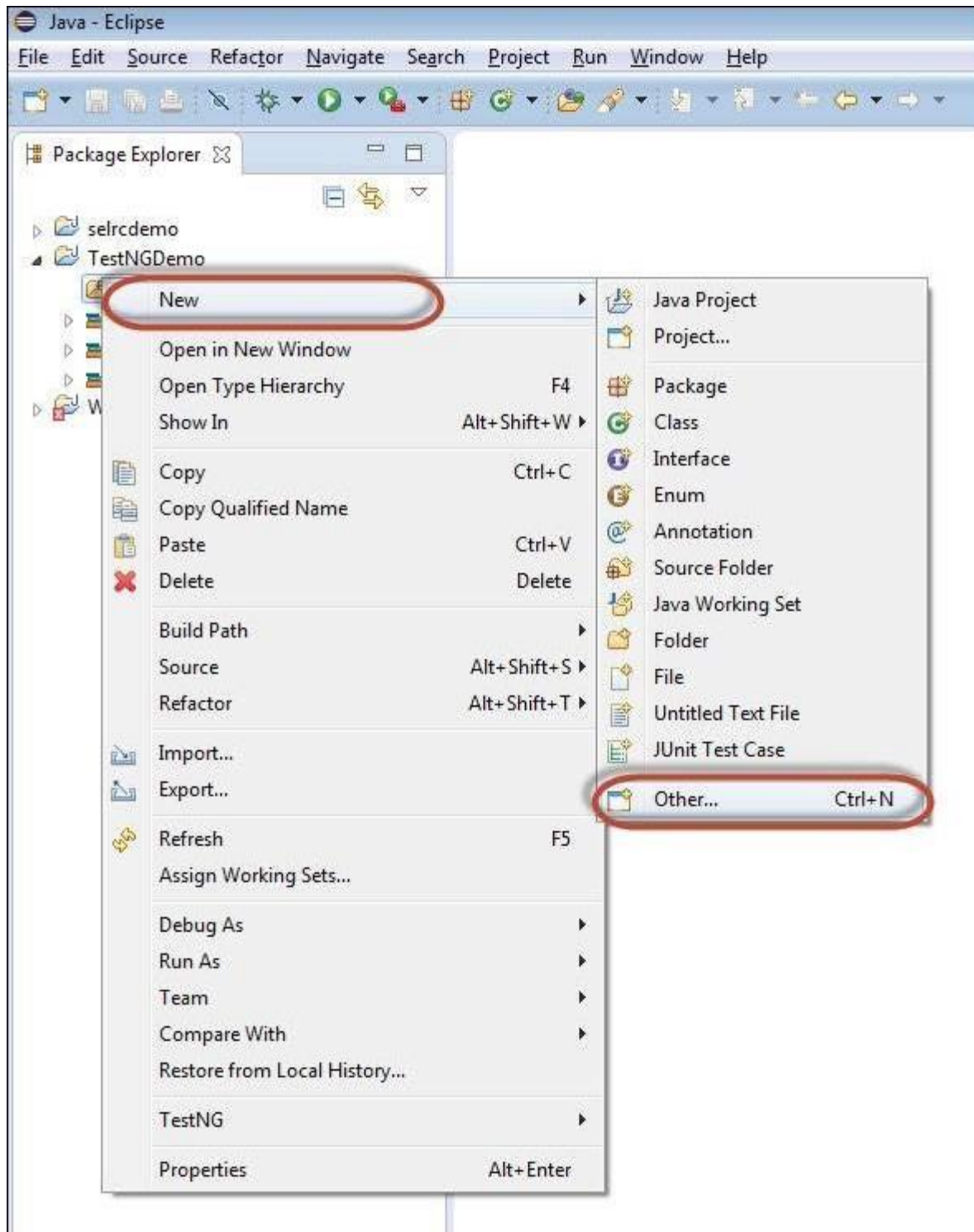
**Step 6** – The added 'TestNG' Library is added and it is displayed as shown below.



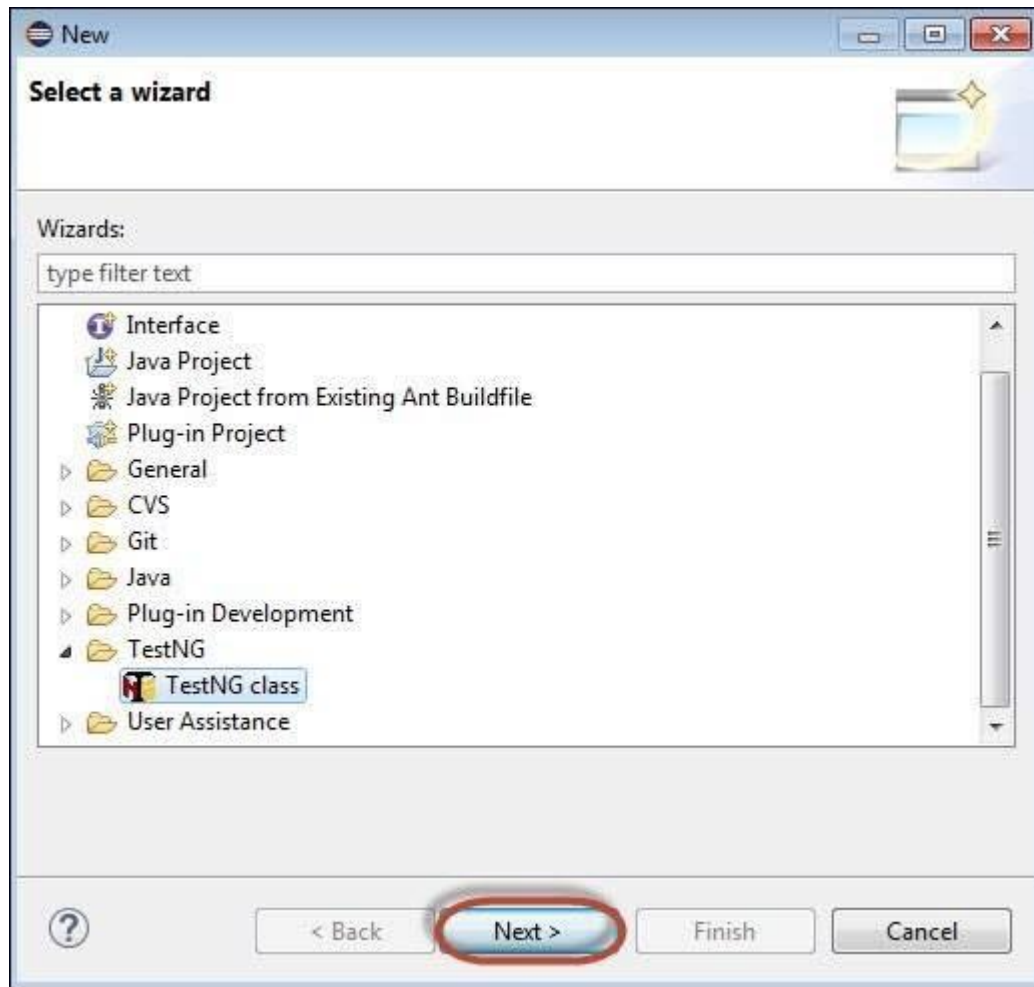
**Step 7** – Upon creating the project, the structure of the project would be as shown below.



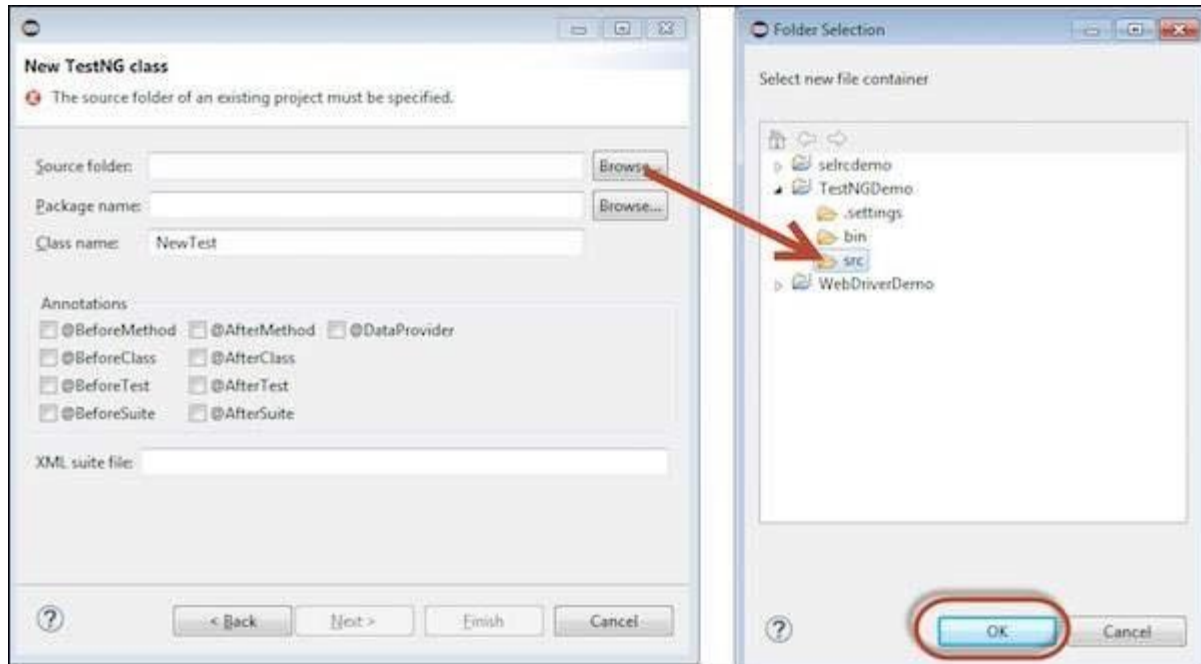
**Step 8** – Right-click on 'src' folder and select New >> Other.



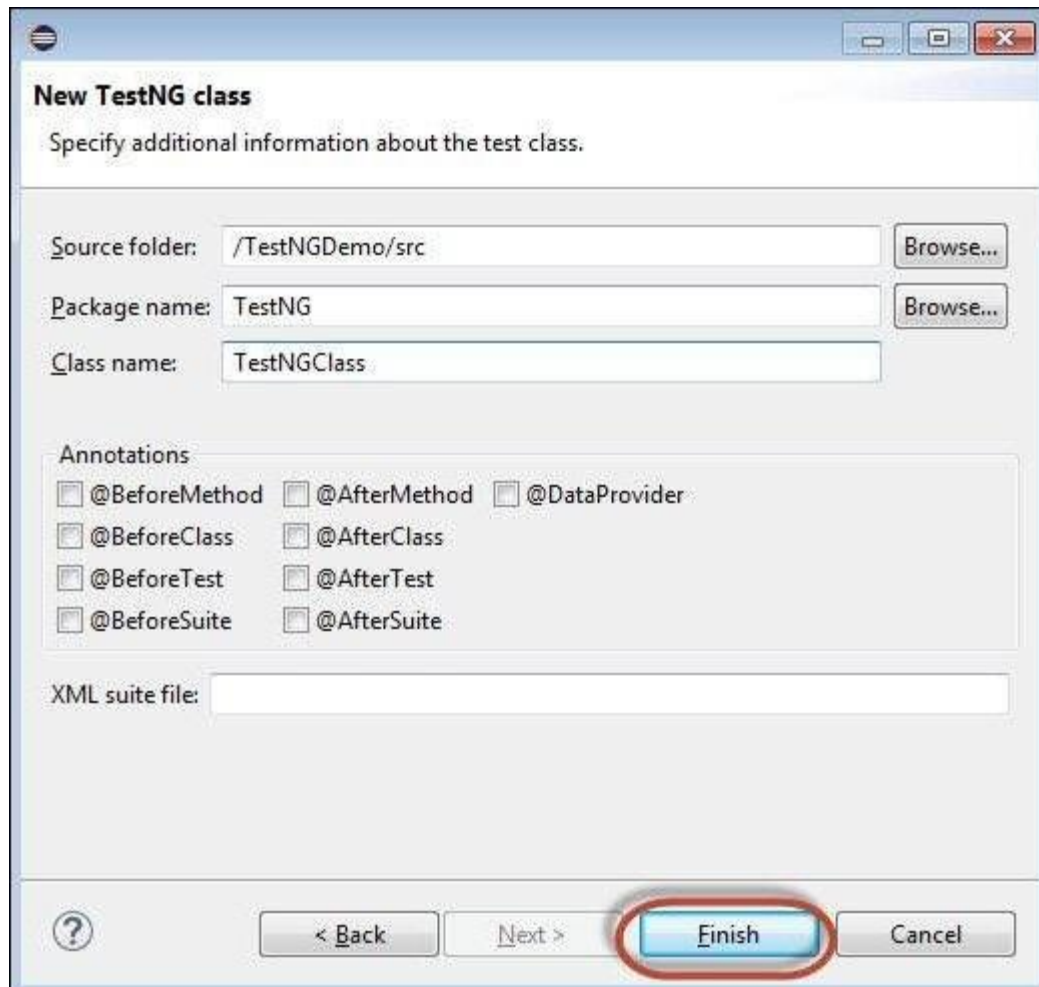
**Step 9** – Select 'TestNG' and click 'Next'.



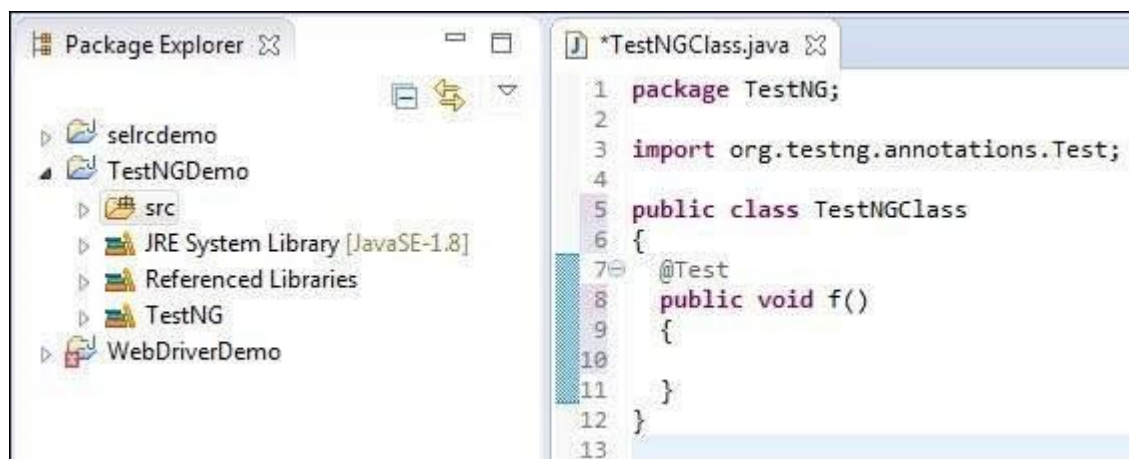
**Step 10** – Select the 'Source Folder' name and click 'Ok'.



**Step 11** – Select the 'Package name', the 'class name', and click 'Finish'.



**Step 12** – The Package explorer and the created class would be displayed.



## First Test in TestNG

Now let us start scripting using TestNG. Let us script for the same example that we used for understanding the WebDriver. We will use the demo application, [www.calculator.net](http://www.calculator.net), and perform percent calculator.

In the following test, you will notice that there is NO main method, as testNG will drive the program execution flow. After initializing the driver, it will execute the '@BeforeTest' method followed by '@Test' and then '@AfterTest'. Please note that there can be any number of '@Test' annotation in a class but '@BeforeTest' and '@AfterTest' can appear only once.

```
package TestNG;

import java.util.concurrent.TimeUnit;

import org.openqa.selenium.*;
import org.openqa.selenium.firefox.FirefoxDriver;

import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeTest;
import org.testng.annotations.Test;

public class TestNGClass {

    WebDriver driver = new FirefoxDriver();

    @BeforeTest

    public void launchapp() {

        // Puts an Implicit wait, Will wait for 10 seconds before throwing exception

        driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);

        // Launch website
```

```

driver.navigate().to("http://www.calculator.net");

driver.manage().window().maximize();

}

@Test
public void calculatepercent() {

    // Click on Math Calculators

    driver.findElement(By.xpath(".*[@id='menu']/div[3]/a")).click();

    // Click on Percent Calculators

    driver.findElement(By.xpath(".*[@id='menu']/div[4]/div[3]/a")).click();

    // Enter value 10 in the first number of the percent Calculator

    driver.findElement(By.id("cpar1")).sendKeys("10");

    // Enter value 50 in the second number of the percent Calculator

    driver.findElement(By.id("cpar2")).sendKeys("50");

    // Click Calculate Button

    driver.findElement(By.xpath(".*[@id='content']/table/tbody/tr/td[2]/input")).click();

    // Get the Result Text based on its xpath

    String result =

        driver.findElement(By.xpath(".*[@id='content']/p[2]/span/font/b")).getText();

```

```
// Print a Log In message to the screen

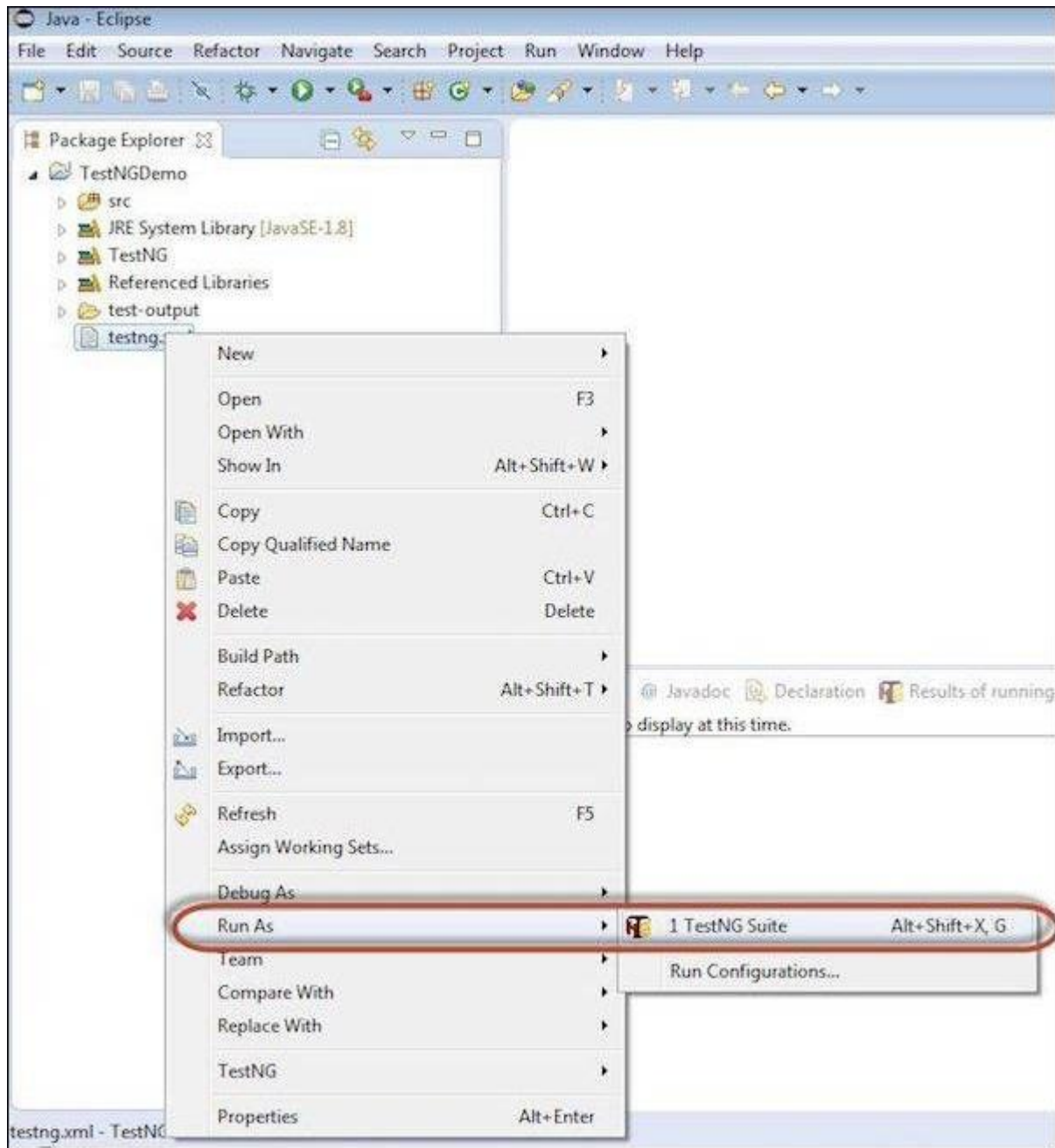
System.out.println(" The Result is " + result);

if(result.equals("5")) {
    System.out.println(" The Result is Pass");
} else {
    System.out.println(" The Result is Fail");
}
}

@AfterTest
public void terminatetest() {
    driver.close();
}
}
```

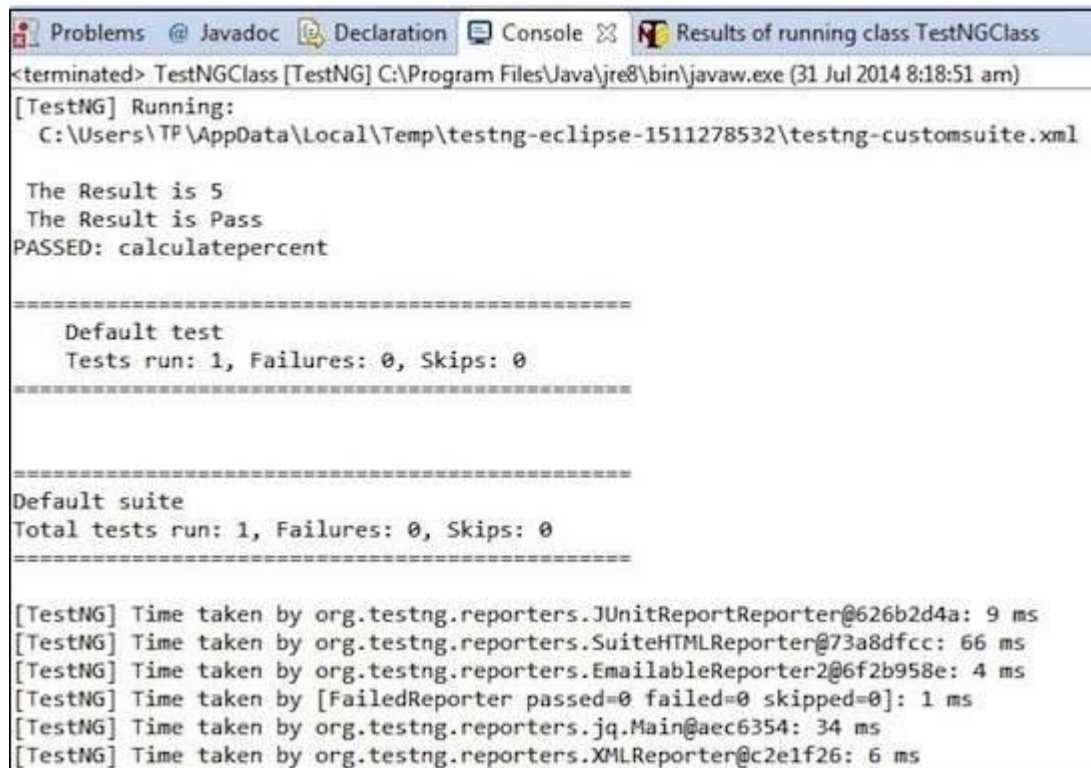
### **Execution**

To execute, right click on the created XML and select "Run As" >> "TestNG Suite"



## Result Analysis

The output is thrown to the console and it would appear as shown below. The console output also has an execution summary.

The screenshot shows the Eclipse IDE's console window with the title 'Results of running class TestNGClass'. The output text is as follows:

```
<terminated> TestNGClass [TestNG] C:\Program Files\Java\jre8\bin\javaw.exe (31 Jul 2014 8:18:51 am)
[TestNG] Running:
  C:\Users\TP\AppData\Local\Temp\testng-eclipse-1511278532\testng-customsuite.xml

The Result is 5
The Result is Pass
PASSED: calculatepercent

=====
    Default test
    Tests run: 1, Failures: 0, Skips: 0
=====

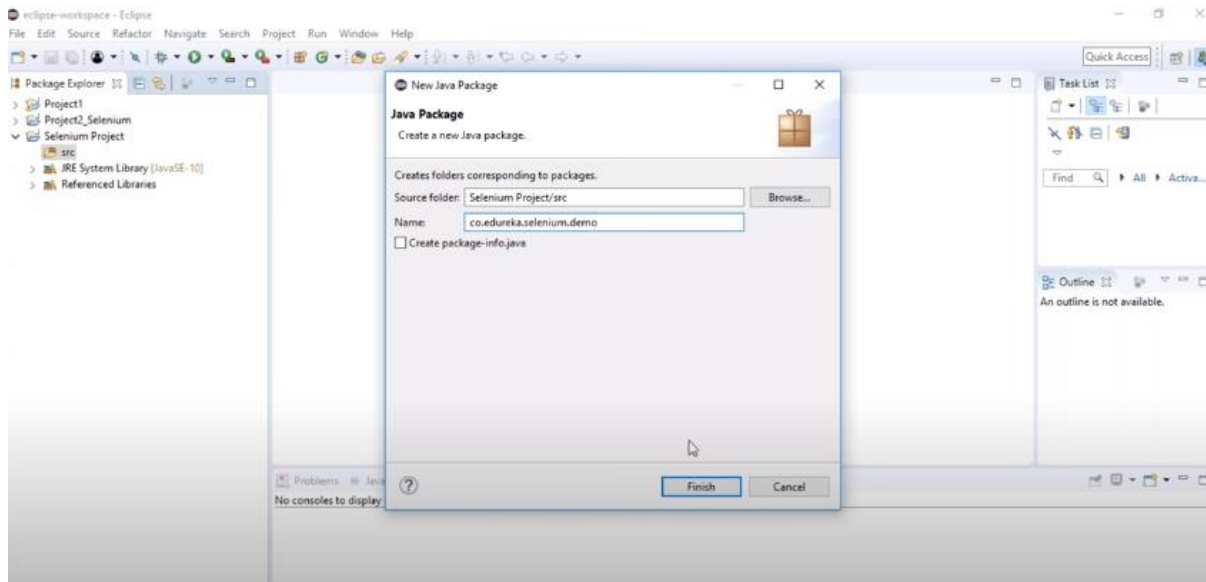
=====
Default suite
Total tests run: 1, Failures: 0, Skips: 0
=====

[TestNG] Time taken by org.testng.reporters.JUnitReportReporter@626b2d4a: 9 ms
[TestNG] Time taken by org.testng.reporters.SuiteHTMLReporter@73a8dfcc: 66 ms
[TestNG] Time taken by org.testng.reporters.EmailableReporter2@6f2b958e: 4 ms
[TestNG] Time taken by [FailedReporter passed=0 failed=0 skipped=0]: 1 ms
[TestNG] Time taken by org.testng.reporters.jq.Main@aec6354: 34 ms
[TestNG] Time taken by org.testng.reporters.XMLReporter@c2e1f26: 6 ms
```

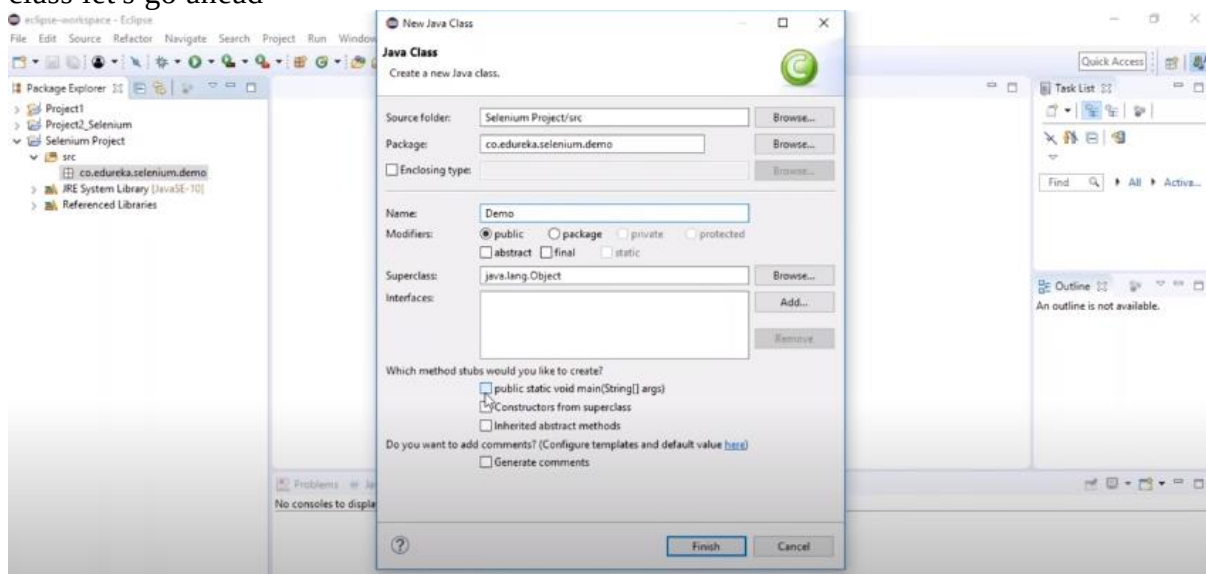
## Experiment-12

**Objective-** Automate and test gmail using Google Chrome

**Step-1** first test script what will we do we'll go ahead and first create a package just like Java goes so co dot edurayker dot let's say selenium dot demo right let's click on finish go ahead and create a class let's create my class

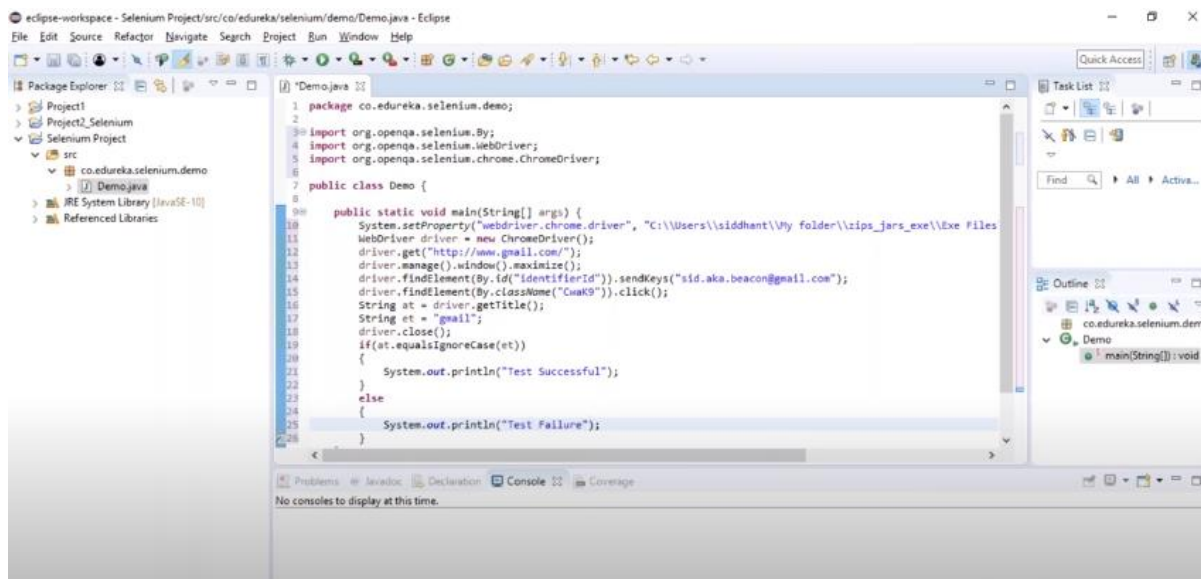


**Step-2** Now make sure this is our main method in it and click on finish so here we have our class let's go ahead

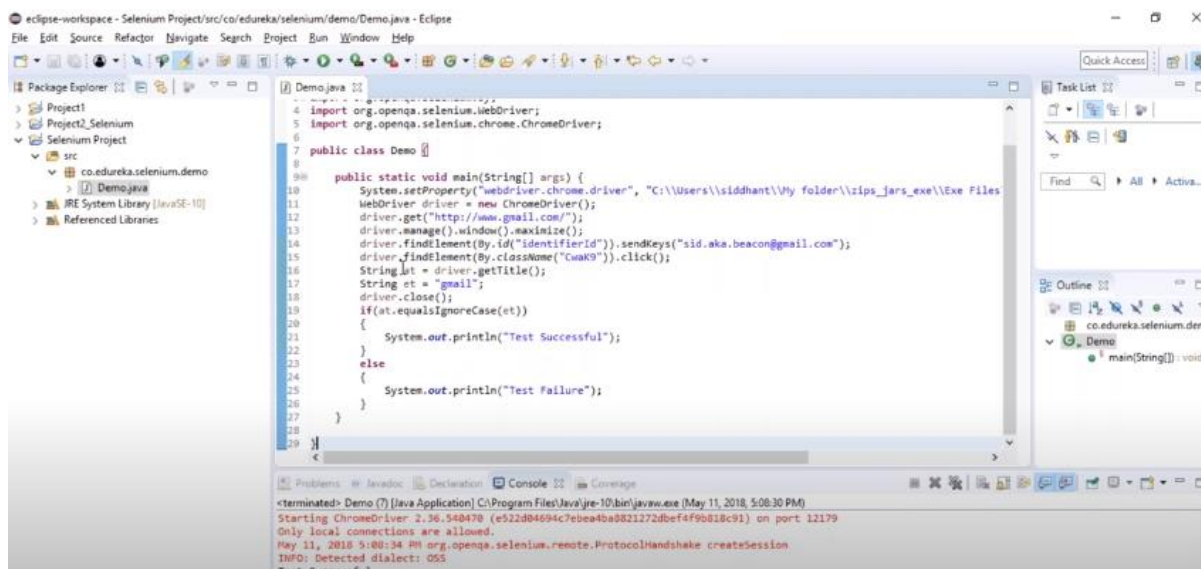


**Step-3** and write the test case so while I write the code. Drivers are really important things they are the ones that can instantiate your browser which means drivers are necessary if you want to automate your web browsers so we'll have to type in a command so that our drivers are connected to our program so the command for it is set property under which we'll have to tell the command about what driver we're talking about first we're talking about the web driver that is of Google Chrome

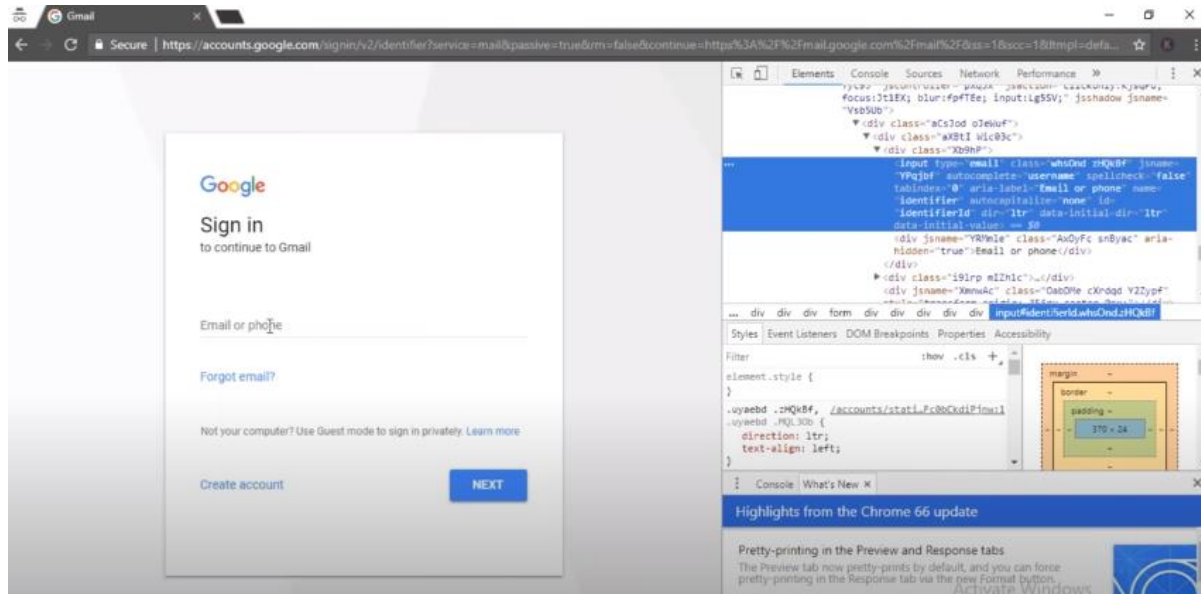
I'll have to go and find my Google Chrome driver which is chrome driver copy the location for it so this is the location of the folder



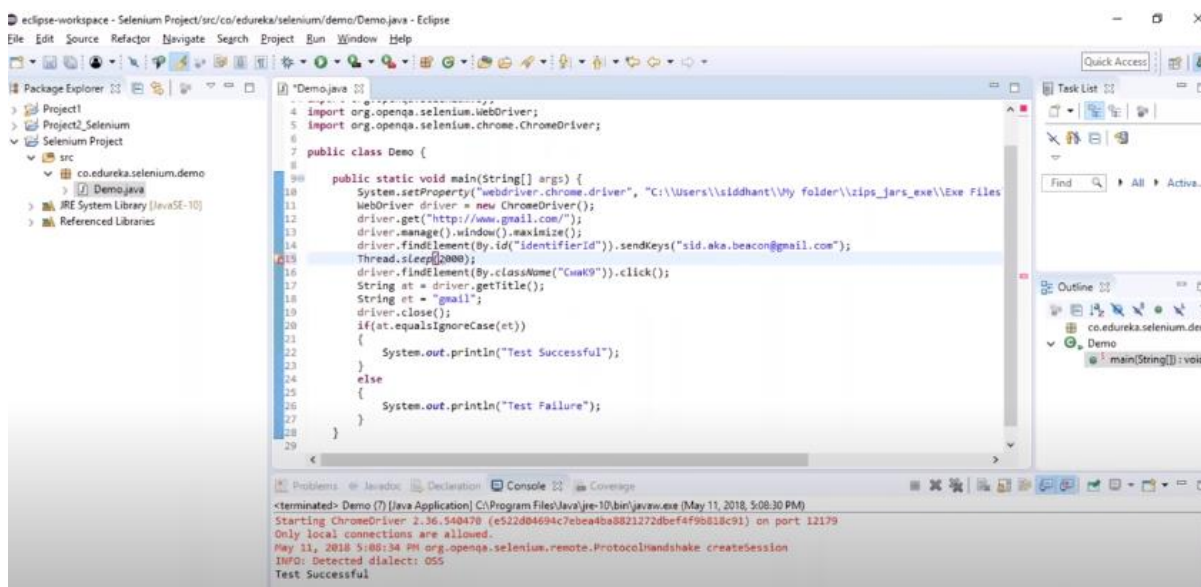
**Step-4** and we have successfully connected the driver to our programs constructor to initialize the reference variable with an object so now you can interface first and then let's go and import the class as well we've imported to call some methods that will now let's say open and website for me first so what website do we want to open we want to open Gmail so I'll have to type.



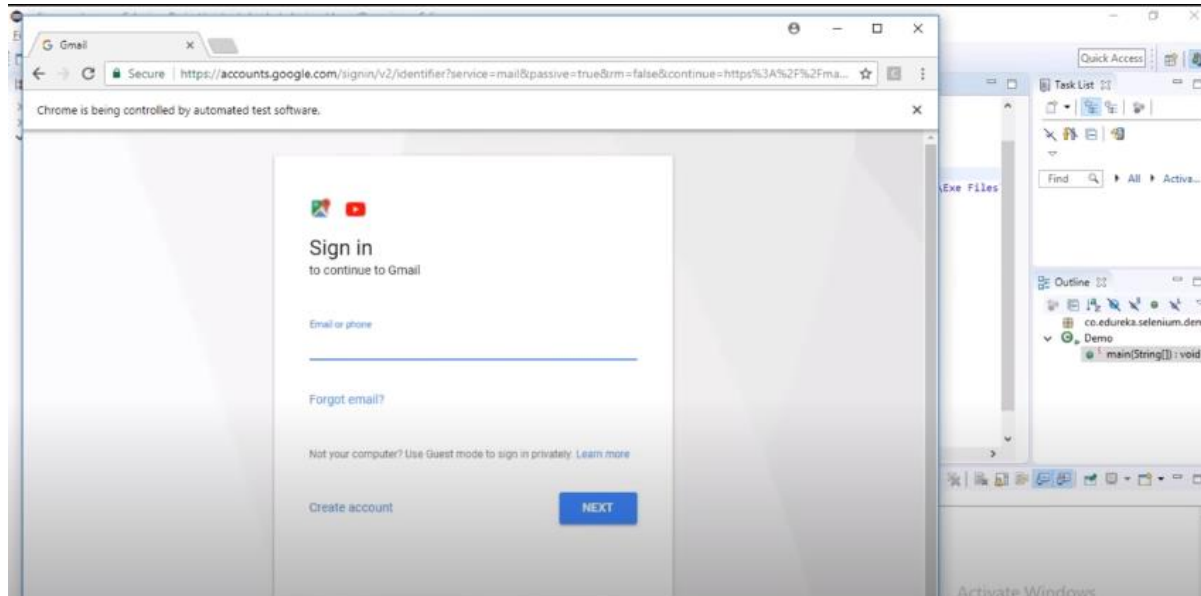
**Step-5** Web element to have a few attributes here so that is how we are going to find that web element and send some text to it which is in my case my email id. The element so we'll be using the ID name to find the element which is identifier ID so let's say by ID we'll have to tell it the name of the ID as well and we just saw the name of the ID



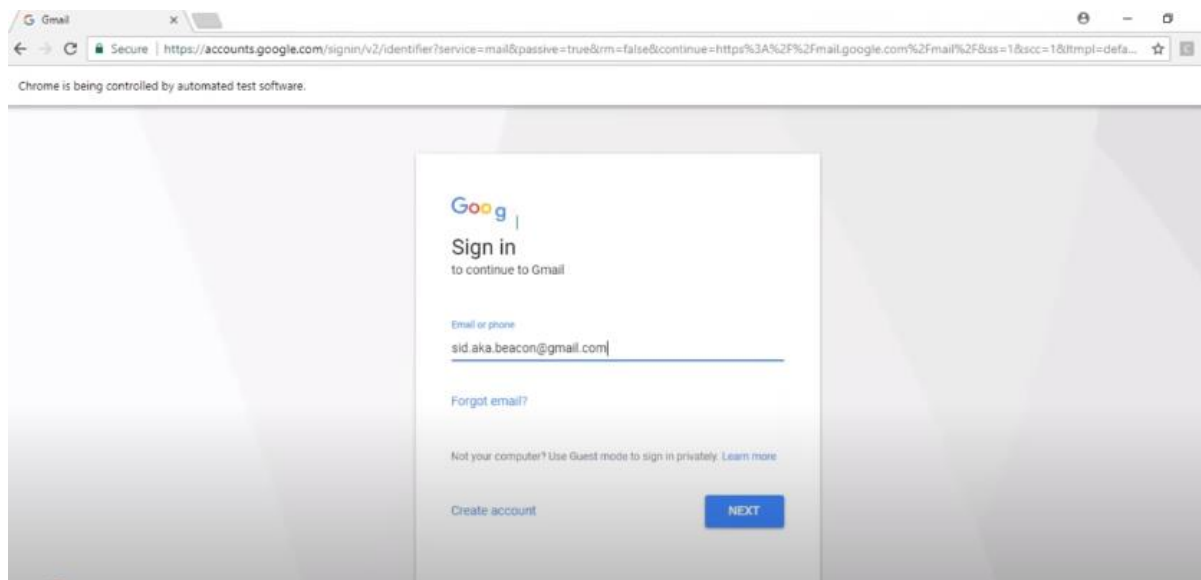
**Step-6** Build a small test case so I declare a variable of the string type. Execute it. Browser open automatically they're also navigated to Gmail and type my ID and close the window what it happened really fast so you people must not have seen everything but it's a successful which means our titles also match



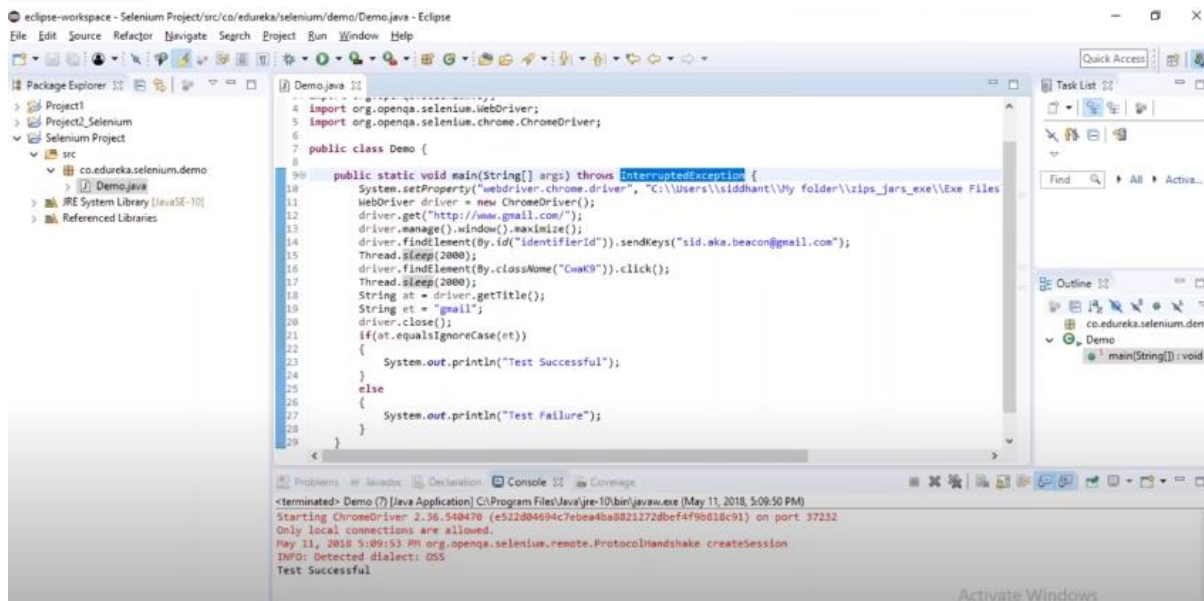
**Step-7** Now go ahead and run it okay so my browser is open and I have Gmail



**Step-8** My ID typed in with the screen got maximized I'm in the window and that's successful



**Step-9** It we also closed the browser and it says test was successful



## VIVA QUESTIONS

### 1. What is Selenium

Selenium is one of the most widely used open source Web UI (User Interface) automation testing suite. It was originally developed by Jason Huggins in 2004 as an internal tool at Thought Works. Selenium supports automation across different browsers, platforms and programming languages.

### 2. What are the limitations of Selenium?

- Selenium requires high skill sets in order to automate tests more effectively.
- Since Selenium is open source software, you have to rely on community forums to get your technical issues resolved.
- We can't perform automation tests on web services like SOAP or REST using Selenium.
- We should know at least one of the supported programming languages to create test scripts in Selenium WebDriver.
- Selenium does not have any inbuilt reporting capability; you have to rely on plug-ins like **JUnit** and **TestNG** for test reports.
- It is not possible to perform testing on images. We need to integrate Selenium with **Sikuli** for image based testing.

### 3. What are the features of Selenium?

- Selenium is an open source and portable Web testing Framework.
- Selenium IDE provides a playback and record feature for authoring tests without the need to learn a test scripting language.
- It can be considered as the leading cloud-based testing platform which helps testers to record their actions and export them as a reusable script with a simple-to-understand and easy-to-use interface.
- Selenium supports various operating systems, browsers and programming languages. Following is the list:
  - Programming Languages: C#, Java, Python, PHP, Ruby, Perl, and JavaScript
  - Operating Systems: Android, iOS, Windows, Linux, Mac, Solaris.
  - Browsers: Google Chrome, Mozilla Firefox, Internet Explorer, Edge, Opera, Safari, etc.
- It also supports parallel test execution which reduces time and increases the efficiency of tests.

#### 4. What are the features of TestNG frameworks?

TestNG framework provides the following features –

- Annotations help us organize the tests easily.
- Flexible test configuration.
- Test cases can be grouped more easily.
- Parallelization of tests can be achieved using TestNG.
- Support for data-driven testing.
- Inbuilt reporting.
- 

#### 5. What are the advantages of Selenium?

Advantages of Selenium

QTP and Selenium are the most used tools in the market for software automation testing. Hence it makes sense to compare the pros of Selenium over QTP.

| Selenium                         | QTP                                          |
|----------------------------------|----------------------------------------------|
| Selenium is an open-source tool. | QTP is a commercial tool and there is a cost |

|                                                                                                    |                                                                                                                                            |
|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                    | involved in each one of the licenses.                                                                                                      |
| Can be extended for various technologies that expose DOM.                                          | Limited add-ons and needs add-ons for each one of the technologies.                                                                        |
| Has capabilities to execute scripts across different browsers.                                     | Can run tests in specific versions of Firefox , IE, and Chrome.                                                                            |
| Can execute scripts on various operating systems.                                                  | Works only with Windows.                                                                                                                   |
| Supports mobile devices.                                                                           | Supports mobile devices with the help of third-party tools.                                                                                |
| Executes tests within the browser, so focus is NOT required while script execution is in progress. | Needs Focus during script execution, as the tool acts on the browser (mimics user actions).                                                |
| Can execute tests in parallel with the use of Selenium Grids.                                      | QTP cannot execute tests in parallel, however integrating QTP with QC allows testers to execute in parallel. QC is also a commercial tool. |

Content Beyond syllabus:

## Study of any Open-Source Tool-

**NAME OF THE EXPERIMENT:** Study of any open source testing tool (TestLink)

Testlink is an open source test management tool. It enables creation and organization of test cases and helps manage into test plan.

### Login to TestLink

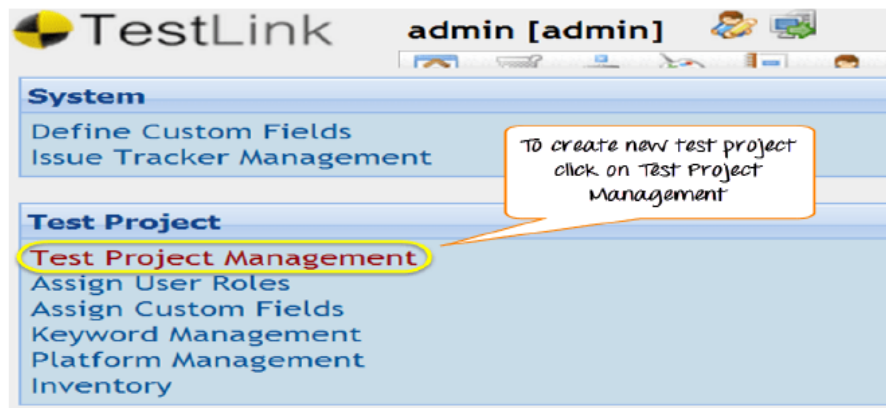
**Step 1 :** Open the Testlink home-page and enter the login details

1. Enter the userID – admin
2. Enter the password
3. Click on the login tab



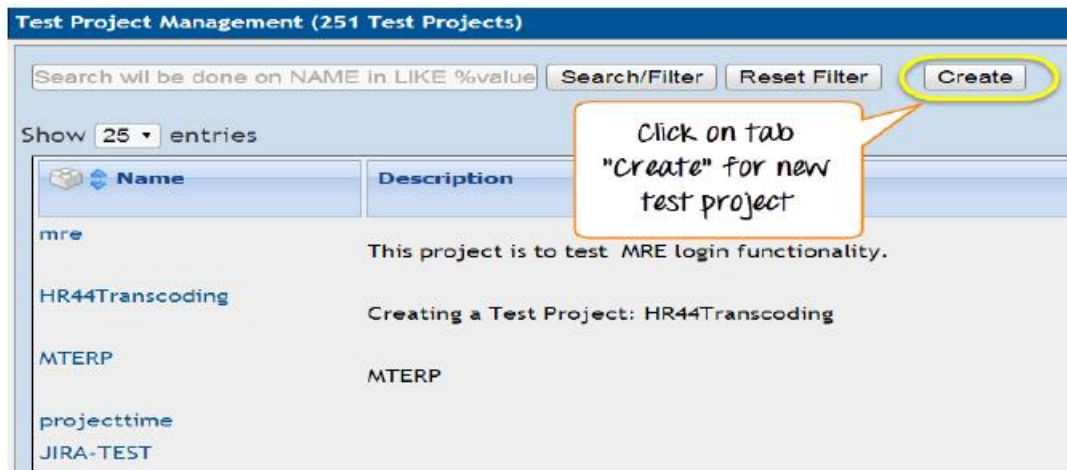
### Creating a Test Project

**Step 1:** In the main window click on Test Project Management, it will open another window

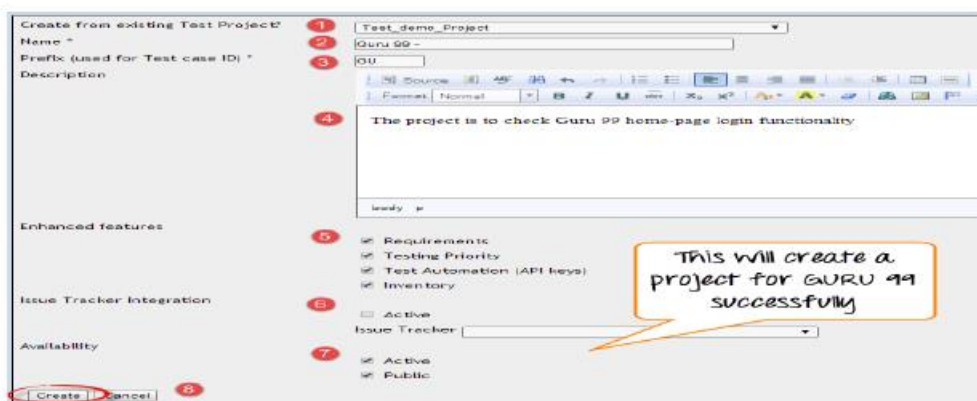


TestLink

Step 2: Click on tab "create" to create a new project.



Step 3: Enter all the required fields in the window like category for test project, name of the project, prefix, description, etc. After filling all necessary details, click on tab "Create" at the end of the window.



This will create your project "Guru99" successfully.



## Creating a Test Plan

Test plan holds the complete information like scope of Software testing, milestone, test suites and test cases. Once you have created a Test Project, next step is to create Test plan.

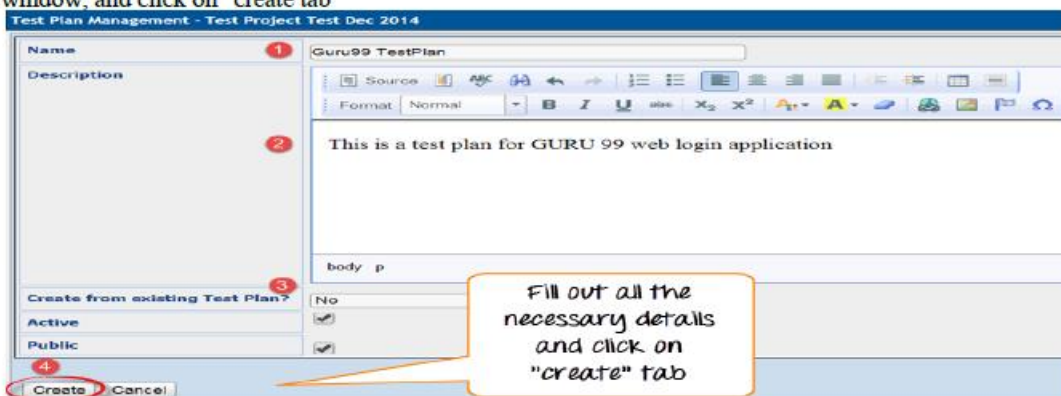
Step 1: From the home-page, click on Test Plan Management from home-page



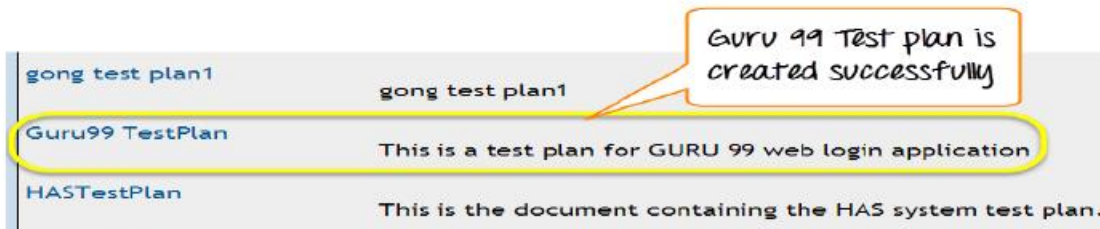
Step 2: It will open another page, at the bottom of the page click on a tab "Create"



Step 3: Fill out all the necessary information like name, description, create from existing test plan, etc. in the open window, and click on "create tab"



Step 4: Guru 99 Test Plan is created successfully



## Build Creation

Build is a specific release of software

Step 1: Click on Builds/Releases under Test Plan from the home page

## Test Plan

### Test Plan Management

#### Builds / Releases

#### Milestones

From home-page select  
Build/Release for creating specific  
release for your software

**Step 2:** In the next window, fill all necessary details for software release and click on create to save your release

1. Enter the title name
2. Enter the description about the software release
3. Mark the check-box for status- Active
4. Mark the check-box for status- Open
5. Choose the data of release
6. Click on create button

Create a new Build

Title: 1 Guru 99- sample build

Description: 2 This is a sample build for Guru 99 web application

Active: 3 ☒

Open: 4 ☒

Release date: 15/01/2016

Copy tester assignments: ☐

Create: 6 Cancel

Once you have a release the software it, will appear like this

| Title                 | Description                                        |
|-----------------------|----------------------------------------------------|
| Guru 99- sample build | This is a sample build for Guru 99 web application |
| Test Build            | Just for testing                                   |
| samplebuild           | this is sample build                               |

Sample build for  
Guru 99 is created  
successfully

### Creating Test suite

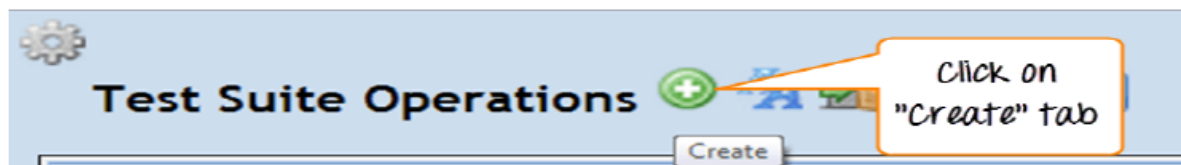
Test suite is a collection of test cases which may be testing or validating the same component. Following steps will explain how to create test suite for your project.

Step 1: Click on test specification option from the home page.



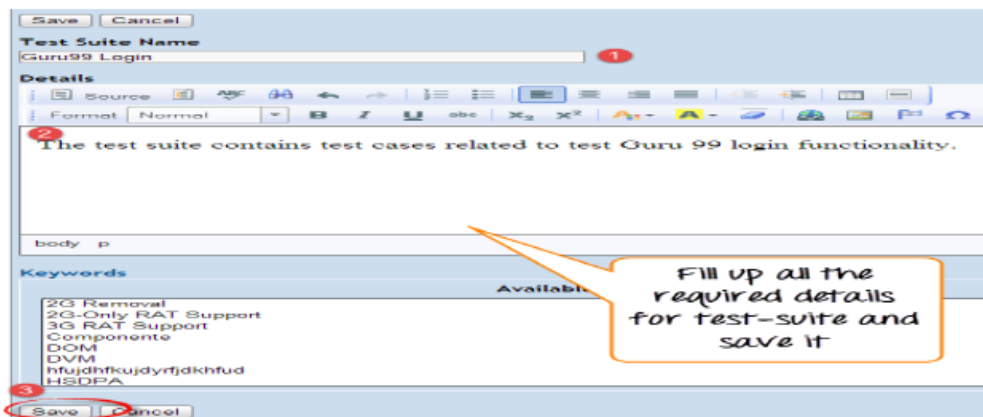
Step 2: On the right-hand side of the panel, click on the setting icon . It will display a series of test operation.

Step 3: Click on the "create" tab for the test suite

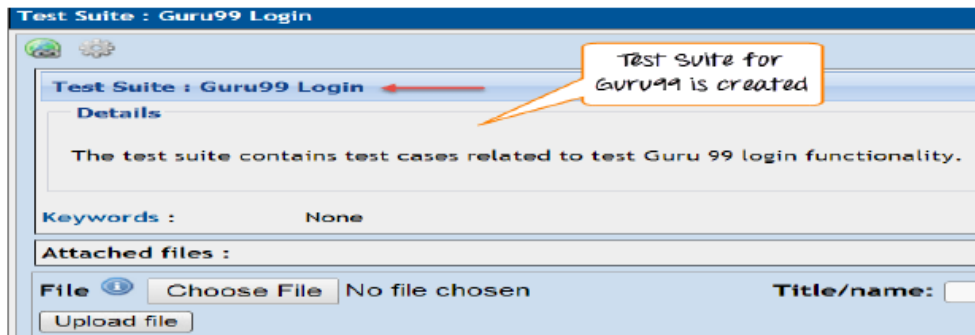


Step 4: Fill-up all the details for test-suite and click on save it tab.

1. Enter the test suite name
2. Enter the details about your test suite
3. Click on save button to save the details of test-suite



You can see test suite for Guru 99 is created

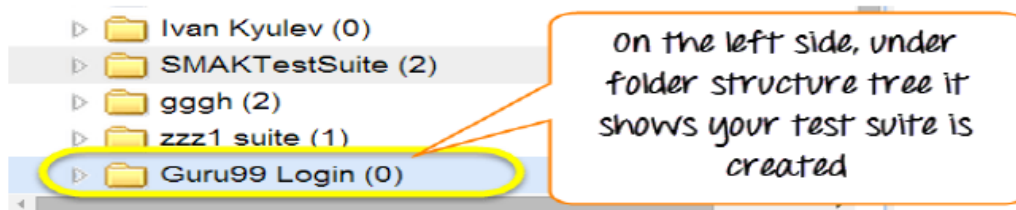


Your test suite appears on the left side of the panel under folder structure tree

### Creating a Testcase

Testcase holds a sequence of test steps to test a specific scenario with expected result. Below steps will explain how to create a test-case along with test steps.

Step 1: Click on the test suite folder on the left side of the panel under folder tree structure



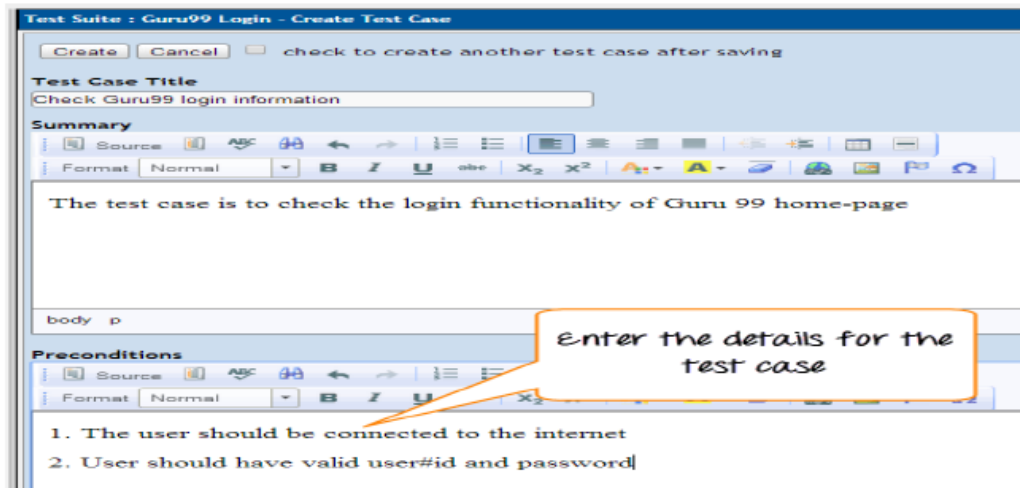
Step 2: Click on the setting icon in the right side panel. List of test case operations will be displayed on the right side panel



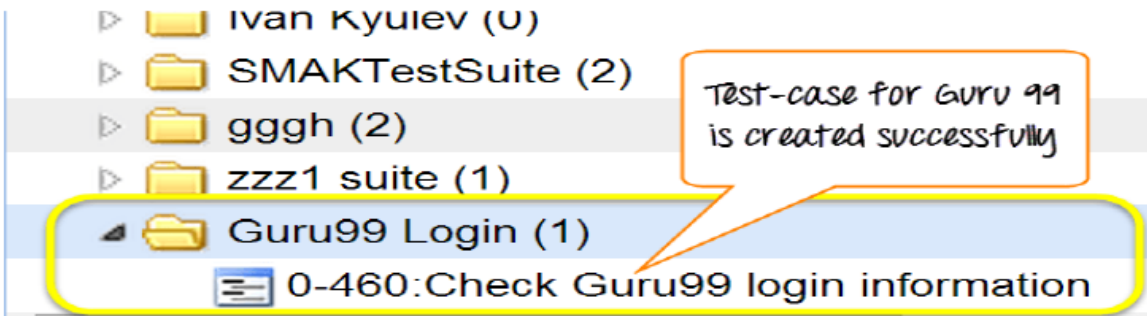
Step 3: New window will open, to create test cases click on create button in test-case operations



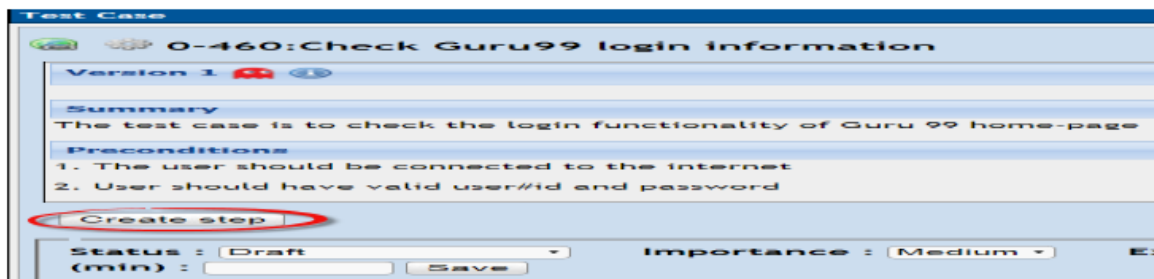
Step 4: Enter the details in the test case specification page



Step 5: After entering the details, click on "create" button to save the details. The test-case for Guru99 is created successfully

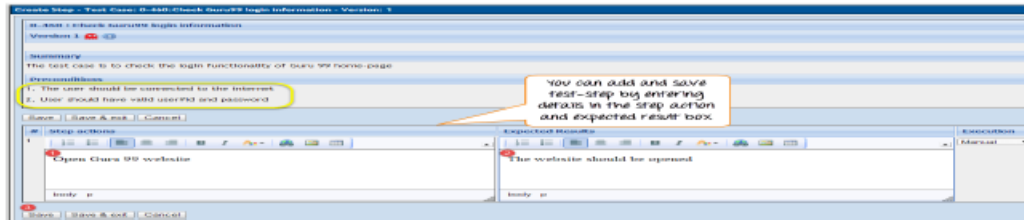


Step 6: Click on test-case from the folder as shown above, it will open a window. Click on "create steps" button in test case. It will open a test case step editor



Step 7) It will open another window on the same page, in that window you have to enter the following details

1. Enter the step-action for your test case
2. Enter the details about the step action
3. Click save it and add another step action OR click save and exit tab if there is no more test step to add



Step 8) Once you save and exit the test step, it will appear like this

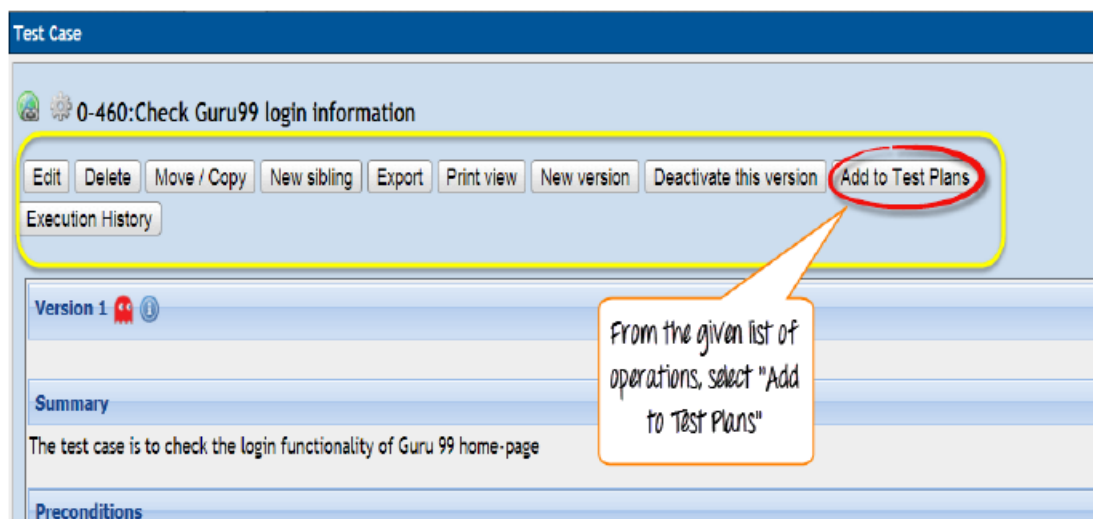
| Version 1                                                              |                                         |                                                    |           |                               |
|------------------------------------------------------------------------|-----------------------------------------|----------------------------------------------------|-----------|-------------------------------|
| Summary                                                                |                                         |                                                    |           |                               |
| The test case is to check the login functionality of Guru 99 home-page |                                         |                                                    |           |                               |
| Preconditions                                                          |                                         |                                                    |           |                               |
| 1. The user should be connected to the internet                        |                                         |                                                    |           |                               |
| 2. User should have valid userID and password                          |                                         |                                                    |           |                               |
| #                                                                      | Step actions                            | Expected Results                                   | Execution |                               |
| 1                                                                      | Open Guru 99 website                    | The website should be opened                       | Manual    | <span>✗</span> <span>✓</span> |
| 2                                                                      | Enter username in the username textbox  | Text-box should accept the entered data            | Manual    | <span>✗</span> <span>✓</span> |
| 3                                                                      | Enter password in the password text-box | Text-box should accept the entered data            | Manual    | <span>✗</span> <span>✓</span> |
| 4                                                                      | Click on "sign in" button               | Login should success and navigate to the home page | Manual    | <span>✗</span> <span>✓</span> |

### Assigning test case to test plan

For test case to get execute, it should be assign to test plan. Here we will see how we can assign a test-case to test plan.

Step 1) Click on the setting icon on the test panel. It will show the list of operations.

Step 2) Click on "Add to Test Plans"



Step 3)

New window will open, search your project "Guru99"

1. Mark the check box against your test plan
2. Click on add button

This will add your test case to your Test Plan.

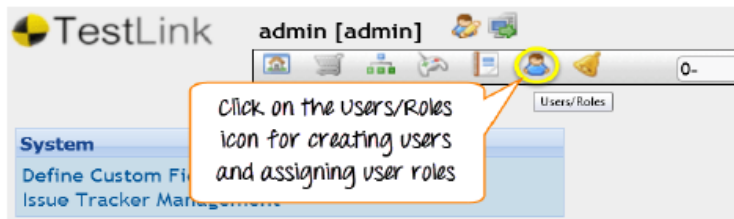
### Creating Users and Assigning Roles in TestLink

Testlink provides User management and authorization features.

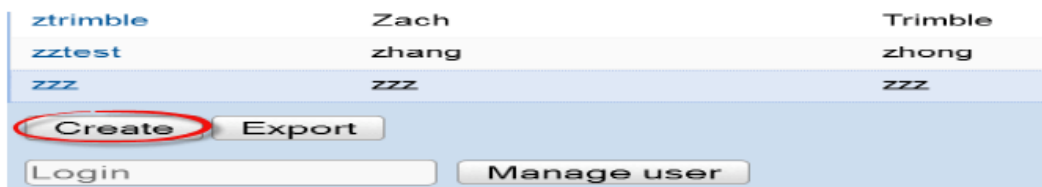
Below is list of default roles in Testlink and their rights -

| Role           | Test Cases     | Test Metrics   |
|----------------|----------------|----------------|
| Guest          | View           | View           |
| Tester         | Execute        | View           |
| Senior Tester  | Edit & Execute | View           |
| Leader & Admin | Edit & Execute | Edit & Execute |

Step 1: From the Testlinks home-page, click on users/roles icon from the navigation bar



Step 2: Click Create



Step 3: Fill out all the users details and click the "Save" button

User Management - Account Settings

**New User** | View Users | View roles | Assign Test Project roles | Assign Test Plan roles

User details

|                       |                                     |
|-----------------------|-------------------------------------|
| Login                 | Jamesguru                           |
| First Name            | Jamesguru                           |
| Last Name             | Bryan                               |
| Password              | *****                               |
| Email                 | jamesguru@gmail.com                 |
| Role                  | Senior Test Engineer                |
| Locale                | English (wide/UK)                   |
| Authentication method | Default(DB)                         |
| Active                | <input checked="" type="checkbox"/> |

Fill out all the details about the user and click on save button

**Save** Cancel

Here in the list we can see the users have been created

|           |           |              |                               |                      |
|-----------|-----------|--------------|-------------------------------|----------------------|
| jafo      | João      |              | joaofc@gmail.com              | leader               |
| james     | james     |              | abc@163.com                   | leader               |
| Jamesguru | Jamesguru | Bryan        | jamesguru@gmail.com           | Senior Test Engineer |
| jan       | jan       | sivaprakasam | jan.sivaprakasam@cancer.or... | leader               |

Step 4: Allotting test project role to the user,

1. Click on "Assign Test Project Roles" tab
2. Choose the project name
3. Select the users role from the drop down

User Management - Assign roles

View Users | View roles | **Assign Test Project roles** | Assign Test Plan roles

Test Project **Guru 99 -** **Change**

Set roles to **Senior Test Engineer** **Do**

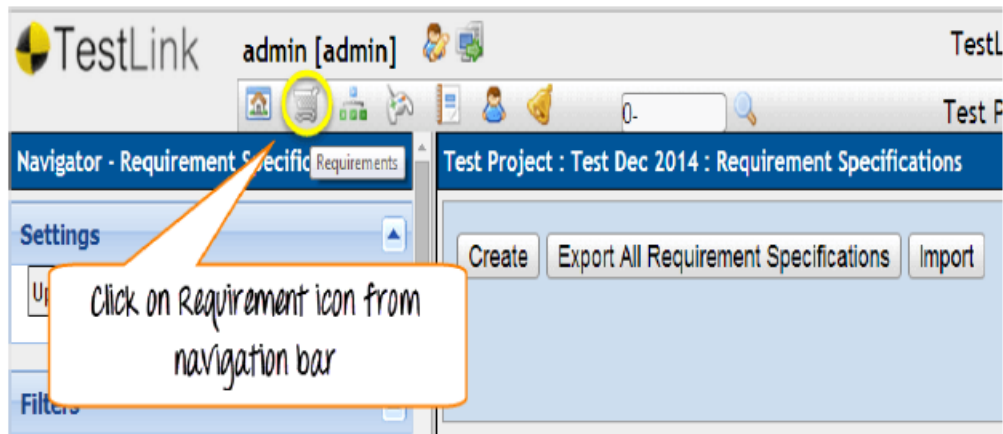
Show 25 entries

| User                        | Test Project Role (Guru 99 -)    |
|-----------------------------|----------------------------------|
| Jamesguru (Jamesguru Bryan) | <inherited> Senior Test Engineer |

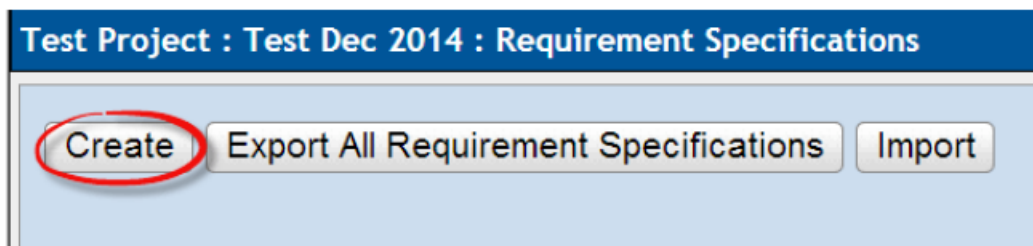
We assigned our test project "Guru99" to senior test engineer- Jamesguru

## Writing Requirements:

Step 1: From the navigation bar select the "Requirements Link", it opens the Requirement page.



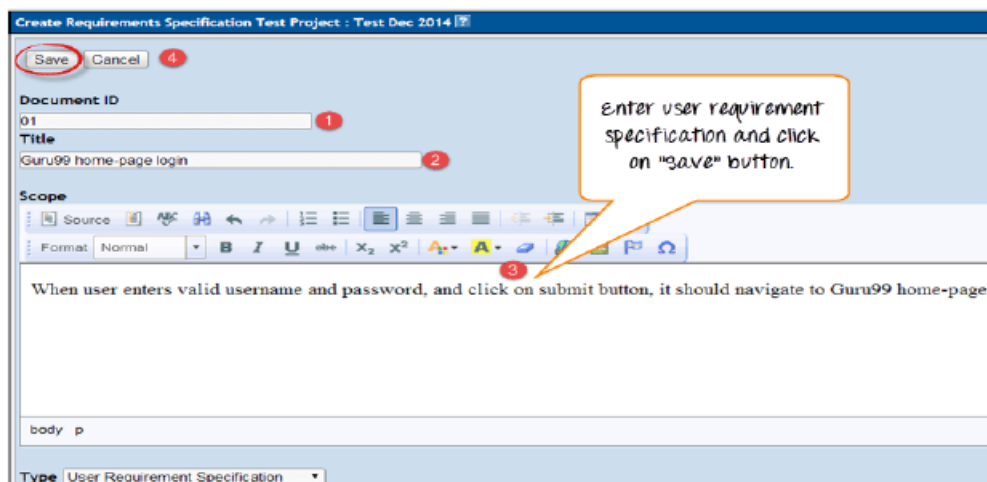
Step 2: From the requirement page, on the right side of the panel click on "create" button



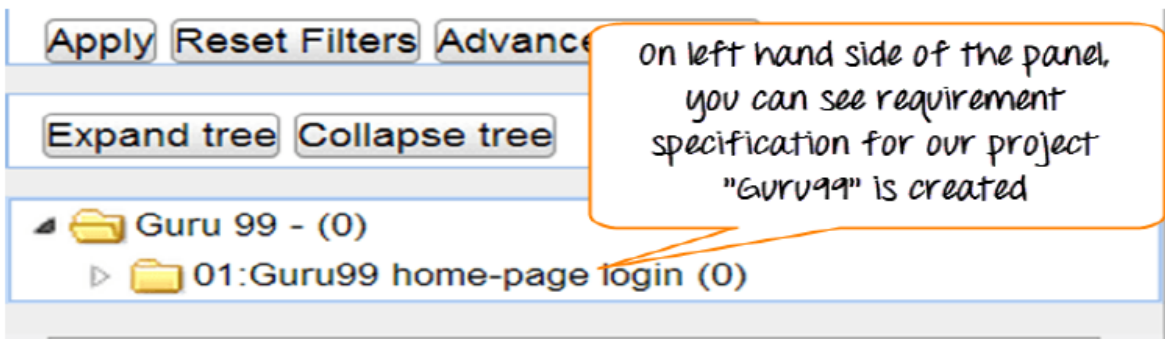
Step 3: A new window will open, enter all the details like

1. Document ID
2. Title name
3. Requirement description
4. And Click "Save" button

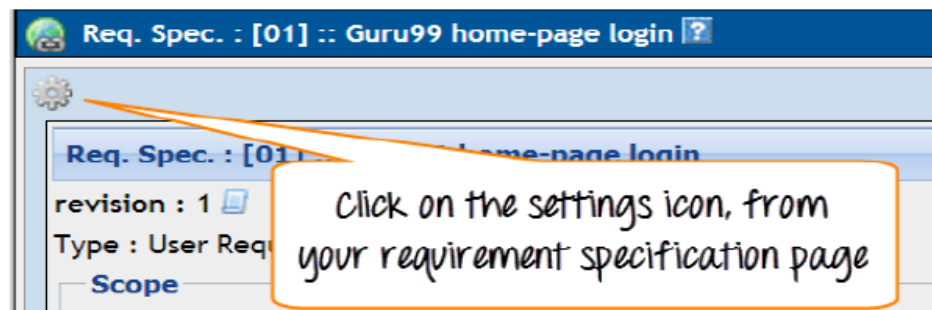
For the type, you can choose the option from the drop-down- here we chose "User Requirement Specification"



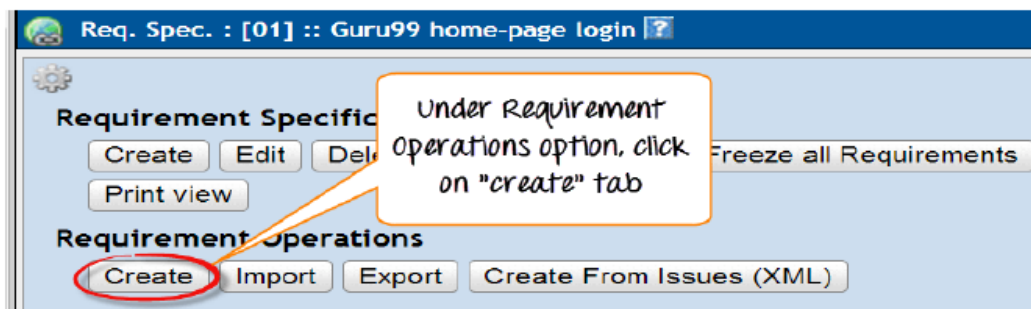
Step 4: It should create Requirement specification and displayed on the left side panel under project "Guru99".



Step 5: Select the setting button from requirements specification home-page. It will open another window.



Step 5: Click "Create" tab under Requirement Operations.

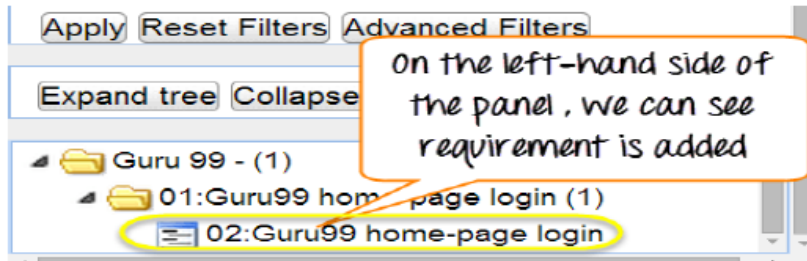


Step 6: Fill out all the specified details and click the "Save" button

1. Enter the document ID
2. Enter the title name
3. Enter the description
4. Enter the status-whether it's in draft, rework, review, not testable, etc. Here we chose valid
5. Enter the type – user interface, non-functional, informational, feature, etc. Here we chose use case
6. Enter the number of test cases needed
7. Enter "Save" button at the end

Note: To add more requirements you can mark the check-box and click save button

On the left side of the panel, we can see that requirement is added.



### Assigning requirement to test-cases

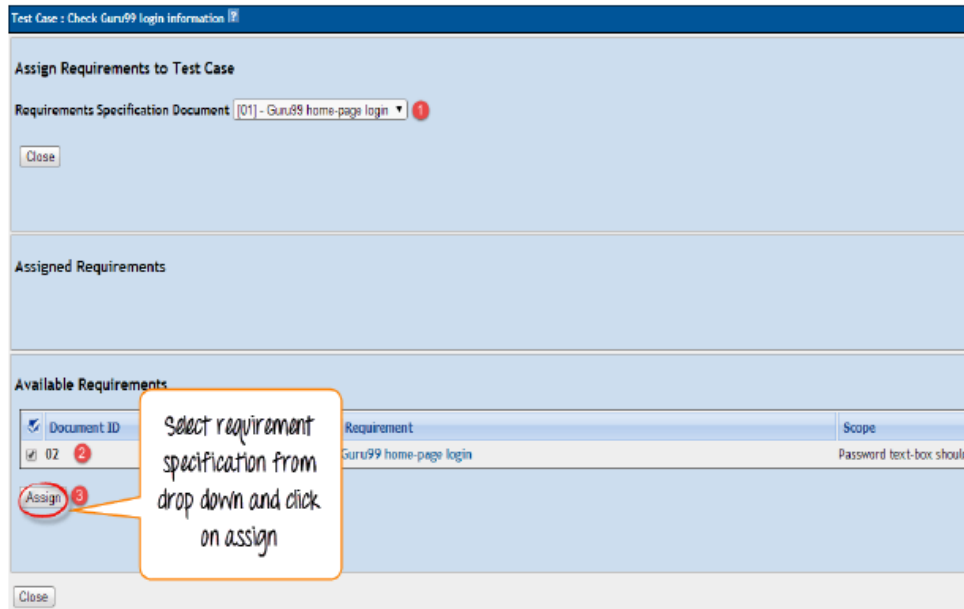
In Testlink, Requirement can be connected to test cases. It is very crucial feature in order to track test coverage based on requirements. In test reports, you can verify which requirements are not covered and act on them to append in test suites for maximum test coverage

Step 1: From test specification section open any single test case and click on requirement icon

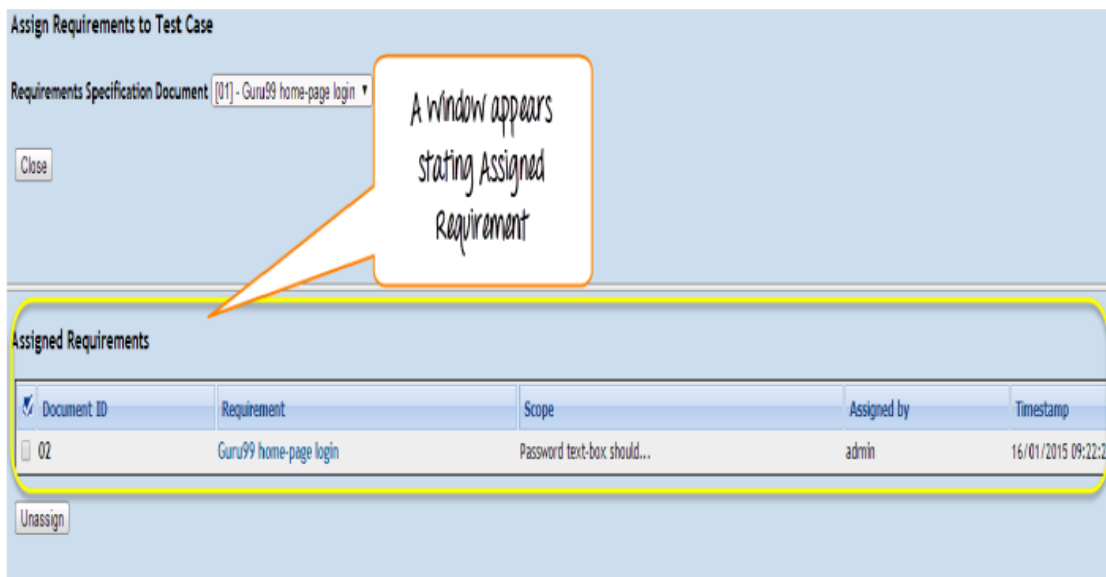
| # | Step actions                            | Expected Results                                   |
|---|-----------------------------------------|----------------------------------------------------|
| 1 | Open Guru 99 website                    | The website should be opened                       |
| 2 | Enter username in the username textbox  | Text-box should accept the entered data            |
| 3 | Enter password in the password text-box | Text-box should accept the entered data            |
| 4 | Click on "sign in" button               | Login should success and navigate to the home page |

**Step 2:** To assign requirements specification to test case you have to follow the following steps

1. Scroll the drop down box to select the requirements specification
2. Mark the requirement check box
3. Click on "assign" tab



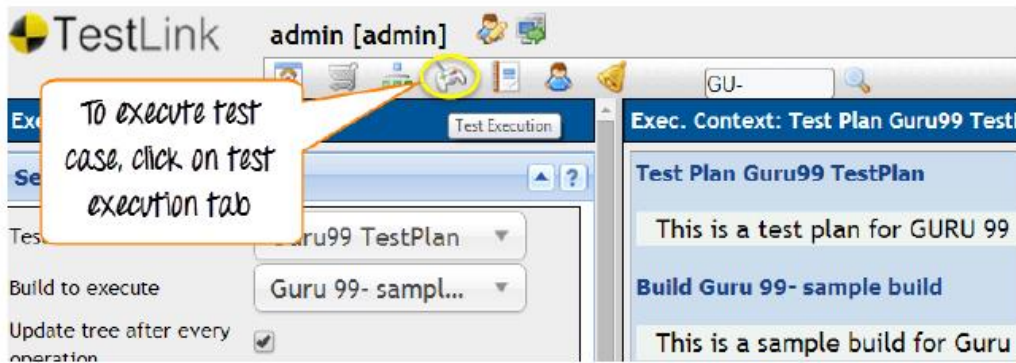
After clicking on "assign" tab, a window will appear stating "Assigned Requirement."



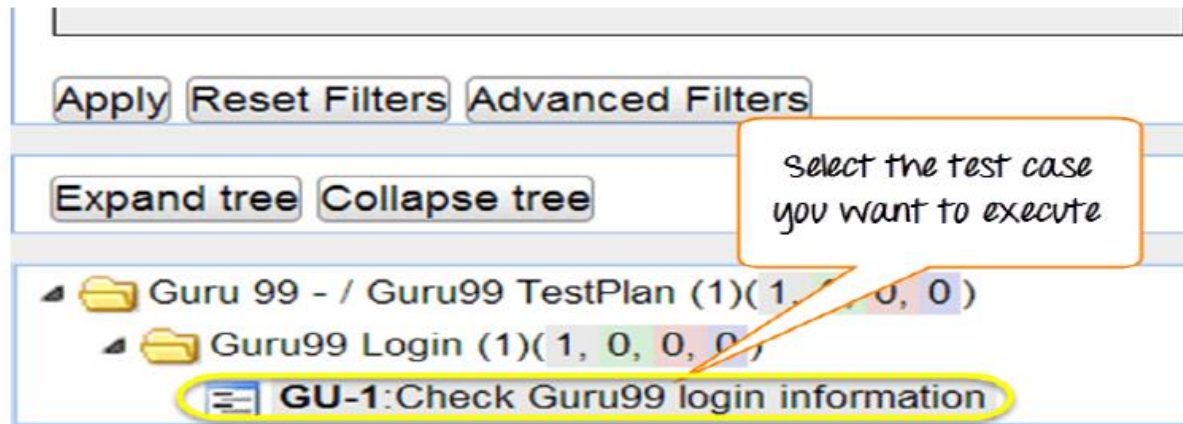
### Executing a test case

In TestLink, we can run a test case and change execution status of a test case. Status of a test-case can be set to "blocked", "Passed", or "failed". Initially, it will be in "not run" status but once you have updated it, it cannot be altered to "not run" status again.

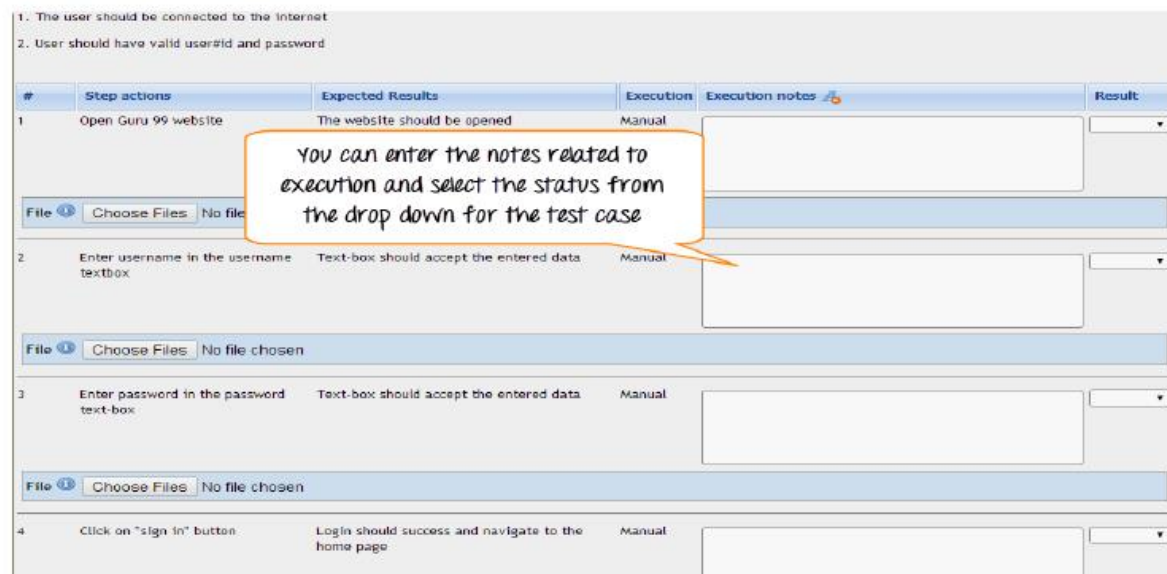
**Step 1:** From the navigation bar click on the "Test Execution" link. It will direct you to the Test Execution Panel.



Step 2: Pick the Test case you want to run from the left side panel



Step 3: Once you have selected the test cases, it will open a window.



Step 4: Follow the following steps

1. Enter the notes related to test case executed
2. Select its status

Note about the test case and its status is selected from drop-down

| Step | Action                                 | Expected Result                         | Actual Result                                                          | Status |
|------|----------------------------------------|-----------------------------------------|------------------------------------------------------------------------|--------|
| 2    | Enter username in the username textbox | Text-box should accept the entered data | When user name is entered in the username, it accepts the entered data | Passed |

**Step 5:** On the same page, you have to fill similar detail about the execution of test-case. Fill the details, select the status and then click on "save execution".

A window will appear showing latest execution of your test case

Test Results on Build Guru 99- sample build

Print Show complete execution history Import Results (XML)

Test Plan Guru99 TestPlan

Build Guru 99- sample build

Test Suite : Guru99 Login/

Latest execution (any build)

Date : 16/01/2015 10:35:06 - Tested by : admin - Build : Guru 99- sample build - Status : Passed

Latest execution (current build) - Build : Guru 99- sample build

| Date                | Build                 | Tested by | Status | Exec (min) | Version | Run mode |
|---------------------|-----------------------|-----------|--------|------------|---------|----------|
| 16/01/2015 10:35:06 | Guru 99- sample build | admin     | Passed | 1          |         |          |

Notes

Test executed successfully in Firefox 12, in windows 8 platform

## Generating Test Reports

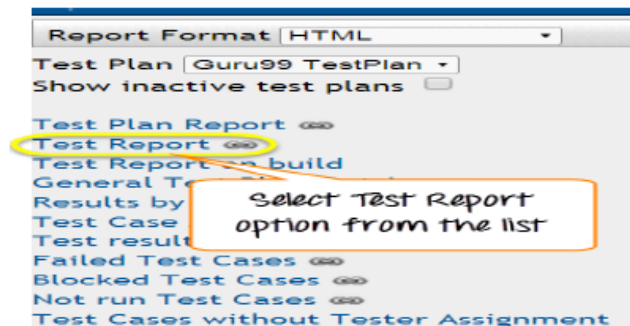
Test link supports various test report formats like

- HTML
- MS Word
- MS excel
- OpenOffice Writer
- OpenOffice calc

**Step 1:** From the navigation bar, click on Test Reports option

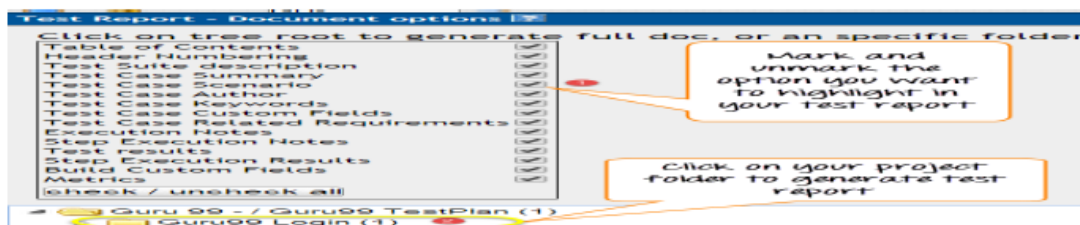


Step 2: From the left side panel, select "Test Report" link



Step 3: To generate a report follow the following steps

1. Mark and unmark the option you want to highlight in your test report
2. click on your project folder



The test report will look like this

**1.1. Test Suite : Guru99 Login**

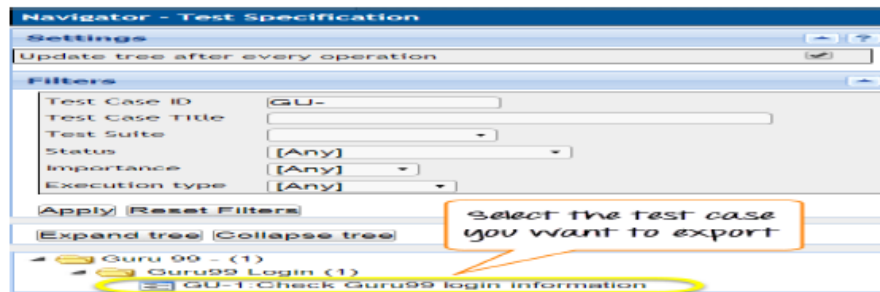
The test suite contains test cases related to test Guru 99 login functionality

| Test Case ID                                                                                                  | Test Case Description                | Author | Expected Results                                   | Execution Notes                                                        | Execution Status |
|---------------------------------------------------------------------------------------------------------------|--------------------------------------|--------|----------------------------------------------------|------------------------------------------------------------------------|------------------|
| TC001                                                                                                         | Check Guru99 login information       | admin  |                                                    |                                                                        |                  |
| <p>1. The user should be connected to the internet</p> <p>2. User should have valid username and password</p> |                                      |        |                                                    |                                                                        |                  |
| 1                                                                                                             | Open Guru 99 website                 |        | The website should be opened                       |                                                                        |                  |
| 2                                                                                                             | Enter username in the username field |        | Text box should accept the entered data            | When user name is entered in the username, it accepts the entered data | Passed           |
| 3                                                                                                             | Enter password in the password field |        | Text box should accept the entered data            |                                                                        |                  |
| 4                                                                                                             | Click on "Sign in" button            |        | Login should succeed and navigate to the home page |                                                                        |                  |
| <p>Execution Date: _____</p> <p>Manual</p>                                                                    |                                      |        |                                                    |                                                                        |                  |

## Export Test case/ Test Suite

Testlink provides the features to export test projects/test suites in your Testlink and then you can import them into another Testlink project on different server or system. In order to do that you have to follow the following step

Step 1: Choose the test case you want to export in the Test specification page

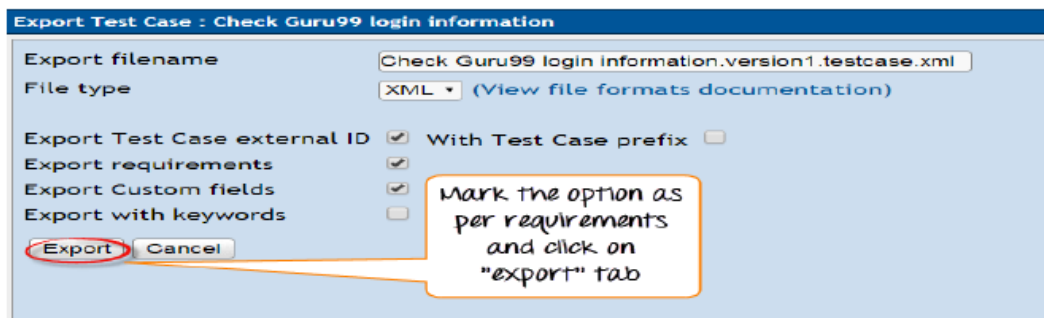


Step 2: Now on the right-hand side of the panel click on the  setting icon, it will display all the operations that can be performed on the test case.

Step 3: Click the "export" button



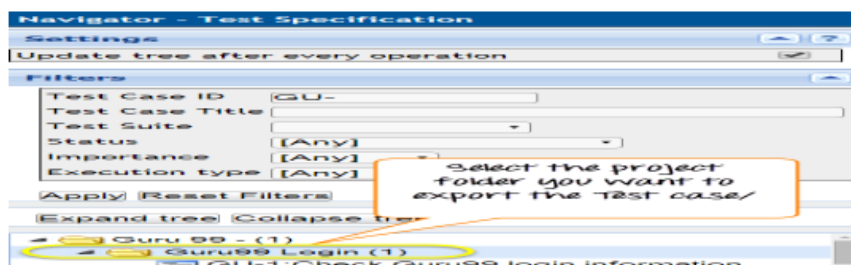
Step 4: It will open another window, mark the option as per requirement and click on the export tab



Following XML is generated

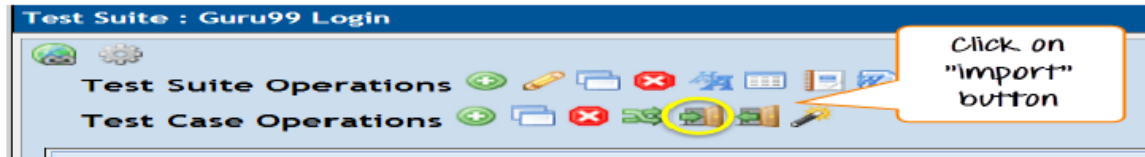
### Importing Test case/ Test suite

Step 1: Select the Test suite folder inside which you want to import the test case



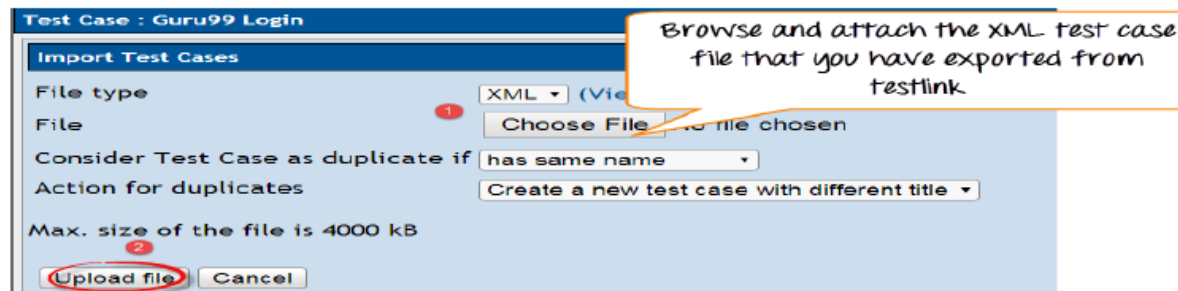
Step 2: Click on the setting icon  on the right hand-side of the panel, it will display all the operations that can be executed on the test suite/test case

Step 3: Click on the import button in the test case operations list as

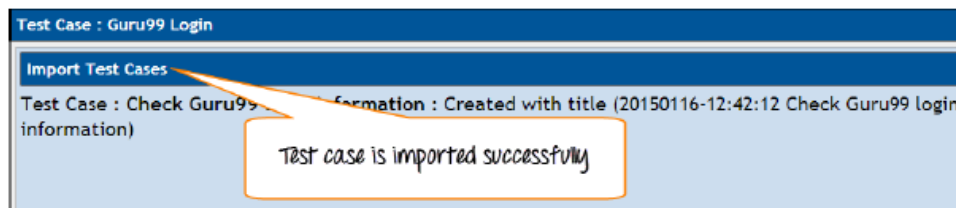


Step 4: Browse and attach the xml test case file that you have exported from test link and click on upload button.

1. Use the browse option to attach the XML test case file that you have exported from testlink
2. Click on upload file



When you upload a file, it will open window stating import test cases



Step 5: Test case will be uploaded and displayed on the right-hand side of the panel

